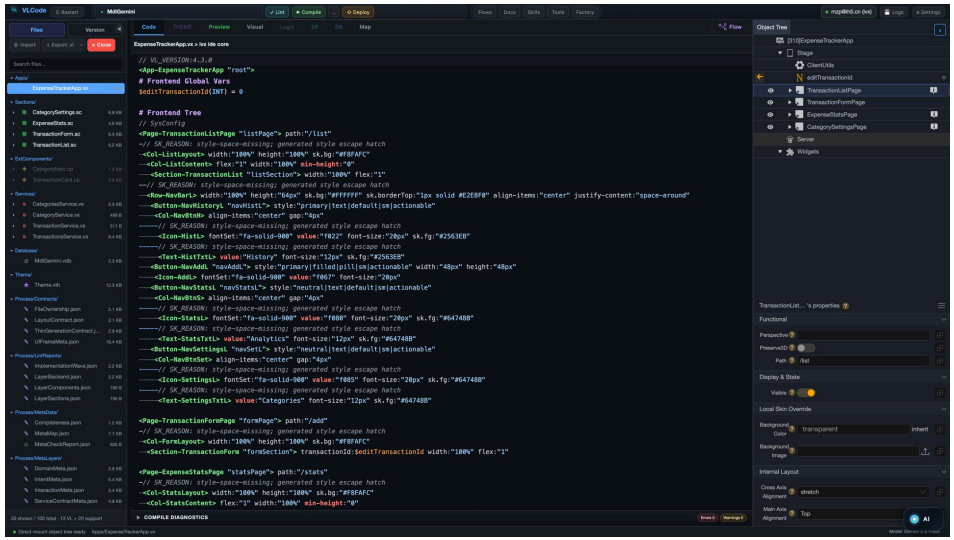
VLCODE

VLCode Complete Tutorial

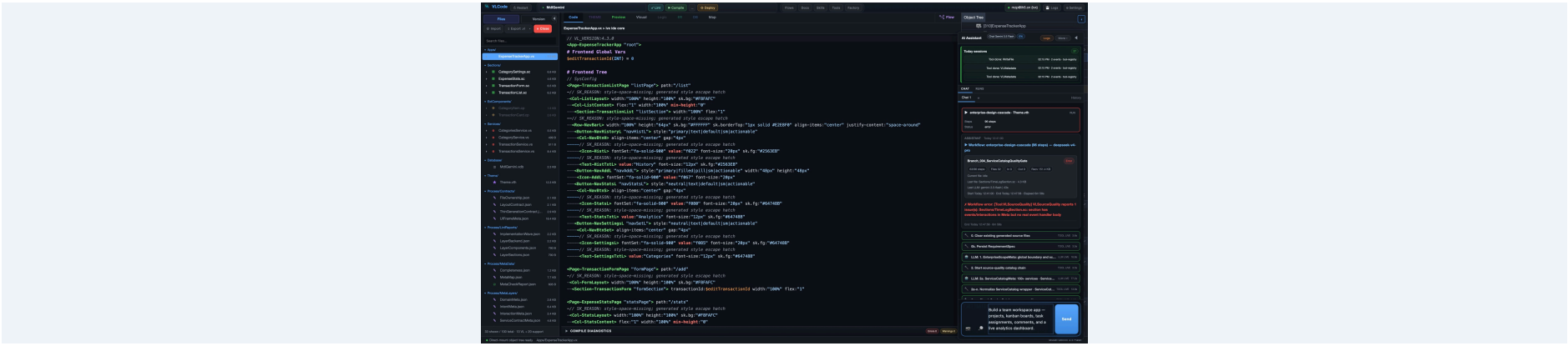
From Concepts And Setup To Daily Operations And Complete Case Studies

- For product managers, implementation consultants, developers, and delivery teams.
- It follows Concept → Operation → Validation → Case Study, so every page becomes a practical exercise.



VLCode shell with workspace, editor, and assistant panels

How To Use This Tutorial



A concise project brief in the assistant panel

LEARNING ORDER

- Read concepts first, follow interface operations next, finish with case studies.
- Do not skip validation chapters; Lint/Compile and the run ledger are central.

PRACTICE METHOD

- Practice each concept with one small scoped operation.
- After each operation, save errors, screenshots, or logs so the run is reproducible.

DELIVERY STANDARD

- Explain the project / workflow structure.
- Run a full flow end to end.
- Read failure logs and make a minimal fix.

Roadmap I

1. PART I: CONCEPTS AND MENTAL MODELS

- What VLCode Is
- AI-Native, Not AI-Assisted
- Why Ordinary Languages Misfit AI
- From Point Intelligence To Process Intelligence
- ...and more pages

2. PART II: INSTALLATION, STARTUP, AND WORKSPACE

- Choose Desktop, Browser, Or Source Run
- Get The Installer From The Release Page
- Start The Web IDE From Source
- Port 4000 And Fixed-Port Discipline
- ...and more pages

3. PART III: INTERFACE TOUR AND DAILY OPERATIONS

- Left File Tree
- Center Code Editor
- Visual Canvas
- Object Tree And Selection Sync
- ...and more pages

4. PART IV: VL PROJECT AND LANGUAGE BASICS

- VL File Types Overview
- VL_VERSION Must Be The First Line
- Indentation Uses Consecutive Hyphens
- App Files .vx
- ...and more pages

Roadmap II

5. PART V: AI CHAT AND WORKFLOW CONTROL

- Write Requirements As Executable Goals
- Choose The Right Workflow
- MetaDirect For Small Projects
- Meta Thin Fast Generation
- ...and more pages

6. PART VI: BUILD THE FIRST APP FROM SCRATCH

- Lab Goal: Team Task Board
- Prepare The TeamPM Workspace
- Write The First Requirement
- Select MetaDirect Or Meta Thin Fast
- ...and more pages

7. PART VII: INCREMENTAL CHANGES AND PROFESSIONAL WORKFLOWS

- Add A Page With add-page
- Add A Service With add-service
- Add A Feature With feature-adjust
- Modify Data With data-adjust
- ...and more pages

8. PART VIII: VALIDATION, DEBUGGING, RELEASE, AND DELIVERY

- Validation Order: Lint → Compile → Preview
- When To Stop Recompiling
- Read A Lint Report
- Read A Compile Report
- ...and more pages

Roadmap III

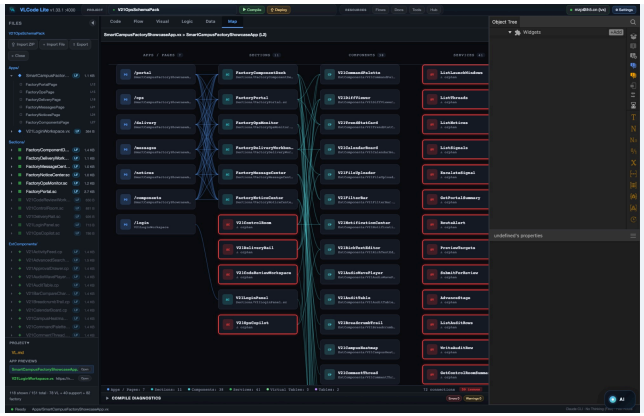
9. PART IX: COMPLETE CASE STUDIES

- Case 1: Todo Application
- Case 1: Requirement Prompt
- Case 1: Data Design
- Case 1: Pages And Interactions
- ...and more pages

10. PART X: APPENDICES AND QUICK REFERENCE

- Common Commands
- Common Workflows
- Common Directories
- AI Prompt Templates
- ...and more pages

Core Glossary



Map mode for application structure

Workspace

The local VL project root currently open, normally under ~/Documents/VLProjects/<ProjectName>/.

VL source files

.vx app entry, .sc section, .cp component, .vs service, .vdb database, .vth theme.

DAG / Flow

Work as nodes and directed edges; nodes create or execute, edges preserve order, dependency, and traceability.

AI Chat

The intent surface and workflow control console; final summaries and human decisions appear here.

Interactive Card

A chat card that gates input or a decision — ask_user, review/approval, the Deploy binding card.

Detail Log

The full run ledger: node progress, tool calls, file writes, validation results, and errors.

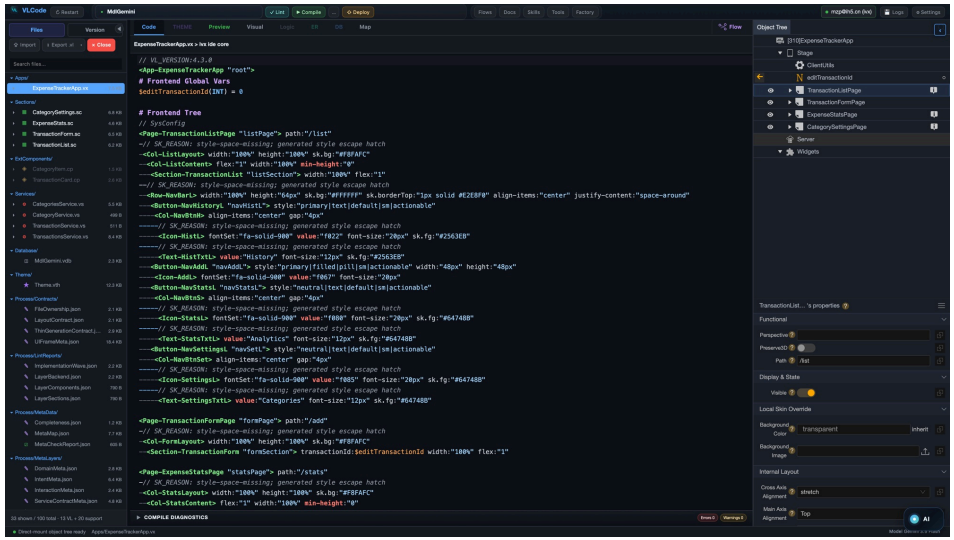
Lint / Compile

ProjectMeta

CHAPTER

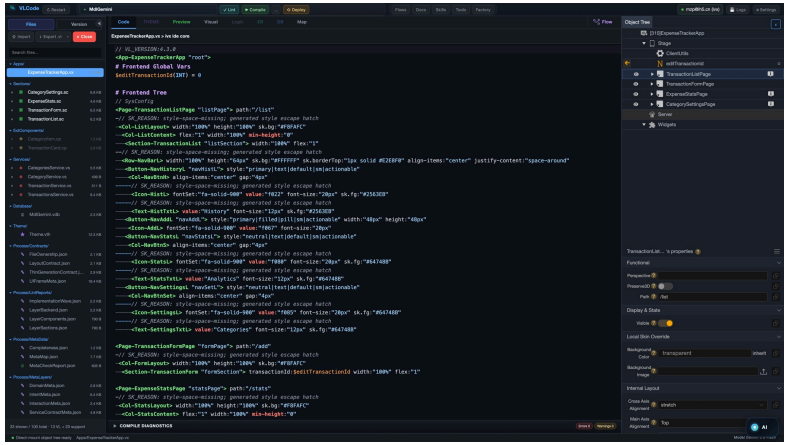
Part I: Concepts And Mental Models

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



VLCode shell with workspace, editor, and assistant panels

What VLCode Is



VLCode shell with workspace, editor, and assistant panels

CORE IDEA

VLCode is a local-first IDE positioned as the "base for building VL enterprise apps" — you use it to generate and repair VL apps fast. It is not a plain code editor: the core interaction is AI Chat plus a set of workflows (codegen, incremental change, repair), the output is VL source files (.vx/.sc/.cp/.wc/.vs/.vdb/.vth), and projects live on your local disk. It is one of two products in the same ecosystem as VL Agent, connected by the .vflow connector bundle and sharing one visuallogic.ai account.

OPERATIONS

1. Download the .dmg/.zip desktop app from releases.visuallogic.ai, or run npm run dev from source to launch the browser Web IDE at http://localhost:4000
2. Confirm the current LLM provider/model, compile status, and port in the bottom status bar
3. Type a natural-language requirement into AI Chat and let a workflow generate the VL app
4. Browse the generated Apps/Sections/Services VL source files in the left file tree
5. Validate the app with the Lint → Compile → Preview chain

EXAMPLE / EXERCISE

You start by typing a requirement into AI Chat, for example:

```
Build me a TeamPM team task board: tasks have a title, assignee, and status (Todo/In Progress/Done); the home page shows three columns by status, and I can create and drag tasks.
```

VLCode picks a codegen workflow and emits a set of VL files under ~/Documents/VLProjects/TeamPM/.

PITFALL

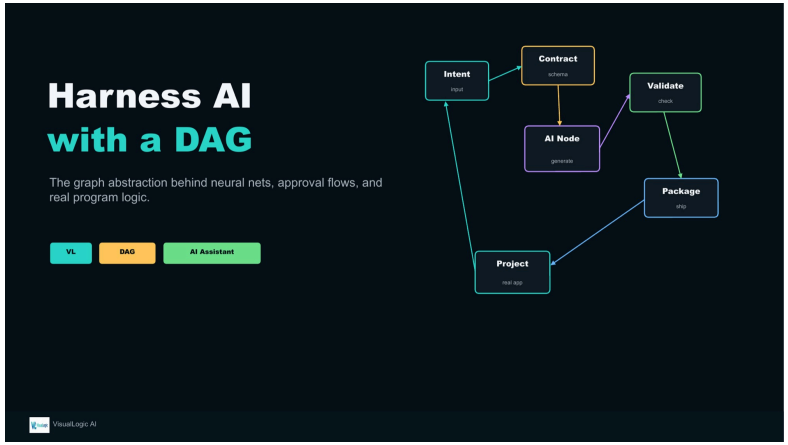
Treating VLCode as an ordinary text editor and trying to hand-type an entire app is the most common mistake. Its strength is "AI-Chat-driven workflow generation plus reviewing through the mode panels" — manual editing is for small touch-ups only.

CHECKPOINT

You can state that VLCode is a local-first IDE for building VL enterprise apps, and point out that it drives workflow generation of VL source through AI Chat rather than writing every line

PART I: CONCEPTS AND MENTAL MODELS

AI-Native, Not AI-Assisted



DAG harness: execution overview

CORE IDEA

"AI-assisted" treats AI as autocomplete or a sidebar plugin — the human writes the mainline and AI helps. VLCode is "AI-native": AI Chat itself is the primary interaction entry and the workflow control console where final summaries and human decisions happen. The entire generate/modify/repair pipeline revolves around workflows and ProjectMeta; AI is the executing agent of the process while humans gate at key nodes, not the other way around.

OPERATIONS

1. Use AI Chat as the main entry: issue every requirement, change, and debug as a natural-language command here
2. Let workflows (not line-by-line completion) carry generation and repair — e.g. codegen, add-page, compile-fix
3. Make human decisions on Interactive Cards (blocking review-card / form-card / picker-card, alongside the ask_user / human_gate chat events) instead of editing all the code yourself
4. Track node progress, file writes, and tool I/O in the right-side Detail Log
5. Switch to Code mode for manual edits only when doing small touch-ups

EXAMPLE / EXERCISE

Contrast the two mindsets. AI-assisted: you write half a line and let autocomplete guess the rest. AI-native (VLCode):

```
Add a "My Tasks" page to TeamPM that only shows tasks assigned to the current user, and add an entry in the sidebar.
```

You state the goal, the add-page workflow generates, writes files, and runs validation, and you confirm on a card.

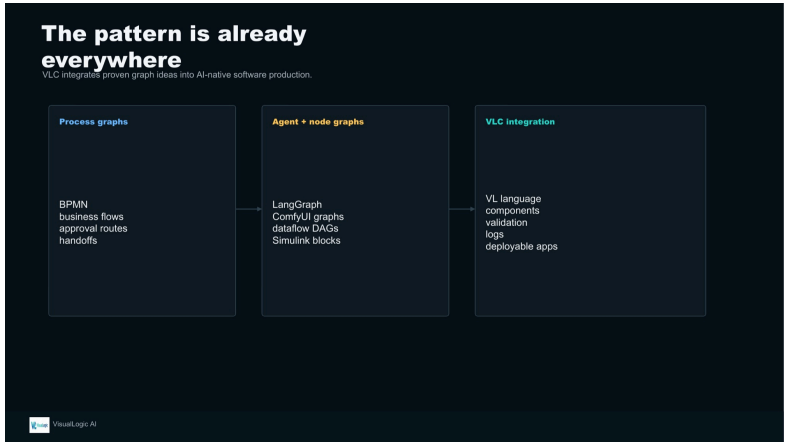
PITFALL

Assuming you must hand-write every file correctly like in a traditional IDE, which bypasses the workflow and card-decision mechanism — slower and prone to breaking ProjectMeta consistency. Let AI run the process while you make decisions.

CHECKPOINT

You can distinguish "AI-assisted" (human-led, AI completes) from VLCode's "AI-native" (AI Chat-driven workflows, humans gate at the card layer) and name AI Chat as the primary

Why Ordinary Languages Misfit AI



DAG harness: node reasoning

CORE IDEA

Ordinary programming languages are designed for humans to hand-write and for compilers to parse character-by-character; they fit AI generation poorly — arbitrary whitespace indentation, implicit dependencies, and cross-file global state make it hard for AI to emit a globally consistent, machine-verifiable artifact in one pass. VL does the opposite: it uses consecutive hyphens (- / -- / ---) for hierarchy instead of spaces, forces VL_VERSION as the first line of every source file, and maintains a machine-readable ProjectMeta so generation and repair revolve around a structured map.

OPERATIONS

1. Remember VL uses consecutive hyphens for hierarchy — never mix spaces or tabs
2. Make sure every VL file's first line is VL_VERSION, with nothing before it
3. Think of project structure as ProjectMeta, a machine-readable map, not a pile of loose files
4. After generation, run Lint first for a low-cost static check that catches structural problems
5. When you hit a "strange" error, suspect indentation or VL_VERSION placement first

EXAMPLE / EXERCISE

VL indents with hyphens; mixing spaces produces "strange" errors. Correct form forces the version on line one:

```
// VL_VERSION:4.3.1
- Section TaskBoard
-- Column Todo
--- TaskCard
```

If you delete the first-line VL_VERSION or replace `--` with two spaces, parsing fails.

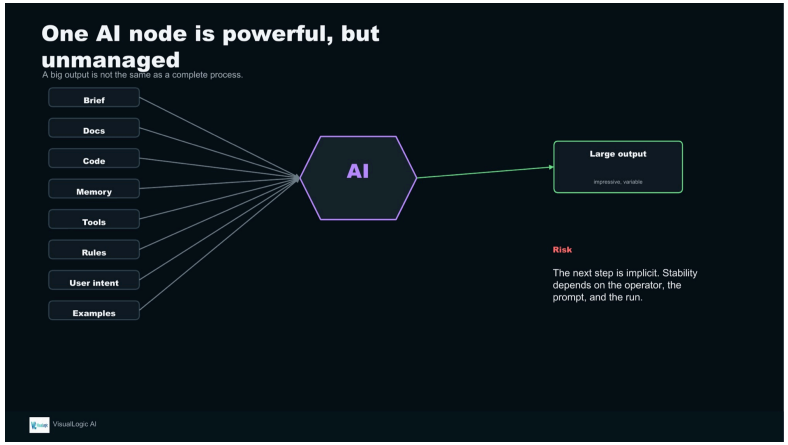
PITFALL

Bringing ordinary-language instincts — relying on space indentation or putting a comment/blank line above VL_VERSION — triggers hard-to-locate parse errors. Always hyphen-indent, and always keep VL_VERSION on line one.

CHECKPOINT

PART I: CONCEPTS AND MENTAL MODELS

From Point Intelligence To Process Intelligence



DAG harness: validation path

CORE IDEA

"Point intelligence" is a one-shot Q&A: you ask, AI answers, with no traceable process, no intermediate artifacts, and no way to review what went wrong. VLCode pursues "process intelligence" — generation is broken into a workflow (DAG) with ordered nodes, validation, and pause/rerun support, where each step's progress, file writes, and tool I/O land in the Detail Log run ledger. AI's work shifts from a single shot to an observable, intervenable, verifiable process.

OPERATIONS

1. Use workflows (codegen/add-page/compile-fix) instead of single-turn questions for any sizable task
2. After commanding in AI Chat, track each node's start/done/error in the Detail Log
3. Make in-process human decisions on the blocking review-card (alongside the human_gate chat event)
4. Pause, resume, or rerun from a specific step when needed, instead of re-asking from scratch
5. Keep final summaries and decisions in AI Chat, and step-by-step detail in the Detail Log

EXAMPLE / EXERCISE

Point: "write a task board" → a blob of code; if wrong, start over. Process (VLCode):

```
Generate the TeamPM task board with a workflow; after the data tables are created, let me confirm the fields before continuing to the pages.
```

The workflow pops a review-card at the data-table node, waits for your confirmation, then continues — every step recorded in the Detail Log.

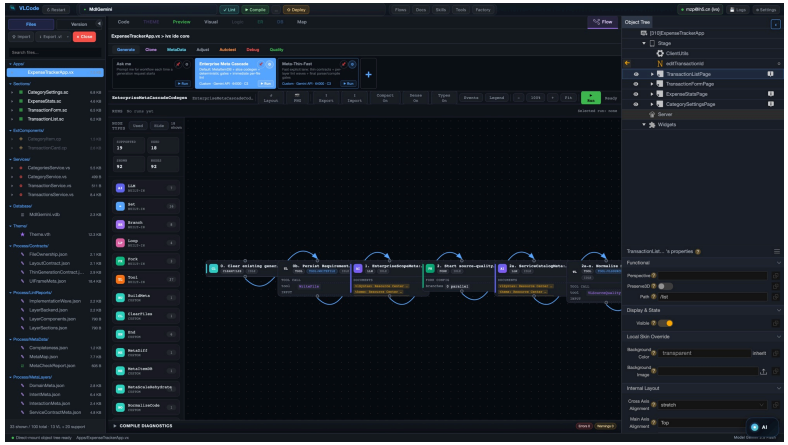
PITFALL

Cramming a sizable task into a single conversational turn and hoping for one perfect answer forfeits the observability and intervenability of the process; if it goes wrong you can only re-ask everything. Use workflows to split the task into verifiable nodes.

CHECKPOINT

You can state the difference between point intelligence (single-turn Q&A) and process intelligence (a workflow with nodes/validation/pause-rerun) and identify the Detail Log as VLCode's run ledger.

How A DAG Tames AI



Flow mode: a DAG workflow shown as executable nodes

CORE IDEA

A DAG (directed acyclic graph) breaks a big goal into nodes with dependency ordering, forcing AI to proceed step-by-step with each step verifiable, instead of free-styling without bounds. In VLCode the Flow panel is the workflow's DAG graph; codegen and repair workflows execute as nodes, and run states settle reliably to done/error/aborted (terminal-state integrity). This constrains uncontrolled "free AI generation" into an observable, pausable, rerunnable ordered process.

OPERATIONS

1. View the current workflow's DAG node graph in the Flow panel
2. Trigger the workflow from AI Chat and let it execute in dependency order across nodes
3. Watch each node's progress and artifacts in the Detail Log
4. Pause, resume, or rerun from a specific step at a node when needed
5. Confirm the run settles into one of the done/error/aborted terminal states

EXAMPLE / EXERCISE

The DAG for board generation looks roughly like:

```
[parse requirement] -> [data tables .vdb] -> [services .vs] -> [pages .sc] -> [Lint check]
```

Each node must finish before the next begins; if the [services] node errors, the run reaches an error terminal state and you can rerun from just that node rather than the whole graph.

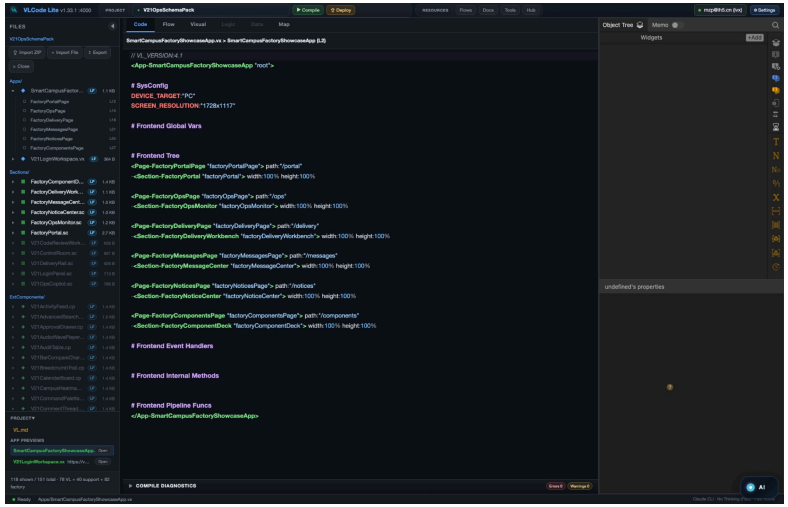
PITFALL

Treating the DAG as a decorative flowchart and missing how it pinpoints which node stalled. Its value is precisely localizing failure to a node and supporting rerun-from-there — don't ignore the Flow panel and terminal states.

CHECKPOINT

You can explain how a DAG constrains free generation into an ordered, verifiable node flow and note that runs converge to done/error/aborted and can rerun from a specific step.

VL As AI Intermediate Representation



Code mode for inspecting and editing VL source

CORE IDEA

VL acts as the intermediate representation (IR) between AI and the final product: AI does not emit runtime code or loose HTML directly but generates structured, machine-verifiable VL source (.vx/.sc/.cp/.wc/.vs/.vdb/.vth), which the platform's parsevl then compiles. VL's consecutive-hyphen hierarchy, mandatory first-line VL_VERSION (current VL_VERSION 4.3.1), and ProjectMeta make the output something AI can produce consistently, that Lint/Compile can strictly verify, and that humans can read and tweak in Code/Visual/Logic and other modes.

OPERATIONS

1. Understand that AI's output target is VL source files, not runtime code directly
2. Read the generated VL in Code mode and confirm the structure (Section/Component/Service/Data/Theme) is clear
3. Run lintPjt for a project-level static check, then parsevl for platform compilation
4. For semantic adjustments, review in Visual/Logic/Data mode rather than editing compiled output
5. Treat ProjectMeta as the shared source of truth for both generation and repair

EXAMPLE / EXERCISE

AI's output is readable VL, not a black box:

```
// VL_VERSION:4.3.1
- Service TaskService
-- method listTasks
-- method createTask
```

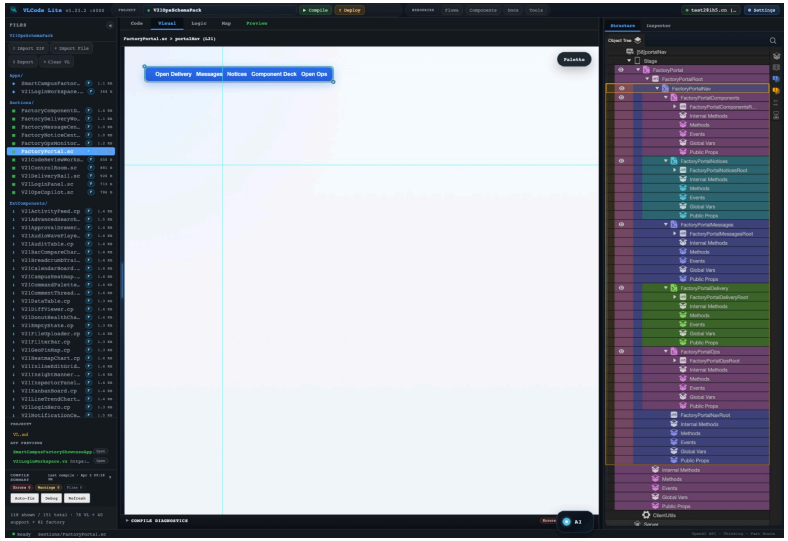
This .vs both compiles via parsevl and is directly readable and tweakable by you in DB/ER mode — VL serves as the verifiable middle layer between AI and the running result.

PITFALL

Trying to bypass VL and edit the compiled runtime output desyncs it from ProjectMeta and gets overwritten next time a workflow regenerates. Make changes at the VL-source IR layer.

CHECKPOINT

Component Model: Properties, Methods, Events



Visual panel with object tree and properties

CORE IDEA

A VL app is made of components, and a component has three operable faces: Properties (its state/appearance/data binding), Methods (callable behaviors defined in Service logic), and Events (actions triggered by user interaction or lifecycle). In VLCode these map to distinct modes: Visual mode shows the object tree and edits properties in the properties panel, Logic mode wires events to actions, and DB mode shows the .vs service methods. Understanding this trio is the basis for reading and modifying any VL page.

OPERATIONS

1. Select a component in Visual mode, view the object tree, and inspect/edit properties in the properties panel
2. View that component's events and their triggered actions in the Logic event panel
3. Inspect the service methods called by those events in DB mode
4. When changing things, separate concerns: appearance/binding → properties, interaction → events, backend behavior → methods
5. After editing, run Lint to verify event and method references still connect

EXAMPLE / EXERCISE

The trio for a "New Task" button:

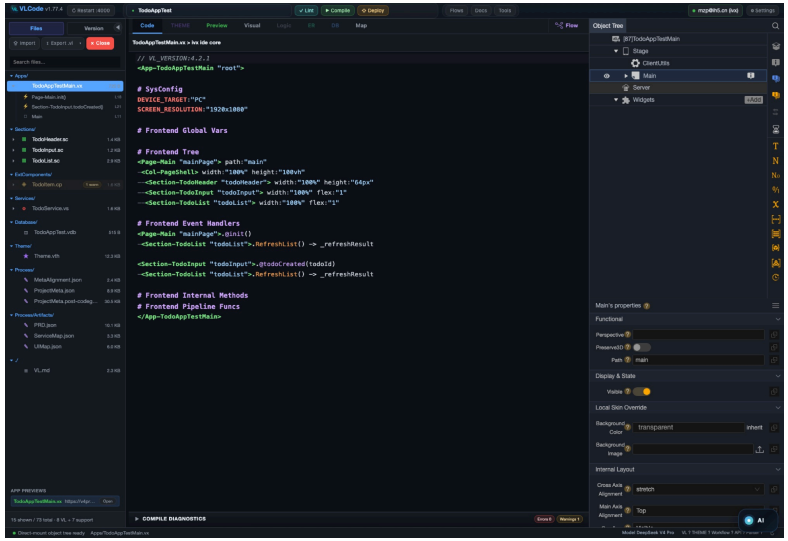
```
Properties: label = "New Task", variant = primary
Event: onClick -> call TaskService.createTask(form)
Method: TaskService.createTask writes the task row and refreshes the board
```

To change the label, edit a property; to change click behavior, edit the event's target method.

PITFALL

Editing a service method for something that's really a property (button label, color), or stuffing business logic into properties, misplaces page/backend responsibilities. Change each in its proper home: properties/methods/events.

Local-First, Cloud Optional



Fresh VLCode page loaded in a browser

CORE IDEA

VLCode's core principle is local-first: your project source lives on your machine under ~/Documents/VLProjects/, which is faster to read/write and smoother for AI; the cloud is just an optional delivery target, not the only surface. You can do generation, Lint, Compile, and Preview entirely locally; only when you want to publish to a team or go live do you deploy to the cloud via a bound GID. This is the opposite of the "online IDE" model — you don't have to be in the cloud to work.

OPERATIONS

1. Create the project under the standard ~/Documents/VLProjects/<PascalCaseName>/ directory
2. Do generation, edits, and Lint → Compile → Preview validation locally
3. When offline, rely on the bundled seed prompts and docs to still run codegen
4. Only when delivering/going live, bind the cloud project via the GID-binding button in the sidebar header
5. Treat cloud deployment as an optional final step, not a starting point

EXAMPLE / EXERCISE

The whole loop works locally; cloud is an optional endpoint:

```
Local: npm run dev -> :4000 -> generate in AI Chat -> Lint -> Compile -> Preview
(Optional) Delivery: click the GID-binding button in the sidebar header -> deploy to cloud
```

The development loop completes without a network; going to cloud is just the step that ships the result out.

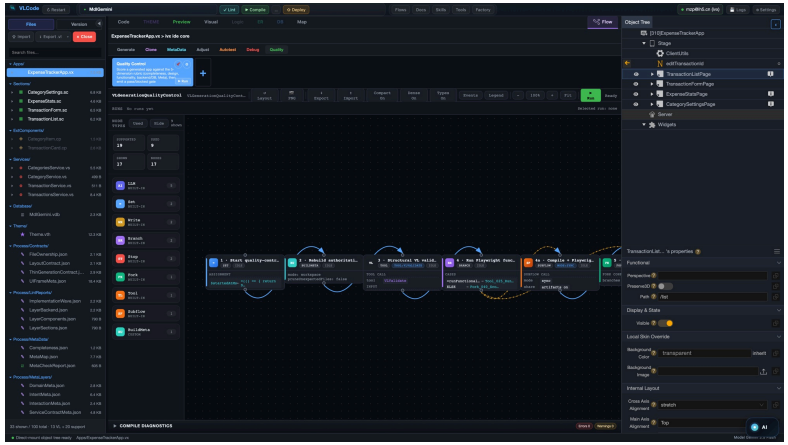
PITFALL

Assuming you must be online or bind a GID before starting, then getting blocked by the network or a slow Resource Center. Local-first means you can work offline — the bundled seed prompts/docs exist exactly for this.

CHECKPOINT

You can explain that local-first means the dev loop (generate/Lint/Compile/Preview) runs entirely on your machine, cloud deployment is an optional delivery target, and you know the

Verified, Not Hopeful



Quality gates inside the workflow graph

CORE IDEA

Something AI generates that "looks right" is not the same as being right. VLCode turns hope into evidence with a validation chain: Lint (low-cost static check, first) → Compile (heavier platform validation via parsevl) → Preview (actually run at /route/<path>). It also follows root-cause-first: never write fallback / error-swallowing / fake-success code; if the same error appears twice, return to the source files and ProjectMeta to find the root cause rather than papering over it with broad rewrites.

OPERATIONS

1. After generation, run Lint first to catch low-cost structural and reference issues
2. Once Lint passes, run Compile (parsevl) for heavier platform validation — don't use Compile as the first trial loop
3. After Compile passes, open Preview at /route/<path> and exercise the real behavior
4. If the same error repeats twice, return to the source files + ProjectMeta to locate the root cause
5. Refuse fallback / error-swallowing / silent fallback so errors surface honestly

EXAMPLE / EXERCISE

The correct validation rhythm, not recompiling on luck:

```
Lint  -> 0 errors
Compile -> ok
Preview /route/board -> the board really renders and tasks can be created
```

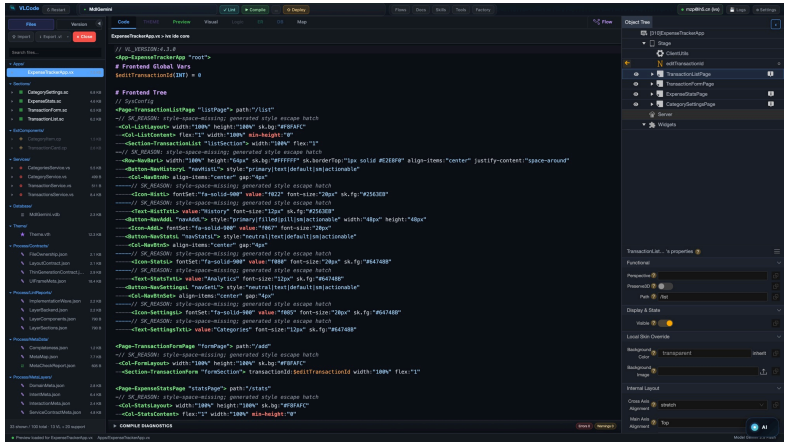
If Preview reports "TaskService.createTask is undefined", go back to the .vs source and ProjectMeta to check the method reference, rather than recompiling a few more times hoping it resolves.

PITFALL

Using Compile as the first trial loop and recompiling on luck is slow and hides root causes; worse is adding a fallback to a real error so it "looks like it passed". Lint first, root-cause-first, no fake success.

CHECKPOINT

Human And AI Collaborate At The Component Layer



AI Chat used as the command and decision surface

CORE IDEA

The division of labor between human and AI is not about "who types which characters" but about the component layer: AI generates and repairs whole components (properties/methods/events, Sections, Services, Data) through workflows, while the human reviews and decides at the component layer — inspecting page structure in Visual, interactions in Logic, backend in DB/ER, and ruling on key nodes through Interactive Cards (blocking review-card / form-card / picker-card, alongside the ask_user / human_gate chat events). Collaboration happens over structured components and cards, not line-by-line keyboard wrestling.

OPERATIONS

1. Let AI generate/modify whole pages or services at the component layer via workflows
2. Review the output by component across Visual/Logic/DB/Data modes
3. Answer trade-offs from the ask_user chat event's options, using the custom-answer path when needed
4. Approve / reject / request_changes on the review-card (human_gate arrives as a chat event)
5. Inject structured parameters into the workflow via the form-card instead of hand-editing code

EXAMPLE / EXERCISE

AI generates; the human gates at the card layer:

```
AI: Generated the TaskBoard three-column layout and two TaskService methods.
[review-card] Should the status column include "Archived"? [approve] [reject] [request_changes]
You: request_changes -> "Add Archived and let it be hidden from the board."
```

You decide at the component/card layer, AI adjusts accordingly, with no line-by-line editing.

PITFALL

Skipping the cards and jumping into Code mode to hand-edit AI-generated components risks ProjectMeta inconsistency and gets overwritten when a workflow regenerates. Collaborate at the component and card layer; reserve line edits for genuine micro-tweaks.

Banking Token-Compute Through Reuse



DAG harness: artifact flow

CORE IDEA

Every from-scratch LLM generation spends token-compute; VLCode "banks" that compute through reuse instead of burning it anew each time. Two main paths: workflows like FactoryTurbo reuse pre-built factory components (speed-first, less generation), and mature workflows/.vflow are reused through the Resource Center, with bundled seed prompts and docs letting codegen run even offline or when the Resource Center is slow. The more you reuse, the fewer tokens each unit of result costs.

OPERATIONS

1. For reusable pages/components, prefer factory-turbo-codegen to reuse pre-built factory components
2. Reuse validated workflows and .vflow through the Resource Center instead of rewriting each time
3. Rely on bundled seed prompts/docs to keep generating when offline or when the RC is slow
4. In Settings, use the cost-sensitive default models (chat: DeepSeek V4 Pro, workflow/fanout: Kimi 2.5)
5. Use chatModelPolicy=auto so the model only upgrades for long/complex prompts, avoiding needless spend

EXAMPLE / EXERCISE

Reuse rather than reburn:

```
Use factory-turbo-codegen to reuse the existing table and form components and quickly build a customer list page and detail page for CustomerCRM.
```

Comparing to making the model write a list/form set from scratch, reusing pre-built components generates far fewer tokens and is more stable.

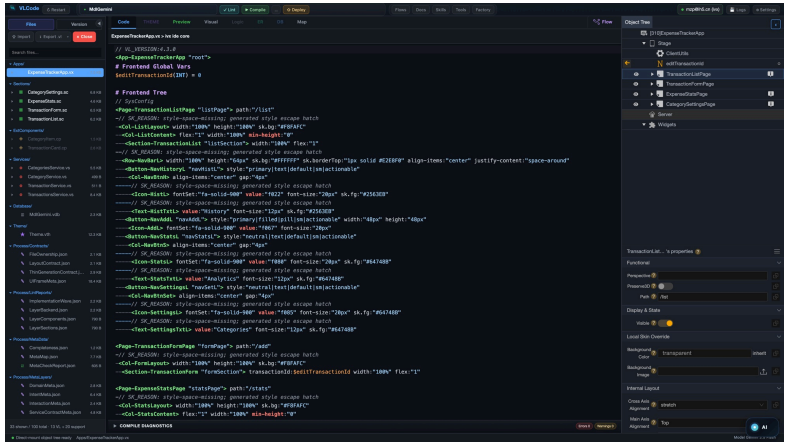
PITFALL

Regenerating every similar page from scratch pays the same token cost over and over. When reuse is possible, use FactoryTurbo / Resource Center and keep the cost-sensitive default model settings.

CHECKPOINT

You can name at least two ways to bank compute through reuse (FactoryTurbo reusing factory components; reusing workflows/.vflow via Resource Center) and note the role of cost-sensitive default models and chatModelPolicy=auto.

When To Use A Workflow, When To Edit Manually



Code and Visual editing modes side by side

CORE IDEA

Default to workflows: sizable generation, adding pages/services, batch changes, and repairs should all go through the matching workflow (codegen, add-page, add-service, batch-adjust, compile-fix, etc.) because workflows preserve ProjectMeta consistency, include validation, and can rerun. Manual editing is reserved for genuine micro-tweaks — change a label, adjust a property, read a line of VL. The test is whether the change spans files or touches ProjectMeta: if yes, use a workflow; only if no, edit by hand.

OPERATIONS

1. Assess the change scope: cross-file or affecting ProjectMeta → use a workflow
2. Use add-page for a new page, add-service for a new service, batch-adjust for many small changes
3. For repairs, use compile-fix / debug-single-file / debug-multi-file rather than large manual edits
4. Switch to Code/Visual mode for manual edits only on single-point tweaks (label, one property)
5. Whichever path you take, run Lint after editing

EXAMPLE / EXERCISE

Pick the path by scope:

```
Rename a button "New Task" -> "Create Task"        => hand-edit in the Visual properties panel
Add a "Weekly Report" page to TeamPM with a service => add-page + add-service workflows
Unify the primary color to brand blue across 10 pages => batch-adjust workflow
```

Micro-tweaks by hand; cross-file/structural changes always go through a workflow.

PITFALL

Hand-making cross-file structural changes to "save time" easily drifts ProjectMeta from the actual files, causing conflicts or overwrites when a workflow regenerates later. Once the scope exceeds a single point, use a workflow.

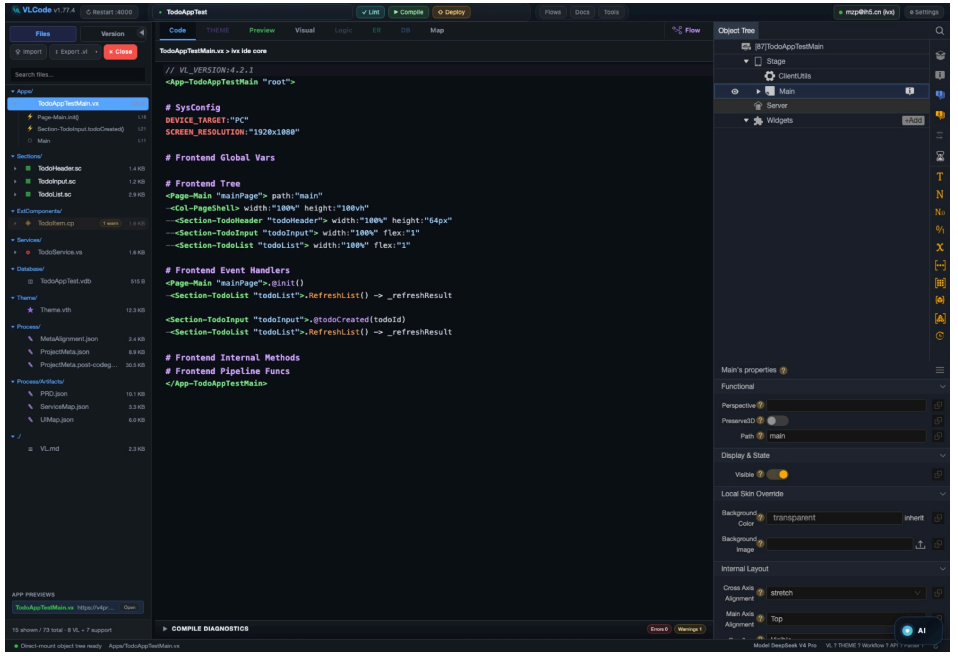
CHECKPOINT

Using the "cross-file / touches ProjectMeta" test, you can correctly classify a change as workflow-worthy or hand-editable and name the matching workflow (e.g. add-page, batch-

CHAPTER

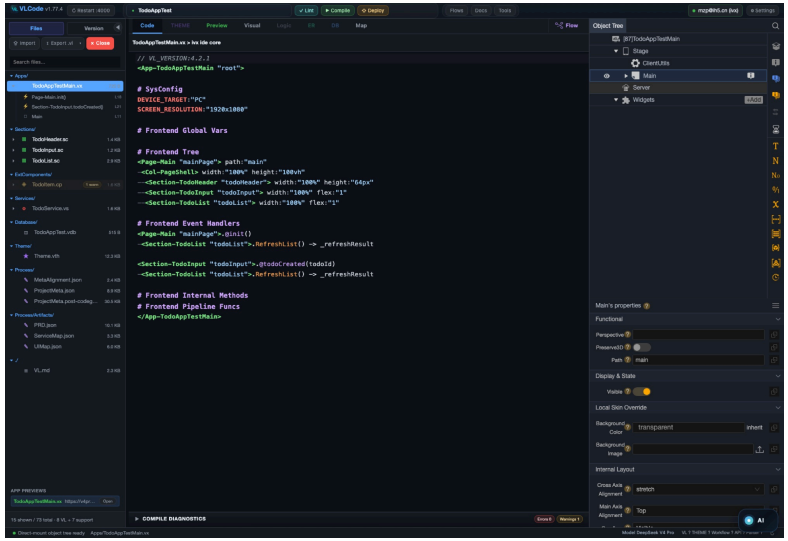
Part II: Installation, Startup, And Workspace

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



Fresh VLCode page loaded in a browser

Choose Desktop, Browser, Or Source Run



Fresh VLCode page loaded in a browser

CORE IDEA

VLCode ships in three startup forms: the Electron desktop app downloaded from releases.visuallogic.ai, the browser Web IDE run from source (http://localhost:4000), and a desktop shell run from source (npm run electron). The desktop app is one-click and needs no terminal, ideal for pure users; the source paths suit anyone editing source or working close to development. All three open the same IDE — they differ only in shell and launch path.

OPERATIONS

1. Decide your role: pure app builder picks the desktop app; source editor or developer picks a source path.
2. Desktop: download the .dmg or .zip from releases.visuallogic.ai, drag it to Applications, and open it.
3. Browser: in the source directory run npm run dev (or npm start), then visit http://localhost:4000.
4. Source desktop shell: in the source directory run npm run electron to launch the Electron shell.
5. Remember all three open the same IDE; pick the lowest-friction path first to get a working environment.

EXAMPLE / EXERCISE

Three commands map to three routes — pick one:

```
# Browser Web IDE (from source)
npm run dev          # -> http://localhost:4000

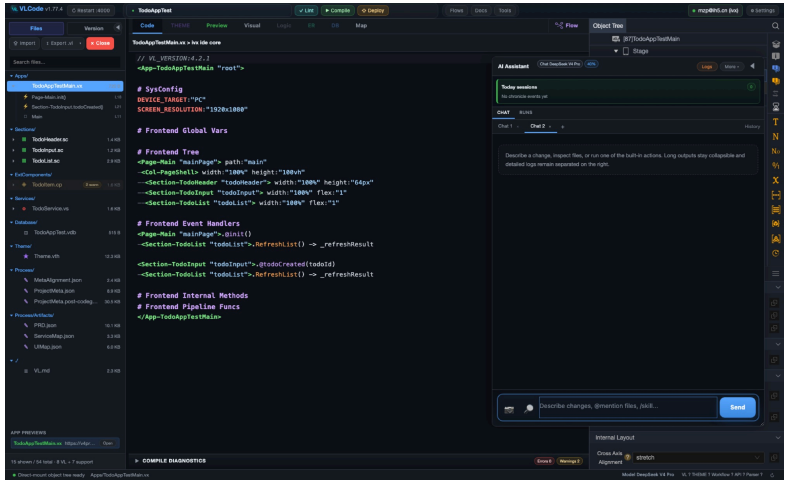
# Desktop shell (from source)
npm run electron

# Desktop app (download & install, no command)
# Download .dmg/.zip from releases.visuallogic.ai, drag into Applications
```

PITFALL

Beginners agonize over the browser vs desktop choice as if features differ; they are the same IDE. Get running first with the desktop app or npm run dev rather than burning time deciding.

Get The Installer From The Release Page



Starting from an empty chat session

CORE IDEA

VLCode's desktop installer is a same-site download from the official release page releases.visuallogic.ai, using your unified visuallogic.ai account — the same account also gets you VL Agent. The page offers .dmg / .zip installers; download, drag into Applications, and run, with no terminal involved.

OPERATIONS

1. Open releases.visuallogic.ai using your unified visuallogic.ai account.
2. Pick the installer for your platform (.dmg or .zip on macOS) and download it.
3. On macOS open the .dmg and drag VLCode into the Applications folder.
4. Launch VLCode from Applications and confirm the IDE window opens cleanly.
5. Note the same account on the same site also downloads VL Agent — both products share one account.

EXAMPLE / EXERCISE

The release page is the single trusted source — don't hunt third-party mirrors:

```
Download page: releases.visuallogic.ai
macOS: the VLCode .dmg  ->  drag into /Applications
Login: unified visuallogic.ai account (same account gets VL Agent)
```

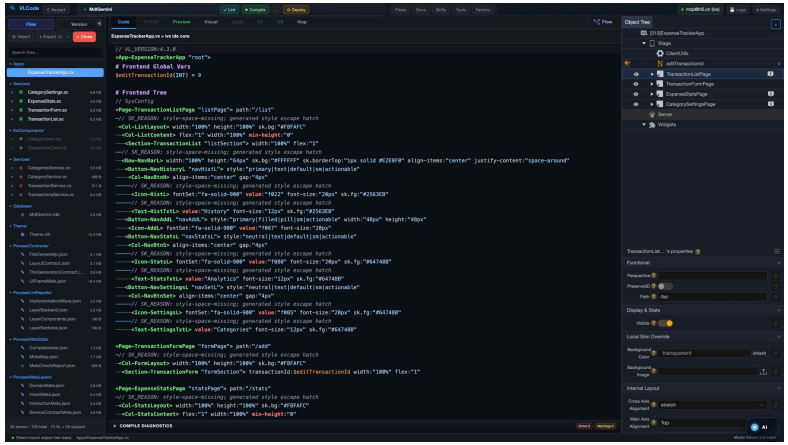
PITFALL

Never download the installer from an unknown third-party link; trust only the official same-site source releases.visuallogic.ai to avoid a tampered or stale build.

CHECKPOINT

You have downloaded the installer from releases.visuallogic.ai and launched VLCode after dragging it into Applications.

Start The Web IDE From Source



VLCode shell with workspace, editor, and assistant panels

CORE IDEA

To start the browser Web IDE from source, run npm run dev (or npm start); the IDE serves at http://localhost:4000 within the 4000–4010 port range. This path fits anyone editing VLCode source, working close to development, or preferring the IDE in a browser. The same source directory also offers npm run electron for the desktop shell and npm run test for tests.

OPERATIONS

1. Open a terminal in the VLCode source directory.
2. Run npm run dev (or npm start) to start the browser Web IDE.
3. Once the server is up, visit http://localhost:4000 in your browser.
4. Confirm the IDE finishes loading and the bottom status bar shows the port and compile status.
5. When needed, use npm run electron for the desktop shell or npm run test in the same directory.

EXAMPLE / EXERCISE

Standard command and URL for the Web IDE:

```
cd <VLCode source directory>
npm run dev          # start the browser Web IDE
# Open in browser:
http://localhost:4000
```

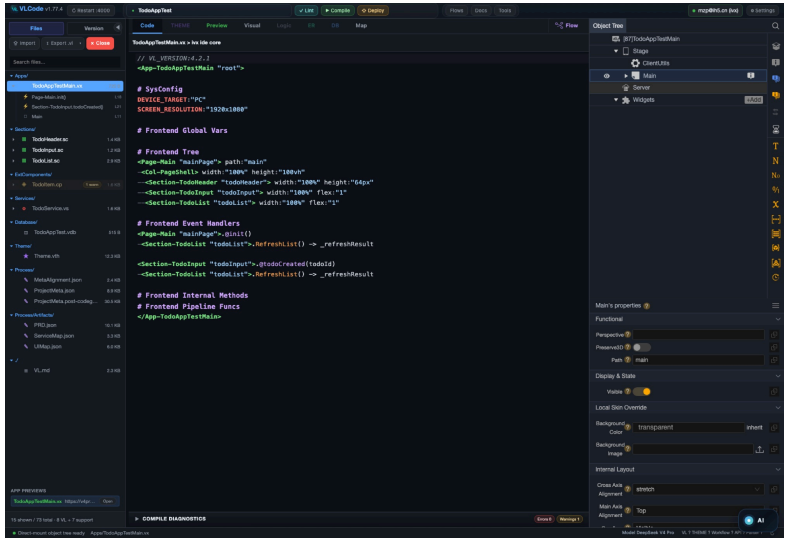
PITFALL

Don't guess common ports like 3000 or 8080 in the browser; the VLCode Web IDE is fixed to the 4000–4010 range (default 4000), and a wrong port just yields a blank page.

CHECKPOINT

After running npm run dev you can open the VLCode Web IDE at http://localhost:4000 with the full interface loaded.

Port 4000 And Fixed-Port Discipline



Fresh VLCode page loaded in a browser

CORE IDEA

The VLCode Web IDE listens on 4000 by default, within the 4000–4010 range, and preview routes are served from that same IDE. Fixed-port discipline is a cross-cutting VL principle: start only on the agreed port and never silently fall back to a random one — otherwise you debug against a stale process or the wrong port. The bottom status bar shows the active port, the fastest way to confirm where the service runs.

OPERATIONS

1. Before starting, confirm 4000 isn't held by a stale process; if it is, clear the old process rather than switching ports.
2. Start with npm run dev so the IDE lands in the 4000–4010 range.
3. Read the bottom status bar to confirm the actual port, LLM provider/model, and compile status.
4. For preview, use the IDE's /route/<path>, not a separate service on a random port.
5. If the port misbehaves, investigate a leftover process first and fix the root cause.

EXAMPLE / EXERCISE

When the port is busy, clear the old process instead of dodging to another port:

```
# See who holds 4000
lsof -ti:4000
# Kill the stale process, then restart (keep the fixed port)
npm run dev          # -> http://localhost:4000
```

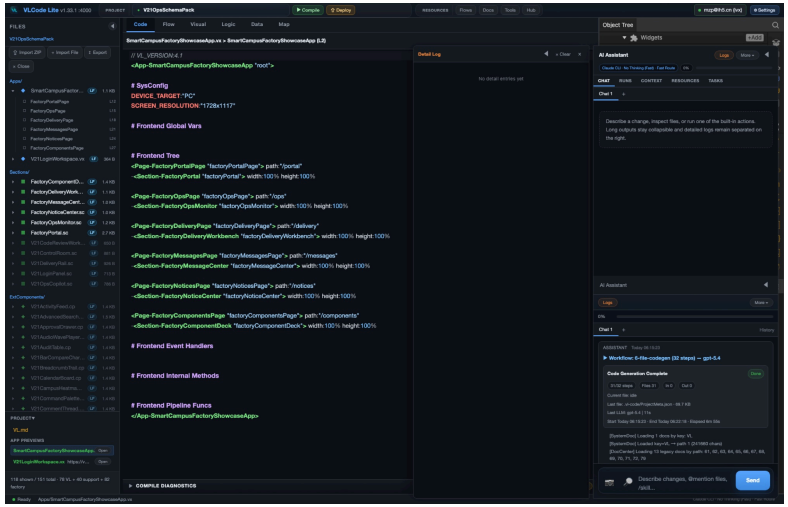
PITFALL

The classic mistake is grabbing a different port when 4000 is busy, then debugging across two instances with confused state. The right move is to clear whatever holds 4000 and stay on the fixed port.

CHECKPOINT

You can read the IDE's active port from the status bar and recover from a busy 4000 by clearing the stale process rather than switching ports.

Configure The LLM Provider And Model



AI Chat with detailed run context

CORE IDEA

VLCode configures provider and model under Settings -> LLM, supporting anthropic-api / openai-api / deepseek-api / kimi-api / glm-api / gemini-api / minimax-api / qwen-api. For cost sensitivity the defaults are DeepSeek V4 Pro for chat and Kimi 2.5 for workflow and fan-out. The chat model and workflow model can differ, but the provider/company should stay consistent unless you deliberately mix.

OPERATIONS

1. Open Settings -> LLM.
2. Choose a provider (e.g. deepseek-api or anthropic-api) and enter its key.
3. Set the chat model (cost-sensitive default DeepSeek V4 Pro) and the workflow/fan-out model (default Kimi 2.5).
4. Tune chatModelPolicy (auto|fast|strong), reasoningLevel (basic|standard|high), and thinkingEnabled as needed.
5. Set workflowMaxConcurrency (default 5) and workflowFanoutProvider; enable allowMixedLlmProviders only for cross-company mixing.
6. Confirm the bottom status bar shows the provider/model that matches your settings.

EXAMPLE / EXERCISE

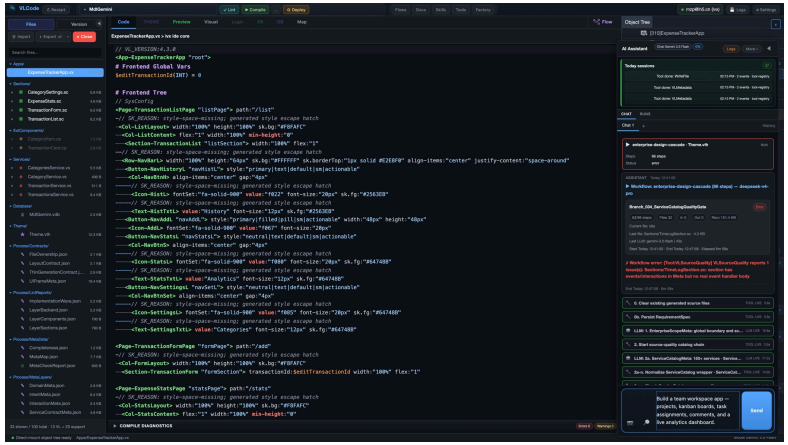
A typical cost-sensitive default setup:

```
Provider:              deepseek-api
Chat model:            DeepSeek V4 Pro
Workflow/fan-out model: Kimi 2.5
chatModelPolicy:       auto   # auto-upgrades for long/complex prompts or multi-round tool use
workflowMaxConcurrency: 5
reasoningLevel:        standard
```

PITFALL

Don't casually split chat and workflow across different companies' providers; unless you deliberately mix and have enabled allowMixedLlmProviders, keep one provider to avoid inconsistent behavior and billing confusion.

Manage Keys With SecretRoot



A concise project brief in the assistant panel

CORE IDEA

SecretRoot is VLCode's built-in paste-in key vault: paste KEY=VALUE directly and it auto-detects common prefixes (sk-ant-, sk-proj-, AIza..., etc.) and manages a masked inventory of your keys. This means you never write keys into project files — secrets must never be committed to project source.

OPERATIONS

1. Open the SecretRoot vault inside Settings.
2. Paste your keys, one KEY=VALUE per line.
3. Let it auto-detect prefixes (sk-ant-, sk-proj-, AIza...) and categorize them.
4. Verify the masked inventory shows every key present and obscured.
5. Return to Settings -> LLM so the matching provider uses the vaulted key.

EXAMPLE / EXERCISE

Paste this shape directly; the vault auto-detects and masks:

```
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxxxxx
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxx
GEMINI_API_KEY=AIzaxxxxxxxxxxxxxxxxx
# Inventory shows masked values only, e.g. sk-ant-****
```

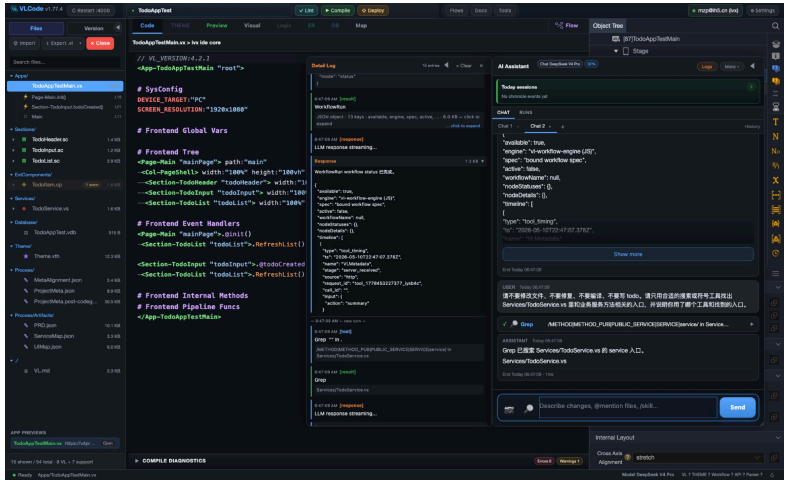
PITFALL

Never hard-code real keys into project source files like .vx/.vs or any committable doc; keep keys only in the SecretRoot vault so they can't leak with the repository.

CHECKPOINT

You have pasted at least one provider key into SecretRoot, see it masked in the inventory, and the provider works while no plaintext key exists in source.

Create ~/Documents/VLProjects



Final IDE state after project creation

CORE IDEA

All VL projects live under one workspace root, ~/Documents/VLProjects/, with each project in its own PascalCase-named subfolder. Keeping projects here lets VLCode locate, open, and manage them consistently and prevents generated files from scattering into a tool's source tree.

OPERATIONS

1. Confirm or create the workspace root ~/Documents/VLProjects/.
2. Create a PascalCase subfolder for each new project (e.g. TeamPM) inside it.
3. Keep every VL project under this single root for one-place management.
4. When opening a project in VLCode, point at ~/Documents/VLProjects/<ProjectName>/.
5. Never write generated project files into VLCode's or another tool's source directory.

EXAMPLE / EXERCISE

Create the workspace root and your first project folder:

```
mkdir -p ~/Documents/VLProjects/TeamPM
# All VL projects then live under ~/Documents/VLProjects/, one folder per project
```

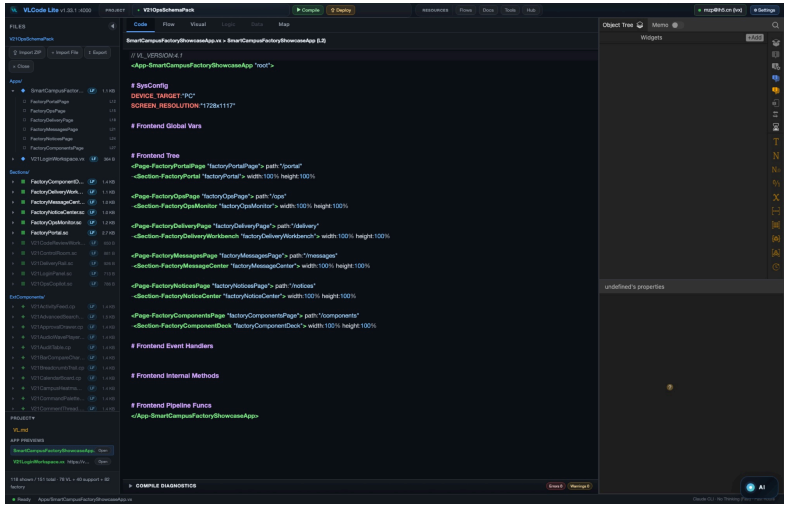
PITFALL

Don't scatter projects across Downloads or a tool's source tree; centralize them under ~/Documents/VLProjects/, or VLCode can't locate and manage them consistently.

CHECKPOINT

Your ~/Documents/VLProjects/ exists with at least one PascalCase-named project subfolder inside it.

Project Naming: English PascalCase



Code mode for inspecting and editing VL source

CORE IDEA

VL file names and project names must use English PascalCase (e.g. TeamPM, CustomerCRM), a cross-cutting hard rule of the VL ecosystem. Non-English names or spaces cause compile and path problems, so from the project folder down to .vx/.sc source files, stick to English PascalCase.

OPERATIONS

1. Pick an English PascalCase project name (e.g. TeamPM, CustomerCRM, CampusOps).
2. Create the folder under ~/Documents/VLProjects/ with that name.
3. Keep generated or new .vx/.sc/.vs/.vdb files in English PascalCase too.
4. Avoid Chinese characters, spaces, or special symbols in file names.
5. Recheck all file names in the VLCode file tree against this convention.

EXAMPLE / EXERCISE

Correct vs incorrect naming:

```
Good:  TeamPM/        TaskBoard.sc   TaskService.vs
Bad:   团队看板/      task board.sc  任务服务.vs
Rule:  English + PascalCase, no spaces, no non-ASCII characters
```

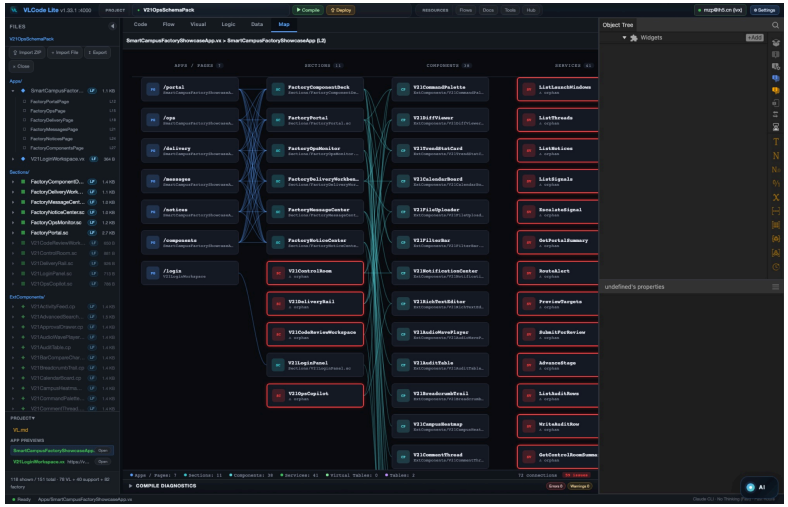
PITFALL

Using a Chinese project name or spaces in file names is a common error that breaks compilation or routing; use English PascalCase from the start rather than renaming after a failure.

CHECKPOINT

Your project folder and all VL source file names are English PascalCase, with no Chinese characters, spaces, or special symbols.

Standard Project Directory Structure



Map mode for application structure

CORE IDEA

A VL project root follows a fixed subdirectory convention: Apps/, Sections/, Services/, Database/, Theme/, ExtComponents/, Process/, Config/. These map to VL's file types — Apps holds .vx app entries, Sections holds .sc pages/regions, Services holds .vs service logic, Database holds .vdb schema, Theme holds one .vth theme (default ThemeDefault_7.0.3.vth), ExtComponents holds .cp/.wc components, and Config holds project.state.json / project.settings.json / secrets.json (the gid binding lives in project.state.json). A consistent layout lets generation, repair, and ProjectMeta reliably locate files.

OPERATIONS

1. Create the standard subdirectories under the project root: Apps/ Sections/ Services/ Database/ Theme/ ExtComponents/ Process/ Config/.
2. Put .vx app entries into Apps/.
3. Put .sc pages/regions into Sections/ and .vs services into Services/.
4. Put .vdb schema into Database/, .vth themes into Theme/, and .cp/.wc components into ExtComponents/.
5. Keep the layout stable so ProjectMeta and every workflow can locate files consistently.

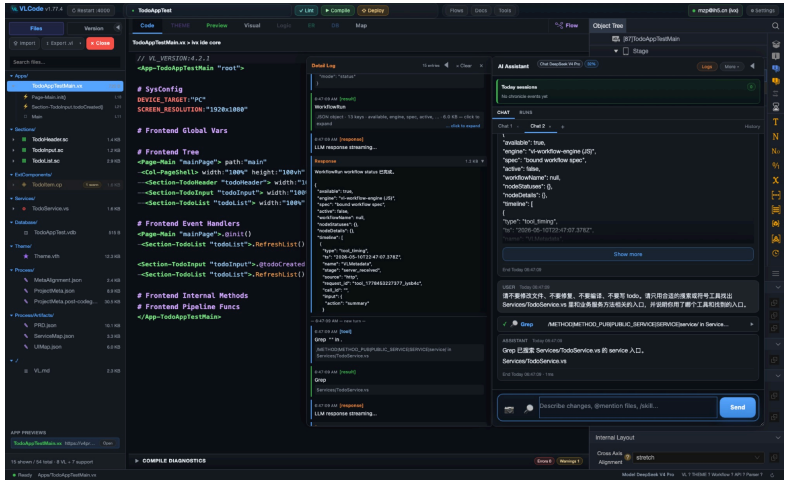
EXAMPLE / EXERCISE

Standard skeleton for a TeamPM project:

```
~/Documents/VLProjects/TeamPM/
  Apps/           TeamPM.vx
  Sections/       TaskBoard.sc
  Services/       TaskService.vs
  Database/       TaskDb.vdb
  Theme/          ThemeDefault_7.0.3.vth
  ExtComponents/  TaskCard.cp
  Process/
  Config/         project.state.json
```


PITFALL

Open An Existing Project



Final IDE state after project creation

CORE IDEA

Opening an existing project means pointing VLCode at a project folder under ~/Documents/VLProjects/ as the current workspace. One IDE window equals one workspace equals one backend; once open, the left file tree shows Apps/Sections/Services and ProjectMeta loads, so you can jump straight into Code/Visual/Data modes.

OPERATIONS

1. Open ~/Documents/VLProjects/<ProjectName>/ in VLCode as the current workspace.
2. Confirm the left file tree lists Apps/Sections/Services/Database/Theme.
3. Switch to Map or Code mode to scan the project structure and files.
4. Check the bottom status bar for a healthy port, provider/model, and compile status.
5. Remember one window is one workspace and one backend; changing projects means switching workspaces.

EXAMPLE / EXERCISE

Open an existing CustomerCRM project and verify its layout:

```
Open workspace: ~/Documents/VLProjects/CustomerCRM/
Left file tree should show:
  Apps/CustomerCRM.vx
  Sections/  Services/  Database/  Theme/
Status bar: port 4000 · provider/model · compile ready
```

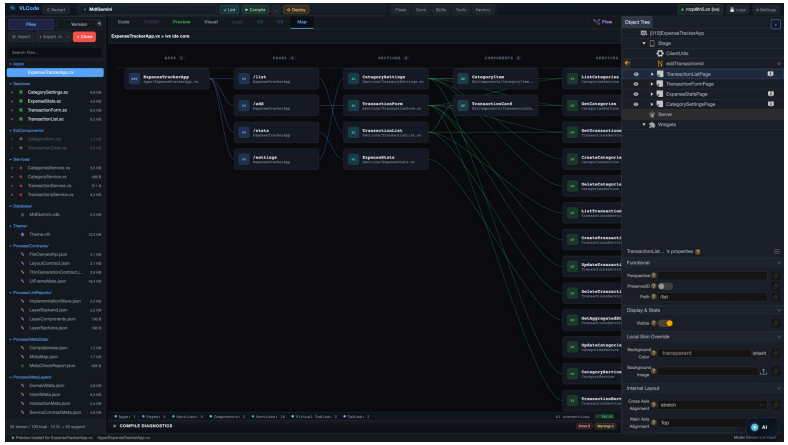
PITFALL

Don't mix two projects' files in one window; an IDE window is one workspace and one backend, and opening the wrong project leaves the file tree and ProjectMeta out of sync with what you expect.

CHECKPOINT

You can point VLCode at an existing project folder, see its standard directories in the file tree, and enter any mode to inspect contents.

Bind The Cloud GID



Map mode for project structure and relationships

CORE IDEA

A GID is a project's cloud identifier; binding it in VLCode links the local project to a cloud project for optional delivery like cloud compile/deploy. The GID binding button lives in the sidebar header (the file-panel header), where you bind or rebind the cloud GID; the binding is recorded in Config/project.state.json. Under local-first, the cloud is an optional target, so binding is not required for local development.

OPERATIONS

1. Find the GID binding button in the VLCode sidebar header.
2. Click to bind and enter the cloud project GID you want to attach.
3. Confirm the bind succeeded; use the same button to rebind when switching cloud projects.
4. Remember the cloud is an optional delivery target; local development doesn't depend on a GID bind.
5. If an old GID lacks permission, rebind to a GID you have access to rather than forcing it.

EXAMPLE / EXERCISE

Bind the cloud GID from the sidebar header:

```
Sidebar header -> [Bind GID] button
Enter cloud project GID (e.g. 4734) -> Bind
To switch projects: same button -> Rebind
```

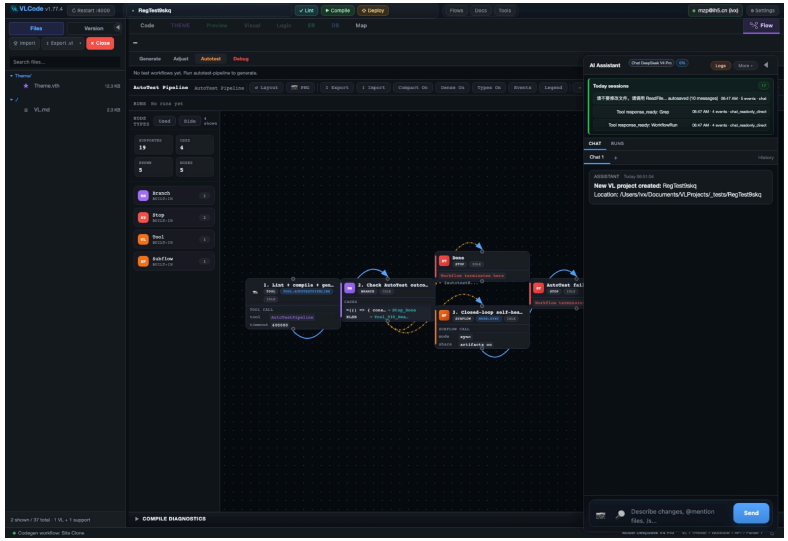
PITFALL

Don't bind an old GID you lack permission for; that triggers permission errors on cloud operations. Confirm the GID is a current project you have access to before binding.

CHECKPOINT

You can use the sidebar-header GID button to bind your local project to a cloud GID you have permission for, and rebind when needed.

First-Open Health Check



Logs and execution details after a run

CORE IDEA

After first opening VLCode, run a health check to confirm the environment is truly ready: the service is on its fixed port, provider/model are set, keys are in SecretRoot, and the workspace structure is sound. This catches the most common blockers — ports, keys, directories — before you write your first requirement, so you don't debug an illusion later.

OPERATIONS

1. Read the bottom status bar: confirm the port is 4000 (range 4000–4010), provider/model are shown, and compile status is healthy.
2. Open Settings -> LLM and confirm a provider is selected with chat/workflow models set.
3. Check the SecretRoot vault that keys are pasted and shown masked.
4. Confirm the workspace points at ~/Documents/VLProjects/<Project> with standard directories in the file tree.
5. Send a trivial AI Chat command and confirm both Chat and Detail Log respond normally.

EXAMPLE / EXERCISE

Walk the health checklist item by item:

```
[ ] Status bar port = 4000, compile status healthy
[ ] Settings -> LLM: provider selected, chat/workflow models set
[ ] SecretRoot: at least one key (shown masked)
[ ] Workspace: ~/Documents/VLProjects/TeamPM/, file tree shows standard dirs
[ ] AI Chat: send "List the current project structure", Chat and Detail Log respond
```

PITFALL

Skipping the health check and diving in is a common error: you discover mid-generation that a key is missing or the port drifted, wasting a whole round. Spend a minute on the checklist to block problems up front.

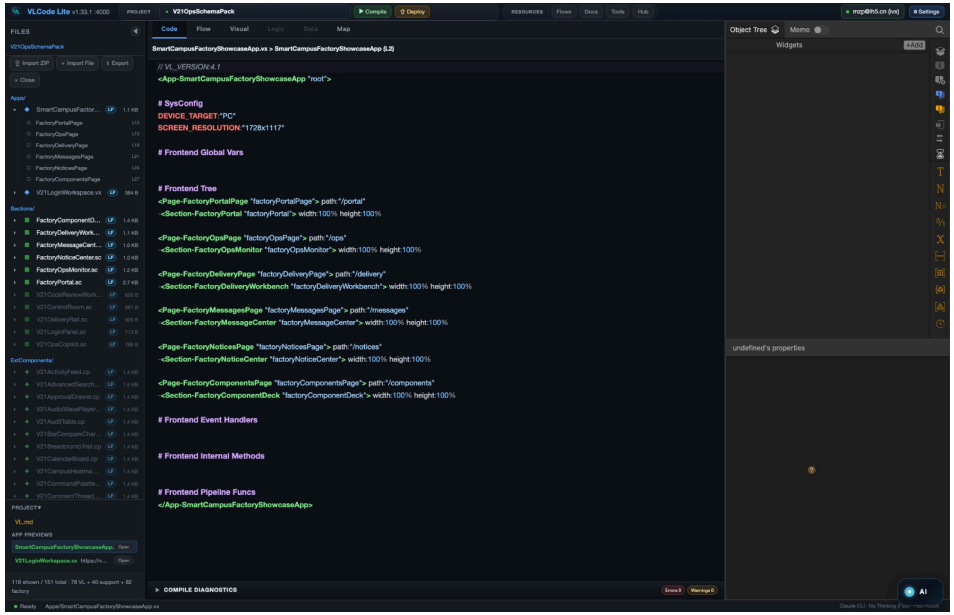
CHECKPOINT

You have verified the port, provider/model, SecretRoot keys, workspace directories, and AI Chat response — all passing before you write a requirement.

CHAPTER

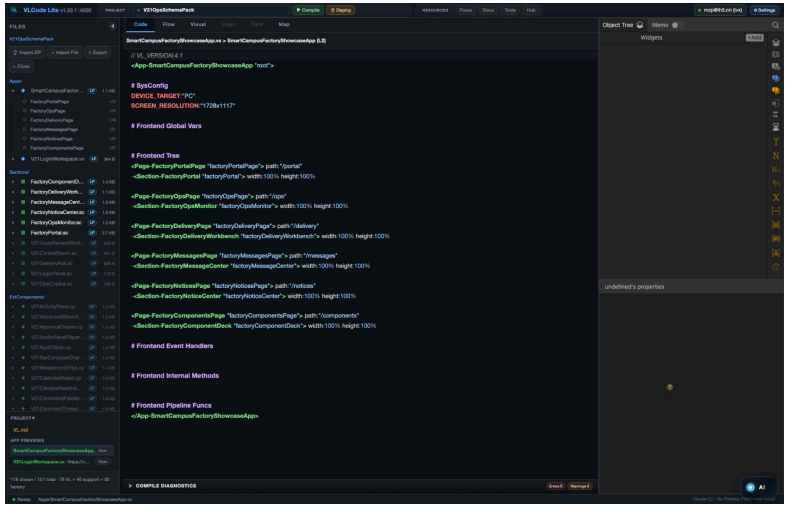
Part III: Interface Tour And Daily Operations

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



Code mode for inspecting and editing VL source

Left File Tree



Code mode for inspecting and editing VL source

CORE IDEA

The left file tree is the workspace navigation root, listing the VL source files under the project's standard directories — .vx in Apps/, .sc in Sections/, .vs in Services/, .vdb in Database/, .vth in Theme/, and .cp/.wc in ExtComponents/. It mirrors the real on-disk layout of ~/Documents/VLProjects/<Name>/ in true local-first fashion, with no cloud dependency. Clicking a file drives the center editor and the mode tabs to that file's content.

OPERATIONS

1. Expand the current workspace in the left rail and browse the Apps/, Sections/, Services/, Database/, Theme/, ExtComponents/ folders
2. Click a .sc file to load that Section's source into the center Code editor
3. Use the tree to confirm a generation workflow wrote all expected files (cross-check against the file writes recorded in Detail Log)
4. Use the GID binding button in the sidebar header to bind/rebind the local project to a cloud GID
5. When switching workspaces, verify the tree root points at the correct PascalCase project directory

EXAMPLE / EXERCISE

Browsing the tree for a TeamPM task board project, the standard layout looks like:

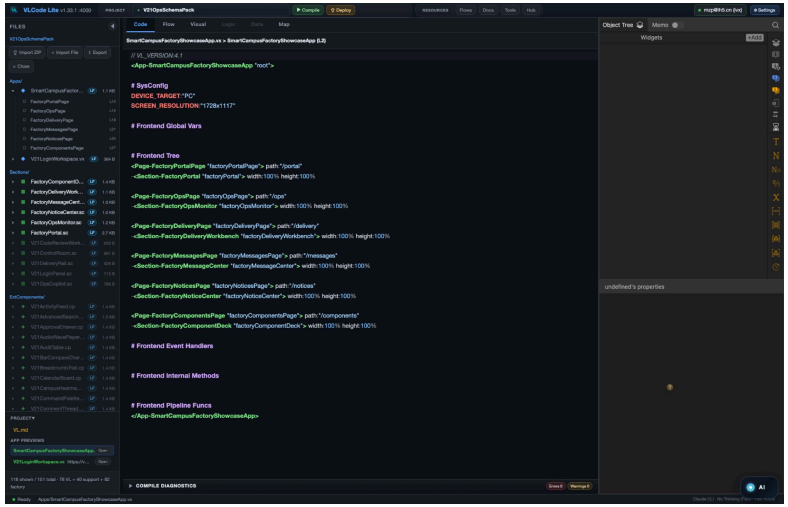
```
~/Documents/VLProjects/TeamPM/
  Apps/TeamPM.vx
  Sections/TaskBoard.sc
  Sections/TaskDetail.sc
  Services/TaskService.vs
  Database/TeamPM.vdb
  Theme/TeamPM.vth
```

Clicking TaskBoard.sc loads that page's VL source in the center editor.

PITFALL

After an interrupted generation, assuming the project is complete because the tree 'looks full' is a classic mistake — reconcile against the Detail Log file-writes list, and recover missing

Center Code Editor



Showcase: code mode

CORE IDEA

The center Code editor is the direct edit surface for VL source, paired with the Code mode tab. The first line of any VL source file must be VL_VERSION (e.g. // VL_VERSION:4.3.1) with nothing before it, and hierarchy is expressed with consecutive hyphens (- / -- / ---), never spaces. This is where you read AI-generated output and make minimal manual edits.

OPERATIONS

1. Click a file in the left tree; the editor opens it in Code mode showing its VL source
2. Confirm the first line is the VL_VERSION declaration with no blank line or comment before it
3. Maintain indentation hierarchy with consecutive hyphens — never mix in spaces
4. Make targeted small edits rather than broad rewrites, preserving the small-file structure
5. After editing, run Lint (the cheapest check) first before deciding whether to Compile

EXAMPLE / EXERCISE

Reading the top of a Section in the Code editor, the correct form is:

```
// VL_VERSION:4.3.1
- Section TaskBoard
-- Column status:todo
--- TaskCard
```

VL_VERSION on line one plus consecutive-hyphen hierarchy is the precondition for valid VL.

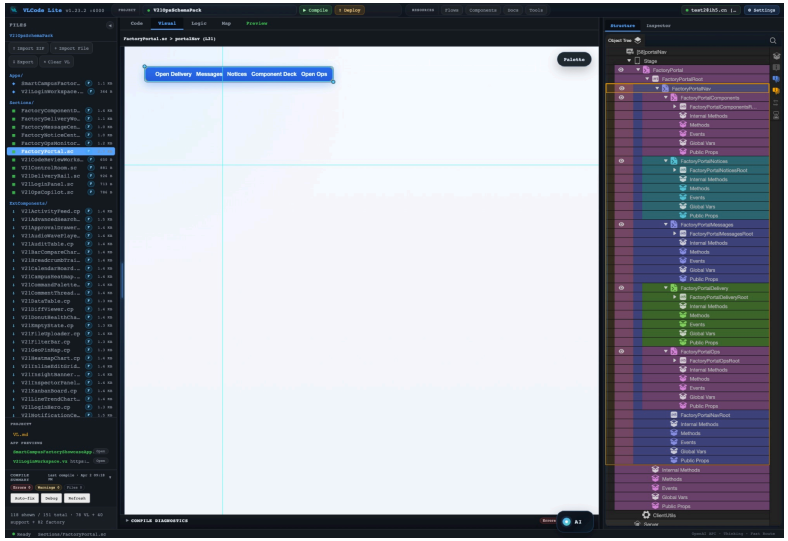
PITFALL

Replacing hyphen indentation with spaces, or putting a blank line/comment before VL_VERSION, triggers 'strange' parse errors. Validate edits with Lint first instead of jumping straight to Compile.

CHECKPOINT

You can recognize and preserve the VL_VERSION first line and consecutive-hyphen indentation, and make one structure-preserving edit that passes Lint.

Visual Canvas



Showcase: visual mode

CORE IDEA

Visual mode gives you a canvas that renders a Section as a visible page layout, alongside the object tree and properties panel. It is the graphical counterpart to the Code text view: selecting an element on the canvas syncs to the object tree and properties, so you can grasp page structure without reading source. It is the primary visual entry for human-and-AI collaboration at the component layer.

OPERATIONS

1. Switch to the Visual mode tab; the canvas renders the current Section's page
2. Click a visible element on the canvas (e.g. a task card) to select it
3. Watch the object tree and properties panel highlight in sync with the selection
4. Use the canvas to quickly verify an AI-generated page's overall layout matches intent
5. When source edits are needed, switch back to Code mode for a minimal change, then recheck in Visual

EXAMPLE / EXERCISE

After generating the TeamPM board, switch to Visual mode and the canvas renders a three-column board (Todo / In Progress / Done). Click a task card in the 'In Progress' column and the object tree highlights the matching TaskCard node while properties list its attributes — confirming the layout without reading the .sc source.

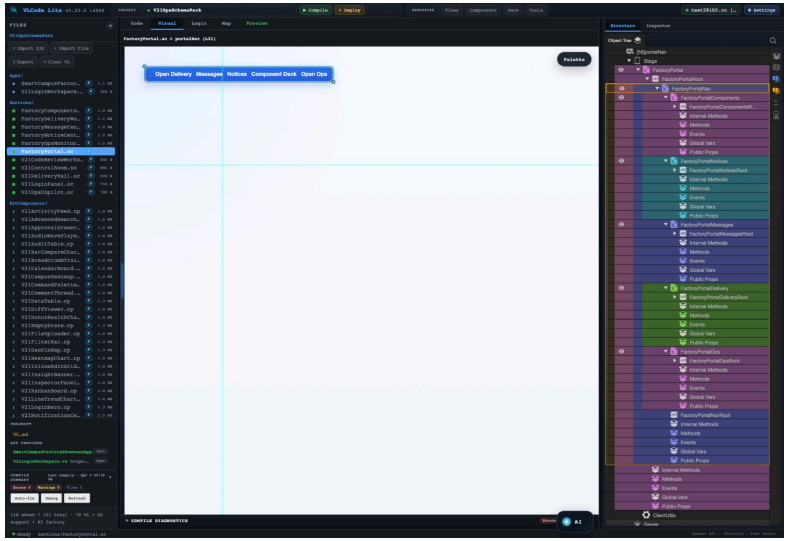
PITFALL

Mistaking the Visual canvas for a free-form WYSIWYG drag designer — it reflects the VL source structure, and real changes still happen in Code or the matching mode. 'Looking' only on the canvas while ignoring source leaves edits unapplied.

CHECKPOINT

You can switch to Visual mode, select a page element on the canvas, and see the object tree and properties panel respond in sync.

Object Tree And Selection Sync



Visual panel with object tree and properties

CORE IDEA

The object tree is the hierarchical list of a page's elements in Visual mode, bidirectionally synced with the canvas: selecting on the canvas highlights the matching tree node, and selecting a tree node selects it on the canvas and loads its attributes into the properties panel. This selection sync lets you move seamlessly between the visible page and the structured element hierarchy — an efficient way to target components.

OPERATIONS

1. Enter Visual mode and find the object tree panel on the right
2. Expand the hierarchy in the tree and select a node
3. Confirm the matching element is selected on the canvas and the properties panel shows its attributes
4. Do the reverse: select an element on the canvas and watch the tree auto-locate its node
5. Use the tree to reliably select hard-to-click elements in deeply nested pages

EXAMPLE / EXERCISE

On a CustomerCRM contact detail page, a contact card is hard to click on the canvas because other elements overlap it. Instead expand `ContactCard > PhoneField` in the object tree and click the PhoneField node — the canvas selects that field and the properties panel lists its attributes, more precisely than blind-clicking the canvas.

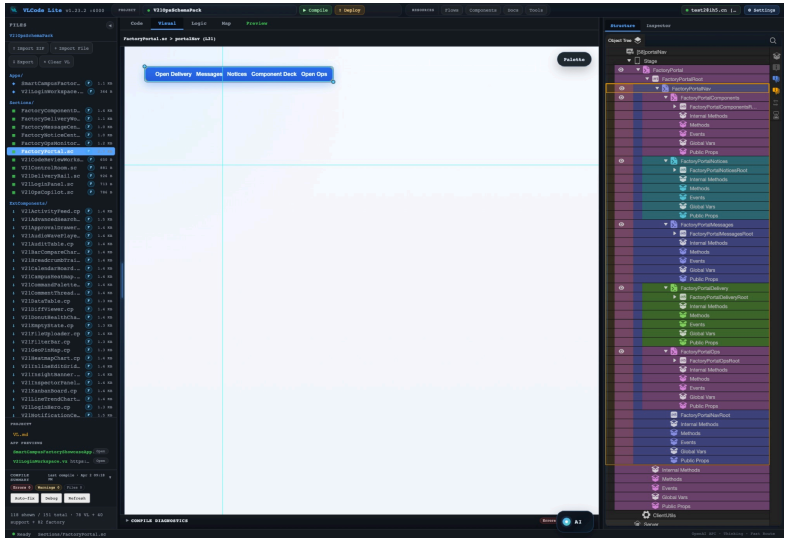
PITFALL

Treating the object tree and canvas as two separate selection systems — they are two views of one selection state; selecting in one place then re-selecting in the other is wasted effort and risks grabbing an adjacent element.

CHECKPOINT

You can select a node in the object tree and see the canvas and properties panel sync, and conversely auto-locate the node from a canvas selection.

Properties Panel



Visual panel with object tree and properties

CORE IDEA

The properties panel shows the attributes (Properties) of the currently selected component — the Properties entry in the component model's properties/methods/events triad. It refreshes in sync with the object-tree and canvas selection, letting you inspect an element's configurable attributes. In VLCode's human-AI collaboration model, it is a key panel where humans review and confirm AI output at the component layer.

OPERATIONS

1. In Visual mode, select a component via the canvas or object tree
2. Read the component's current attribute values in the properties panel
3. Verify the attributes match intent (e.g. bound field, displayed text)
4. When a change is needed, edit the VL source in Code mode, then re-check the properties panel
5. Select different elements to confirm the panel refreshes with each selection

EXAMPLE / EXERCISE

Selecting a TaskCard in the TeamPM board lists its bound attributes in the properties panel, such as the title text and status field. If the AI mistakenly bound the column to `priority` instead of `status`, the properties panel surfaces that deviation immediately so you can fix the binding in Code mode.

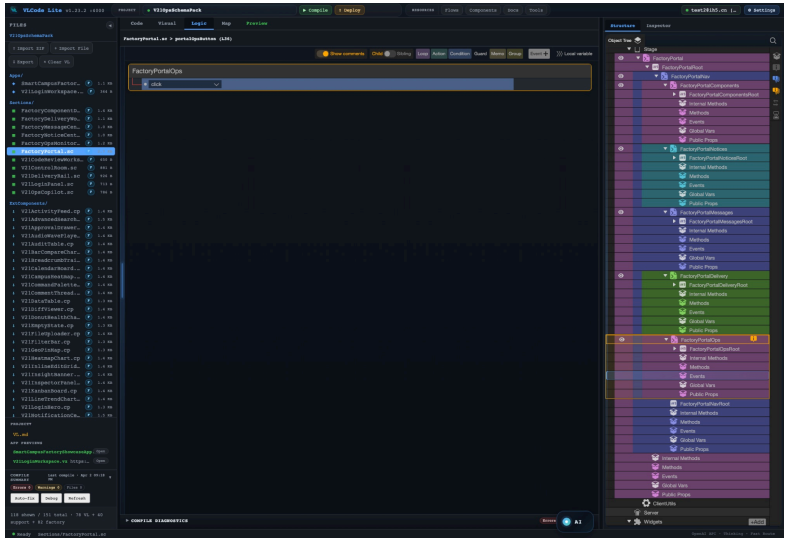
PITFALL

Treating the properties panel as the only edit path and expecting a 'save' there to change source — structural changes still live in the VL source; reading attributes without confirming in source risks missing the real binding issue.

CHECKPOINT

You can select a component, read its attributes in the properties panel, and judge whether the AI's binding is correct.

Logic Event Panel



Logic panel for event and action blocks

CORE IDEA

Logic mode is the event/action panel — the 'events' member of the component model triad. It associates an element's events (such as a click) with the actions to run, letting you inspect and edit interactions without hunting line-by-line through source. Front-end interaction logic is centralized here, decoupled from back-end Service logic.

OPERATIONS

1. Switch to the Logic mode tab to open the event/action panel
2. Select a page element and view the events attached to it
3. Expand an event to see the sequence of actions it triggers
4. Verify the interaction matches intent (e.g. does the click navigate to the detail page)
5. When adjustment is needed, edit the source, then re-check the event binding in Logic mode

EXAMPLE / EXERCISE

To check a task card's click behavior in TeamPM: enter Logic mode, select TaskCard, and verify its onClick event is bound to an action that opens the TaskDetail page. If the AI omitted the navigation, you immediately see the event has no matching action.

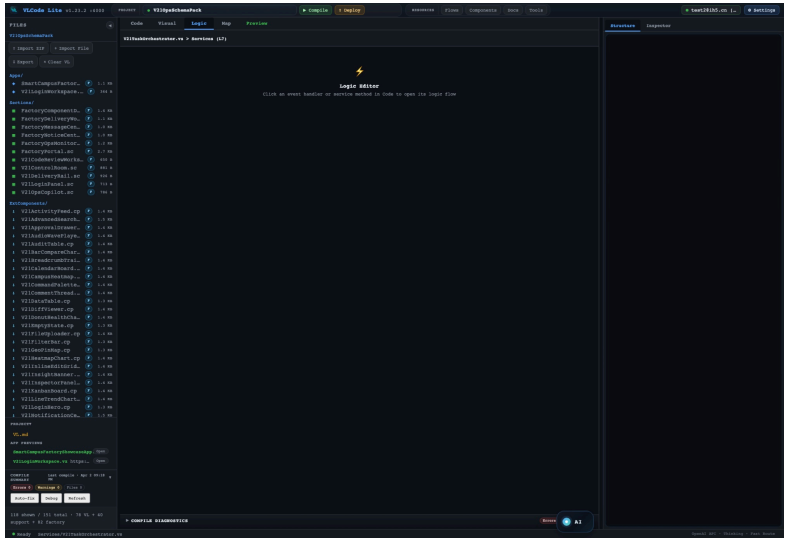
PITFALL

Conflating front-end interaction (Logic) with back-end business logic (Service) — click behavior is checked in the Logic panel, while data processing lives in the DB panel; looking in the wrong panel makes it seem 'logic is missing' when you just looked in the wrong place.

CHECKPOINT

You can select an element in Logic mode, read its events and actions, and judge whether a click's interaction is bound per the requirement.

ER And DB Data/Service Tabs

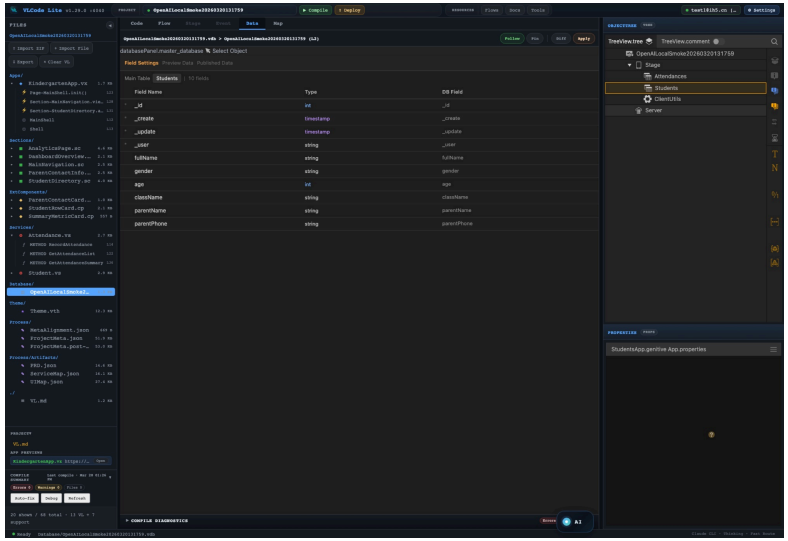


Service logic panel for backend behavior

CORE IDEA

(Content pending.)

Data Panel



Data panel for tables and schema

CORE IDEA

Data mode is the tables/schema panel, paired with .vdb database files. It shows the project's table structure — tables, fields, and their definitions — and is where you check that data modeling matches the requirement. Front-end bindings, service methods, and data tables must agree on field names, and the Data panel is the authoritative view for reconciling them.

OPERATIONS

1. Switch to the Data mode tab to open the data panel
2. Open the .vdb file and browse the table and field list
3. Verify field names and types match what front-end bindings and service methods expect
4. Confirm the schema covers the entities in the requirement (e.g. task, user, status)
5. When field drift appears, locate the specific field and make a minimal correction

EXAMPLE / EXERCISE

Inspecting the Task table in TeamPM's Data panel, it should contain fields like `title`, `status`, `assignee`. If board columns group by `status` but the field is actually named `state` in the Data panel, that is field drift; treat the Data panel as the source of truth and align the front-end binding and service method names to it.

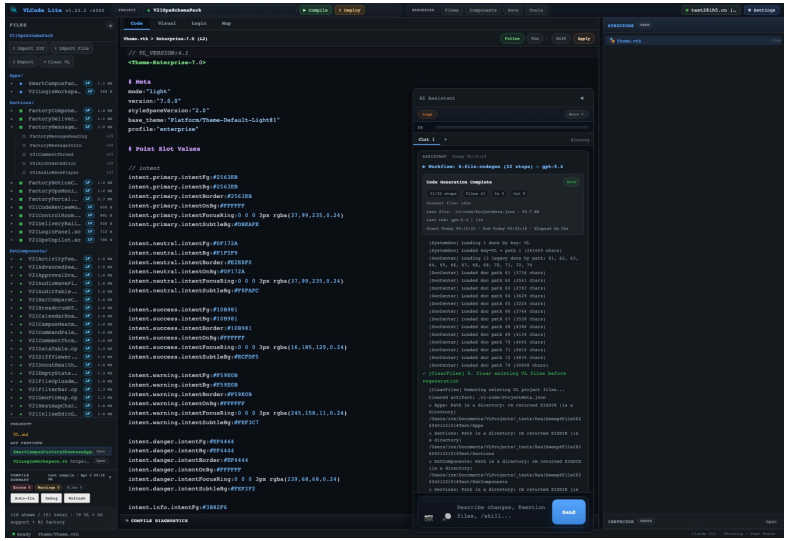
PITFALL

Front end, service, and data table each using a different field name (e.g. status vs state) causes 'field drift' — bindings look populated but don't line up; make the Data panel's schema the single source of truth and unify field names against it.

CHECKPOINT

You can open a .vdb in Data mode, read a table's fields, and confirm the front end and service use matching field names.

THEME Panel



Theme panel for enterprise visual tokens

CORE IDEA

THEME mode is the visual token workbench, paired with .vth theme files. It centralizes visual tokens like colors and fonts, decoupled from front-end structure and back-end logic, so the whole app's visual style stays unified and controllable. In VLCode the theme is an independent layer — changing it does not touch business logic, and components pull values from tokens.

OPERATIONS

1. Switch to the THEME mode tab to open the token workbench
2. Open the project's .vth file and browse the defined visual tokens
3. Check that every token components actually use is defined in the theme
4. Verify color/font tokens match the design requirement
5. To unify the look, use the theme-customize workflow rather than hard-editing each component

EXAMPLE / EXERCISE

The TeamPM three-column board needs a consistent primary color. Open Theme/TeamPM.vth and confirm the primary token (e.g. the brand color) is defined and components reference it. If one card hard-codes a color instead of using the token, it looks 'inconsistent' on theme switch — point it back at the theme token to unify it.

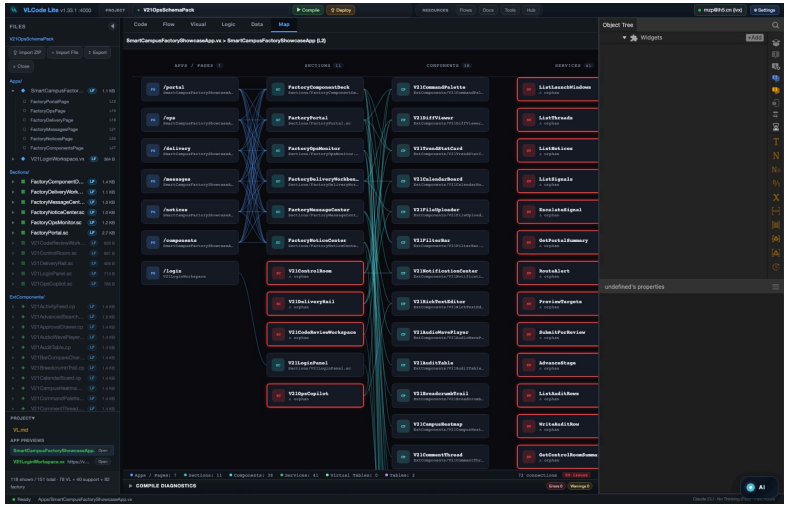
PITFALL

Components hard-coding colors and bypassing theme tokens cause a 'theme/component mismatch' — on a theme switch some elements don't follow; keep components referencing .vth tokens and avoid scattered hard-coded styles.

CHECKPOINT

You can open a .vth in the THEME panel, read its key visual tokens, and confirm components use tokens rather than hard-coded values.

Map Project View



Showcase: map mode

CORE IDEA

Map mode is the project map, a visualization of ProjectMeta — the project's machine-readable global picture: apps, Sections, services, data, theme, and the dependencies between them. It gives a whole-system view of how the pieces connect and is the bird's-eye entry for finding which part broke. Generation and repair revolve around ProjectMeta, and Map is its visual surface.

OPERATIONS

1. Switch to the Map mode tab to open the project map
2. Survey the project's apps, pages, services, data, and theme from the map
3. Follow dependencies to see which services a page calls and which tables a service touches
4. Use the map to scope which related files a change will affect
5. When chasing a root cause, return to the matching source file and ProjectMeta to verify

EXAMPLE / EXERCISE

In CustomerCRM's Map mode you see the customer list page depends on CustomerService, which reads/writes the Customer table. If the list is blank, walk this dependency chain segment by segment — page → service method → data table — far more efficient than blindly opening files.

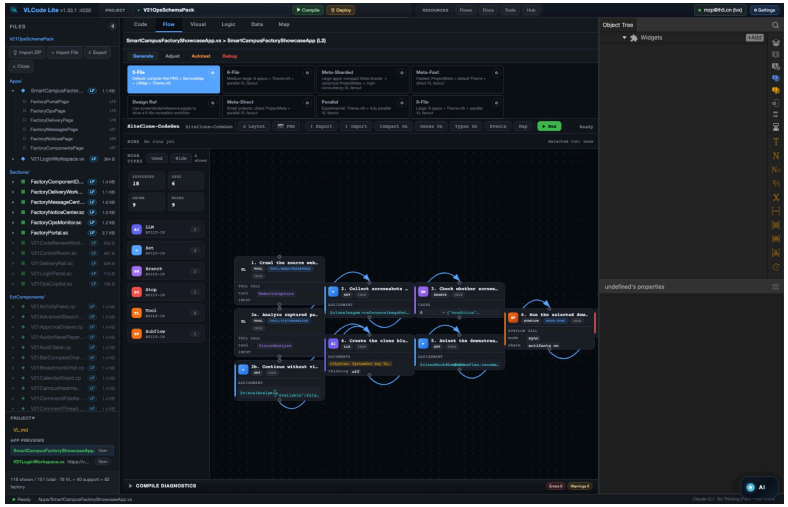
PITFALL

Treating Map as a static structure diagram and ignoring the dependencies it shows — its value is exactly the connections; looking at nodes but not edges misses breaks like 'a page references a non-existent service'.

CHECKPOINT

You can start from a page in Map mode and follow its dependencies to the service it calls and the data tables involved.

Flow Workflow Graph



Showcase: flow mode

CORE IDEA

Flow mode is the DAG workflow graph, presenting generation/repair/adjust workflows as a directed acyclic graph of nodes and edges. You import a .vflow into the Flow panel by drag-drop or the '+ Upload .vflow' button, choosing one of 8 categories on import (Generate / Enterprise / Clone / MetaData / Test / Adjust / Autofix / Quality). A .vflow is a connector bundle (a zip of workflow JSON + manifest + support docs/prompts), not a VL source file. This is the surface in VLCode that hosts workflows shipped over from VL Agent via the Resource Center.

OPERATIONS

1. Switch to the Flow mode tab to open the workflow DAG graph
2. Import a .vflow into the Flow panel by drag-drop or the '+ Upload .vflow' button
3. Choose a category for it in the import dialog (e.g. Adjust or Autofix)
4. Inspect the workflow's nodes and execution order on the DAG graph
5. After a run, confirm states settle reliably into the done/error/aborted terminal state

EXAMPLE / EXERCISE

Given an approval-flow .vflow shipped from VL Agent, click '+ Upload .vflow' in Flow mode and pick the category Enterprise on import. The DAG graph then shows the workflow's node chain; run it and the state settles to done (or error/aborted) instead of hanging in an intermediate state.

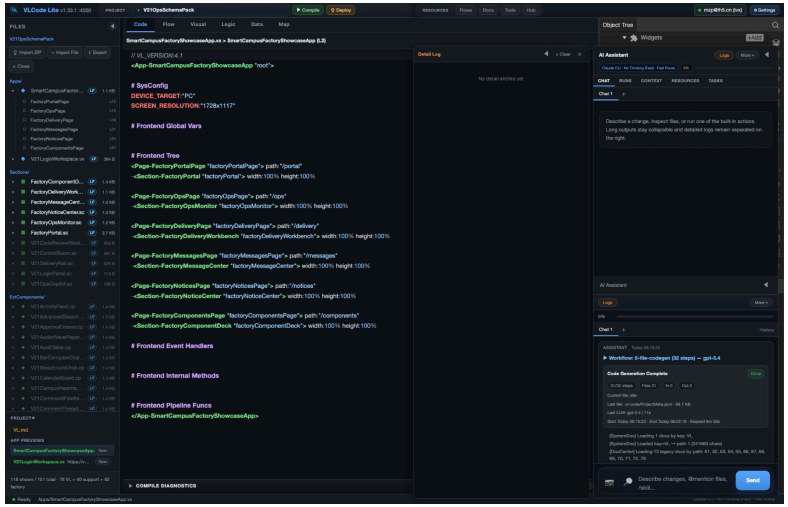
PITFALL

Carelessly picking the wrong category on import puts the .vflow under a mismatched bucket and makes it hard to find — choose the category by the workflow's real purpose (generate/adjust/debug, etc.).

CHECKPOINT

You can import a .vflow with '+ Upload .vflow', pick the correct category, and see its DAG graph in the Flow panel.

AI Chat Main Conversation



AI Chat with detailed run context

CORE IDEA

AI Chat is VLCode's primary interaction entry and its workflow control console: you issue natural-language commands, and final summaries plus human-decision moments appear here. It is strictly divided from the right-side Detail Log — per-node progress, file writes, tool I/O and the full run ledger go to Detail Log only and are never repeated in chat. Chat is for 'see results and decide'; Detail Log is for 'see the whole process'.

OPERATIONS

1. Type an executable requirement or command in natural language in the AI Chat input
2. Kick off a generation/repair workflow so the run's final summary returns to Chat
3. Handle human-decision interaction cards in Chat
4. Leave per-node detail to Detail Log — don't look for the ledger in Chat
5. Use the bottom status bar alongside to see the current step, provider/model, and compile status

EXAMPLE / EXERCISE

Write the requirement as an executable goal in Chat:

```
Generate a team task board TeamPM: three columns (Todo/In Progress/Done), tasks movable between columns, click a task to open its detail page; back end stores title, status, assignee in one Task table.
```

After the run, Chat returns the final summary and any confirmation cards, while per-node file-write detail lives in Detail Log.

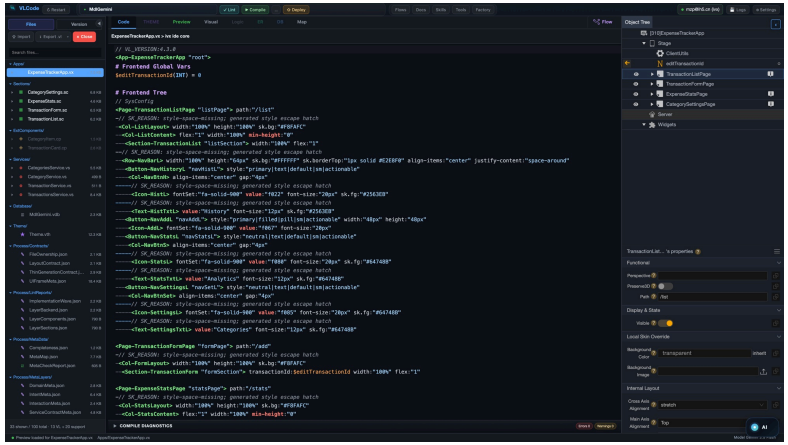
PITFALL

Treating Chat as a log and scrolling for every step — per-node progress and file writes simply aren't here; to see the whole process go to Detail Log, scrolling Chat finds nothing.

CHECKPOINT

You can start a workflow from AI Chat in natural language, receive the final summary in Chat, and know to read detail in Detail Log.

Interactive Cards



AI Chat used as the command and decision surface

CORE IDEA

Interactive Cards are structured cards rendered in AI Chat. There are 12 card kinds: run-card, llm-live-card, tool-live-card, node-result-card, artifact-card, review-card, form-card, picker-card, context-card, resource-card, requirement-card, binding-card. The blocking cards (review-card / form-card / picker-card) gate your input and require a choice or decision; the live cards (llm-live-card / tool-live-card / node-result-card, etc.) are just a passive feed. Note: ask_user / human_gate are chat events (SSE), not card kinds. Together they turn 'human gating' into a structured interaction entry rather than scattered free text.

OPERATIONS

1. While a workflow runs, watch for interactive cards appearing in Chat
2. On a picker-card, pick a given option (the ask_user chat event surfaces the options) or take the custom-answer path
3. On a review-card, choose approve / reject / request_changes (human_gate arrives as a chat event)
4. On a form-card, fill the fields and submit
5. After clicking an action button (e.g. Deploy), complete workspace binding in the binding-card that follows

EXAMPLE / EXERCISE

While generating TeamPM, a review-card appears asking you to confirm the AI's proposed data table design. You inspect it, find a missing field, click `request_changes`, and write in the custom answer:

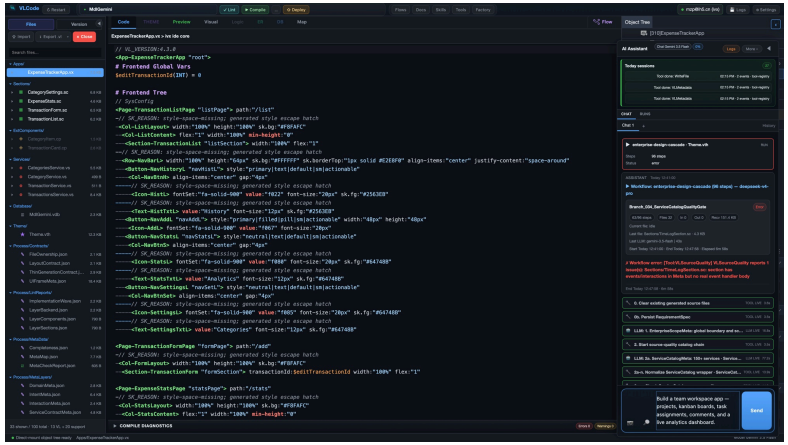
```
Please add a dueDate (due date) field to the Task table
```

The workflow adjusts and returns for confirmation.

PITFALL

Ignoring a blocking card as if it were an ordinary message — it blocks further input and workflow progress; a review-card / form-card / picker-card must be answered or filled for the run to continue (live cards are a passive feed and need no action).

Detail Log



Detail Log with node progress and tool output

CORE IDEA

Detail Log is the right-side panel and the complete ledger of a run: node progress, file writes, tool input/output, validation results, and errors all live here — never repeated in AI Chat. It is the authoritative source for finding which step failed and which files were written, in clear division from Chat's 'results + decisions'. Reading errors, reconciling files, and tracing tool calls all start here.

OPERATIONS

1. Open the right-side Detail Log panel and follow the run's per-node progress
2. Reconcile which files the workflow actually wrote in the file-writes records
3. Expand a node's tool I/O to inspect its input and output
4. Read Lint/Compile reports and errors in the validation results section
5. If the same error repeats twice, return to source and ProjectMeta for the root cause instead of broad rewrites

EXAMPLE / EXERCISE

Preview errors out after generating TeamPM. Open Detail Log; in validation results a Compile error points to a method in TaskService.vs, and the file-writes records confirm that file was written. From this you make a minimal fix in the DB panel rather than re-running the entire generation.

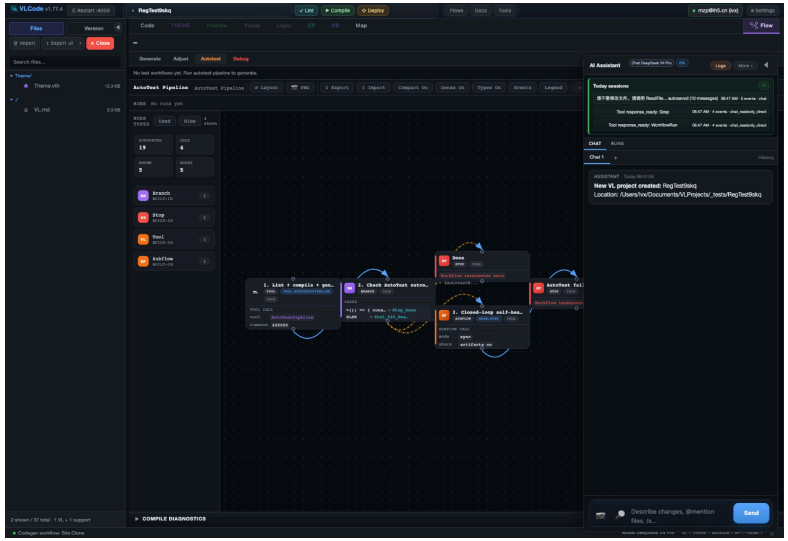
PITFALL

Looking for node progress or file writes in Chat — those are only in Detail Log; another mistake is recompiling repeatedly while the same error recurs, when the right move on the second repeat is to return to source for the root cause.

CHECKPOINT

You can locate a specific error in Detail Log, find the related file-write record, and decide the minimal fix point from it.

Status Bar



Logs and execution details after a run

CORE IDEA

The bottom status bar is the at-a-glance state line, showing the current step, the LLM provider/model in use, compile status, and ports. It condenses run state scattered across panels into one line, so you know which step is running, which model is in use, and whether compile passed without switching panels. It's a lightweight status reference beyond Chat and Detail Log.

OPERATIONS

1. Glance at the bottom status bar during a run to read the current step name
2. Confirm the provider/model shown matches what's selected in Settings
3. Use the status bar's compile status to judge whether validation passed
4. Check the port in the status bar (Web IDE defaults to 4000, range 4000–4010)
5. Pair status bar with Detail Log: status bar for 'right now', Detail Log for 'the whole run'

EXAMPLE / EXERCISE

While generating TeamPM, the status bar may show the current step, the provider/model (cost-sensitive defaults: DeepSeek V4 Pro for chat, Kimi 2.5 for workflow), plus compile status and port 4000. If the model isn't the one you expected, recheck the provider in Settings → LLM.

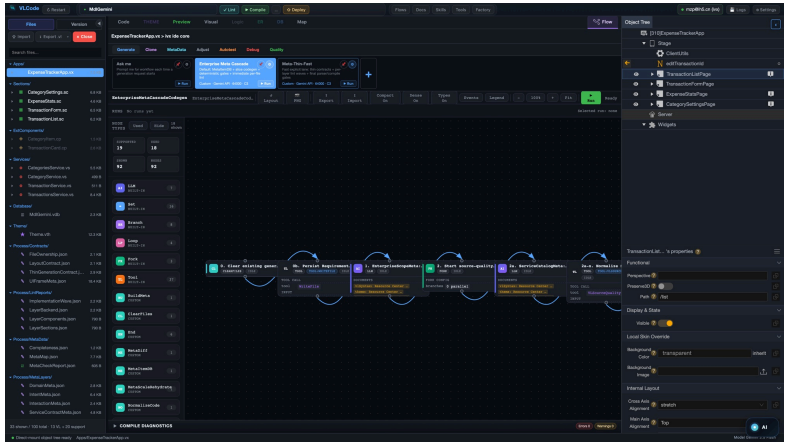
PITFALL

Ignoring the provider/model shown in the status bar and unknowingly running on the wrong model — in cost-sensitive setups especially, verify it matches Settings; the port too is per the status bar (fixed port, no silent fallback).

CHECKPOINT

You can read the current step, provider/model in use, compile status, and port from the status bar and confirm the run environment is correct.

Multi-Task Concurrency And Abort Scope



Flow mode: a DAG workflow shown as executable nodes

CORE IDEA

VLCode supports multi-task concurrency: one IDE window = one workspace = one backend, but multiple chat sessions and multiple workflow runs can run in parallel underneath it. Aborting has two scopes — chat scope (stops this chat plus the runs it started) or workflow scope (stops a single run). Understanding this lets you advance several threads at once yet cut losses precisely.

OPERATIONS

1. Open several chat sessions in parallel within one IDE window
2. Start multiple workflow runs in parallel from different sessions
3. Track each run's own progress and terminal state in Detail Log
4. To cut losses, choose chat-scope abort (stops that chat and its runs)
5. Or choose workflow-scope abort to stop only one specific run

EXAMPLE / EXERCISE

In the TeamPM workspace, session A runs a generation while session B adjusts the theme. You notice session A's generation has gone off-track, so you use chat-scope abort to stop A and its runs, while session B's theme adjustment keeps running unaffected. If only one run inside A needs stopping, use workflow-scope abort.

PITFALL

Wanting to stop one off-track run but using chat-scope abort, which also kills other healthy runs in that session — distinguish 'stop the whole chat' from 'stop one workflow' and pick the scope deliberately.

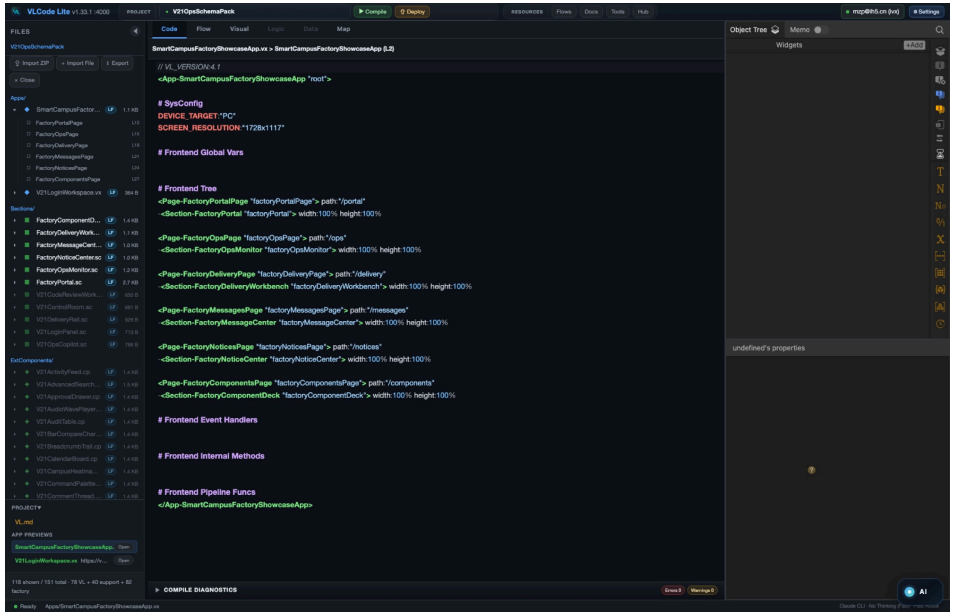
CHECKPOINT

You can run two runs in parallel in one workspace and use the correct abort scope (chat or workflow) to stop only the target run without affecting the rest.

CHAPTER

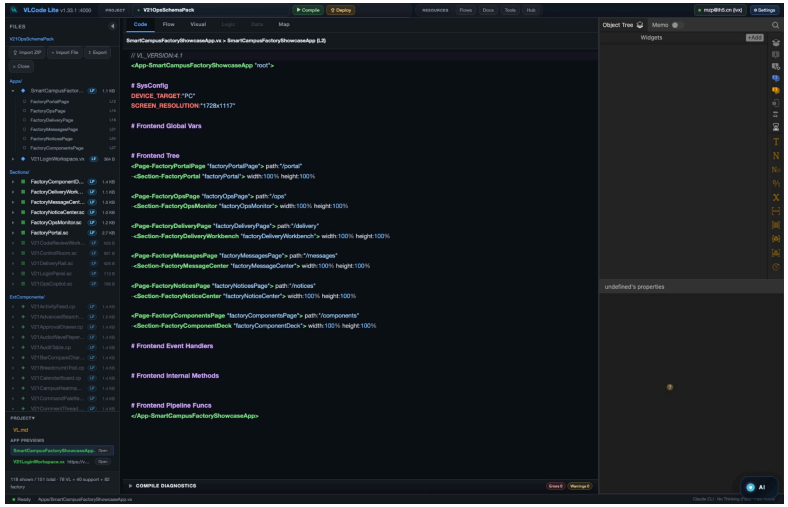
Part IV: VL Project And Language Basics

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



Code mode for inspecting and editing VL source

VL File Types Overview



Code mode for inspecting and editing VL source

CORE IDEA

A VL project is composed of seven single-responsibility, compilable source file types: .vx (app entry), .sc (page or region / Section), .cp (pure UI Component, no business logic), .wc (WebComponent — a project-local native web extension under ExtComponents/), .vs (backend service logic), .vdb (database tables/schema), and .vth (theme). A .vflow is NOT a VL source file — it is a workflow / agent-app connector bundle (a zip of workflow JSON + manifest + support docs/prompts), imported via bundle tooling and never compiled by the VL parser, so don't list it alongside .vx/.sc as a source type. Each source type expresses exactly one concern, and that separation is what lets the AI generate files in parallel and repair them surgically instead of rewriting everything. Knowing this type map is the prerequisite for reading any generated output.

OPERATIONS

1. Expand the project in the left file tree and observe how files sort into Apps/ Sections/ Services/ Database/ Theme/ ExtComponents/
2. Open a .vx file and confirm it is the application entry, not a page
3. Open a .sc file and switch between Code and Visual modes to confirm it maps to one page
4. Open a .vdb in Data mode and a .vs in DB mode to feel the frontend/backend split
5. Open a .vth in THEME mode and confirm theme tokens are decoupled from pages

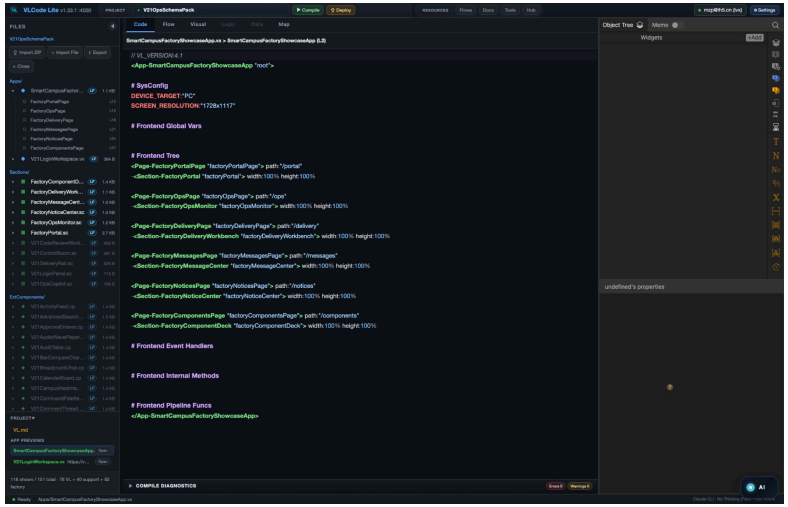
EXAMPLE / EXERCISE

A typical layout for a TeamPM task board:

```
Apps/TeamPM.vx              # application entry
Sections/Board.sc           # board page
Sections/TaskDetail.sc      # task detail page
Services/TaskService.vs     # task CRUD backend logic
Database/TeamPM.vdb         # task / member tables
Theme/TeamPM.vth            # color, spacing tokens
ExtComponents/TaskCard.cp   # pure UI component (.wc native web extensions sit here too)
```

PITFALL

VL_VERSION Must Be The First Line



Showcase: code mode

CORE IDEA

The very first line of every VL source file must be the VL_VERSION declaration, with nothing before it — no blank line, no comment, no character. It tells the compiler which syntax-rules version to parse the file with, and it is the entry point the platform parse/compile chain uses to recognize the file. A missing or misplaced version line makes the whole file unparseable.

OPERATIONS

1. When creating any .vx/.sc/.cp/.wc/.vs/.vdb/.vth file, put VL_VERSION on the physical first line
2. Confirm there is no BOM, blank line, comment, or any character above VL_VERSION
3. When you need the current syntax version, refresh the SysDoc VL doc per CLAUDE.md to get the latest number
4. After editing, run lintPjt for a project-level static check and confirm no version-line errors

EXAMPLE / EXERCISE

Correct (version must be the physical first line):

```
// VL_VERSION:4.3.1
- App TeamPM
-- Section Board
```

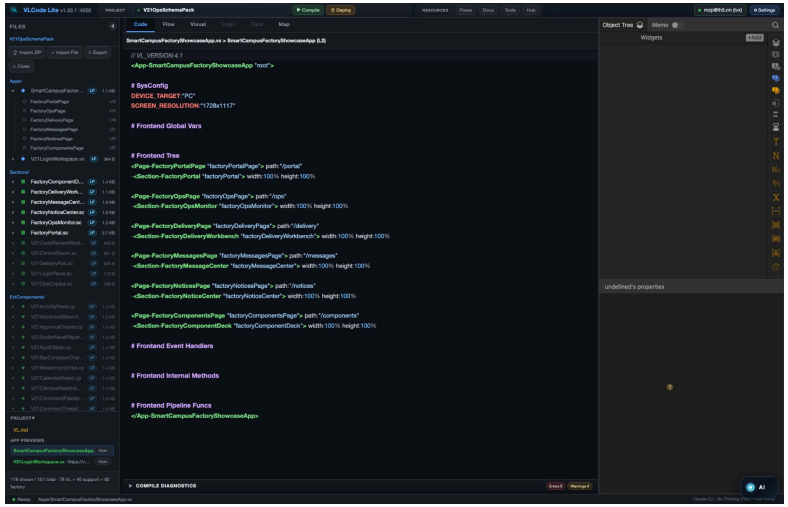
Wrong (a comment above the version line breaks parsing):

```
// Team board entry
// VL_VERSION:4.3.1
- App TeamPM
```

PITFALL

The most common mistake is adding a descriptive comment or leaving a blank line above the version line — even one invisible blank line makes the file fail to parse. Ensure VL_VERSION is truly the physical first line.

Indentation Uses Consecutive Hyphens



Code mode for inspecting and editing VL source

CORE IDEA

VL expresses hierarchy with consecutive hyphens, not spaces or tabs: one - is one level, -- the next, --- the next again. This explicit level marker gives AI-generated structure clean boundaries and stable parsing. The moment you mix space indentation with hyphens, you get the kind of seemingly inexplicable ("strange") errors VL warns about.

OPERATIONS

1. When editing in Code mode, use - / -- / --- for parent-child levels, adding one hyphen per level of depth
2. Avoid using spaces or tabs to express indentation level
3. When a baffling hierarchy error appears, first check whether space indentation got mixed with hyphens
4. Run lintPjt after editing to confirm the hierarchy parses correctly

EXAMPLE / EXERCISE

Hyphens express the App → Section → control hierarchy:

```
// VL_VERSION:4.3.1
- App TeamPM
-- Section Board
--- Title "Task Board"
--- List taskList
```

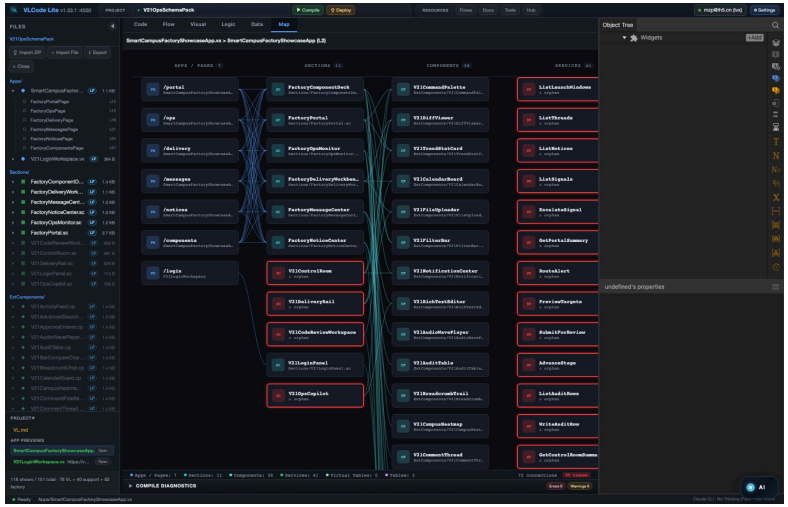
Each level is distinguished only by hyphen count, with no leading spaces.

PITFALL

The subtlest trap is sneaking a space before the hyphens (to make things look more aligned), which mis-parses the hierarchy and produces hard-to-locate errors. Stick to pure hyphens and never mix in spaces.

CHECKPOINT

App Files .vx



Map mode for application structure

CORE IDEA

.vx is the entry file of a VL app, living under the project's Apps/ directory. It defines the application itself — which Sections assemble into the whole app, plus app-level structure and dependencies — and does not carry the detail of any single page. It is the top-level node through which ProjectMeta wires the whole project together.

OPERATIONS

1. Find <PascalCaseName>.vx in the Apps/ directory of the file tree
2. In Code mode confirm line one is VL_VERSION and the body starts with - App
3. In Map mode see how this .vx connects to each Section
4. When you need an overview of app composition, start from the .vx and follow its references downward

EXAMPLE / EXERCISE

Apps/TeamPM.vx is the entry that assembles the board and detail pages:

```
// VL_VERSION:4.3.1
- App TeamPM
-- Section Board
-- Section TaskDetail
```

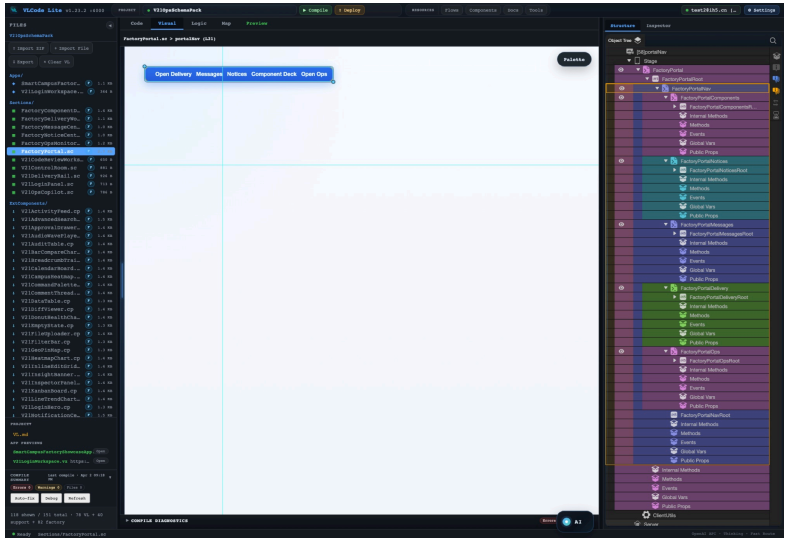
PITFALL

Do not put page controls or backend logic inside a .vx. A bloated entry file makes it hard for the AI to scope changes and easy to mistake a page-level issue for an app-level one.

CHECKPOINT

You can locate the project's entry .vx under Apps/ and state which Sections it assembles.

Section Files .sc



Visual panel with object tree and properties

CORE IDEA

.sc is a Section file representing a page or region of the app, living under Sections/. It carries that page's layout and controls and is the smallest visible unit a route can point to. In VLCode the same .sc can be seen as source in Code mode and as a canvas in Visual mode — two views of one underlying data.

OPERATIONS

1. Open a .sc file under the Sections/ directory
2. In Code mode confirm it starts with -- Section and contains page controls
3. Switch to Visual mode and observe the same page's layout on the canvas
4. Select a control in the object tree and confirm Code and Visual selection stay in sync

EXAMPLE / EXERCISE

Sections/Board.sc describes the board page:

```
// VL_VERSION:4.3.1
-- Section Board
--- Title "Task Board"
--- List taskList
---- Item card
```

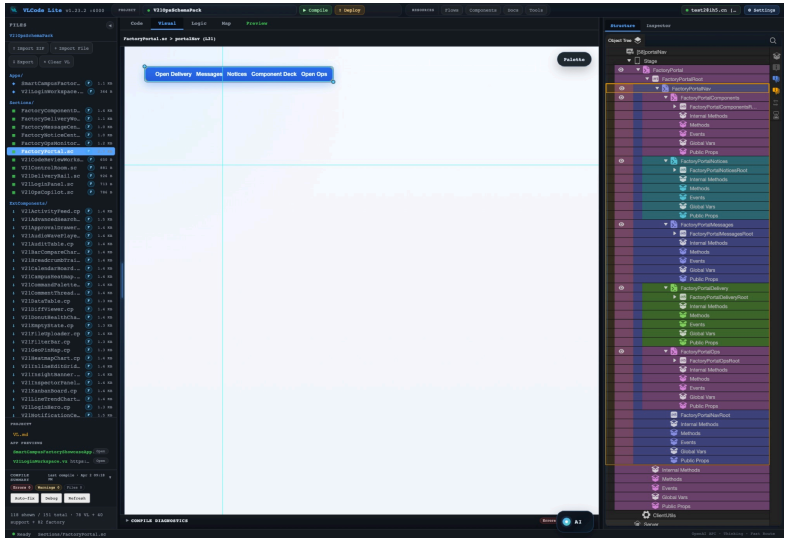
PITFALL

Do not write backend data-query logic directly into a .sc — data should come from a .vs service. Mixing service logic into the page couples frontend and backend and breaks the maintainability the small-file strategy buys you.

CHECKPOINT

You can open a .sc and confirm in both Code and Visual modes that they render the same page.

Component Files .cp And .wc

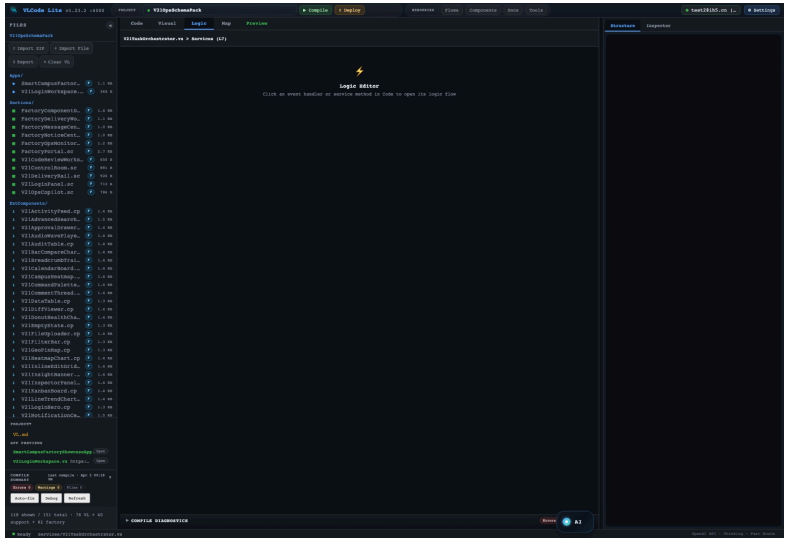


Visual panel with object tree and properties

CORE IDEA

(Content pending.)

Service Files .vs



Service logic panel for backend behavior

CORE IDEA

.vs is a Service file carrying the app's backend logic, living under Services/. It defines the service methods pages call — CRUD, business rules, operations on data tables — keeping backend concerns fully separate from pages (.sc). In VLCode you view and edit this service logic specifically in DB mode (the data/service layer is split in the mode bar into the ER and DB tabs).

OPERATIONS

1. Open a .vs file under the Services/ directory
2. Switch to DB mode to view service methods and their logic
3. Confirm the service methods operate on tables defined in a .vdb
4. Call service methods from .sc pages rather than writing data logic inside the page

EXAMPLE / EXERCISE

Services/TaskService.vs exposes task-related service methods:

```
// VL_VERSION:4.3.1
- Service TaskService
-- Method listTasks
-- Method createTask
-- Method updateStatus
```

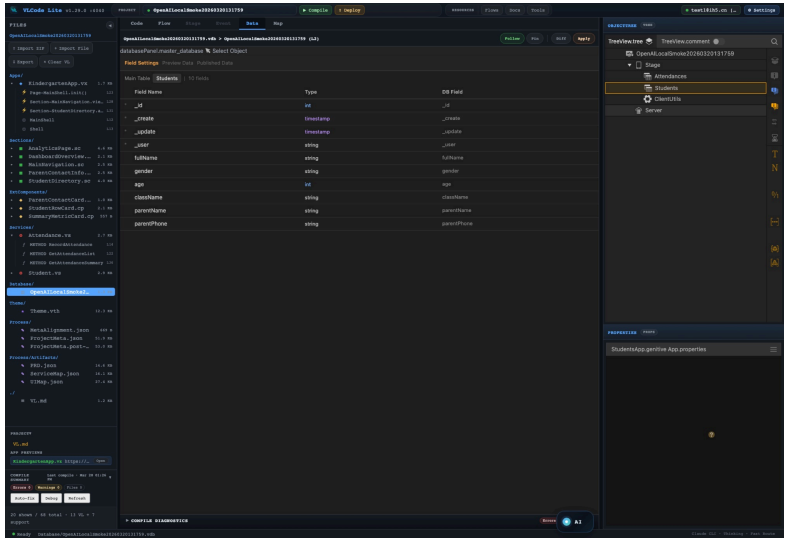
PITFALL

Renaming a service method or changing its signature without updating the page's call site produces a "broken service method reference." After renaming, verify references in Map mode so every caller is updated.

CHECKPOINT

You can open a .vs in DB mode, list the service methods it exposes, and identify which page calls them.

Database Files .vdb



Data panel for tables and schema

CORE IDEA

.vdb is a Database file defining the app's tables and schema, living under Database/. It describes which tables exist and each table's fields and types, and it is the source of truth that services (.vs) operate against. In VLCode you view and edit table structure specifically in Data mode.

OPERATIONS

1. Open a .vdb file under the Database/ directory
2. Switch to Data mode to view tables, fields, and types
3. Verify the fields referenced by .vs service methods actually exist here
4. When adding a field, check whether related service methods need updating

EXAMPLE / EXERCISE

Database/TeamPM.vdb defines task and member tables:

```
// VL_VERSION:4.3.1
- Database TeamPM
-- Table task
--- Field title text
--- Field status text
--- Field assignee text
-- Table member
--- Field name text
```

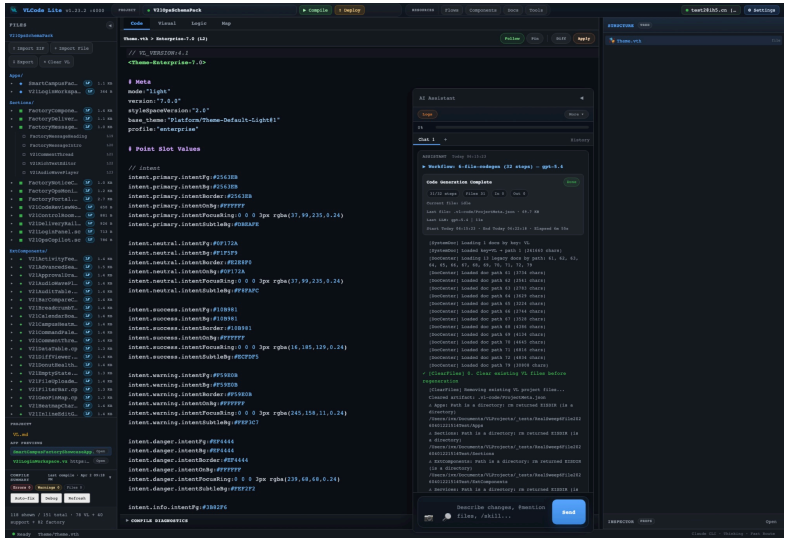
PITFALL

Renaming a table field (data field drift) without updating the service methods that reference it leaves services unable to fetch the field at runtime. After a field change, always return to the .vs and verify every reference.

CHECKPOINT

You can open a .vdb in Data mode, list its tables and fields, and confirm a field a .vs method references actually exists.

Theme Files .vth



Theme panel for enterprise visual tokens

CORE IDEA

.vth is a Theme file defining the app's visual tokens — colors, spacing, typography, and other design variables — living under Theme/. It decouples visuals from page structure: components reference tokens instead of hard-coding values, so one theme edit unifies the global look. In VLCode you view and adjust these tokens in THEME mode (the token workbench).

OPERATIONS

1. Open a .vth file under the Theme/ directory
2. Switch to THEME mode (the token workbench) to view color, spacing, and other tokens
3. Confirm page components reference theme tokens rather than hard-coded values
4. After adjusting a token, preview whether pages update consistently

EXAMPLE / EXERCISE

Theme/TeamPM.vth defines the board's visual tokens:

```
// VL_VERSION:4.3.1
- Theme TeamPM
-- Token colorPrimary "#2D6CDF"
-- Token spacingMd "16px"
-- Token radius "8px"
```

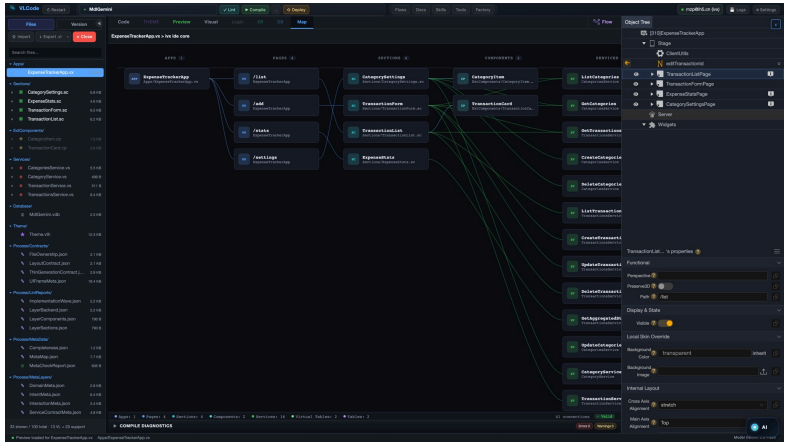
PITFALL

Hard-coding colors in a component instead of referencing theme tokens causes a "theme/component mismatch" — you edit the .vth but some places do not follow. Keep components referencing tokens and avoid scattered hard-coded styles.

CHECKPOINT

You can adjust one token in THEME mode and see multiple pages that reference it update consistently.

Purpose Of ProjectMeta



Map mode for project structure and relationships

CORE IDEA

ProjectMeta is the machine-readable map of the whole project, stored at .vl-code/ProjectMeta.json ($schema VL-ProjectMeta/3.0, Metadata Spec 3.2). Its canonical root keys are: apps, sections, components, services, dataSchema, theme, dependencyGraph (plus fileManifest, valueDomains) — note it is dataSchema (not data), dependencyGraph (not dependencies), and it includes components. VLCode's generation and repair revolve around it: the AI reads ProjectMeta to understand the whole before deciding which file to change. It is the source of truth that turns a pile of separate files into a coherent system.

OPERATIONS

1. Switch to Map mode and read ProjectMeta as the project overview
2. Follow the App → Section → Service → data connections to understand dependencies
3. Before generating or repairing, confirm ProjectMeta matches the actual files
4. When the same error repeats twice, return to ProjectMeta and source to find root cause instead of blindly rewriting

EXAMPLE / EXERCISE

Anchoring a question on ProjectMeta in AI Chat is more reliable than chasing files one by one:

```
"Per ProjectMeta, which pages call TaskService.updateStatus? Which Sections would changing it affect?"
```

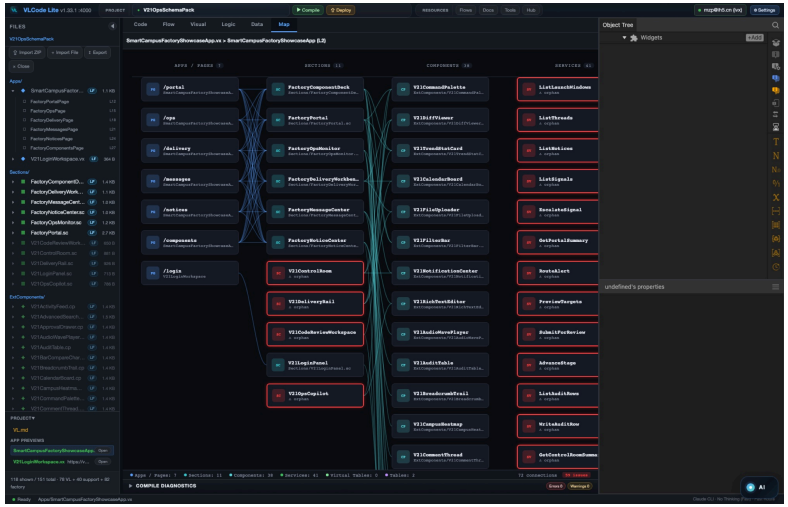
PITFALL

Bypassing ProjectMeta and editing a single file in isolation easily misses cross-file dependencies, so fixing A breaks B. Assess the blast radius of any structural change against ProjectMeta first.

CHECKPOINT

You can read in Map mode which Sections depend on a given Service and judge a change's impact accordingly.

Routes, Pages, And Sections



Map mode for application structure

CORE IDEA

In VL a Section (.sc) is a page-level unit, and a route points at a Section to determine which page opens in preview. VLCode's preview reaches a page via the /route/<path> form. Understanding the route → Section → page correspondence is fundamental to locating preview content and navigation behavior.

OPERATIONS

1. In Map mode see which Sections are mounted under the App
2. Confirm every externally reachable page has a corresponding Section
3. Use Preview to open a Section's page via /route/<path>
4. Verify the address path matches the Section you opened

EXAMPLE / EXERCISE

TeamPM's board page opens through its preview route:

```
Preview: /route/board   ->  opens the page for Sections/Board.sc
```

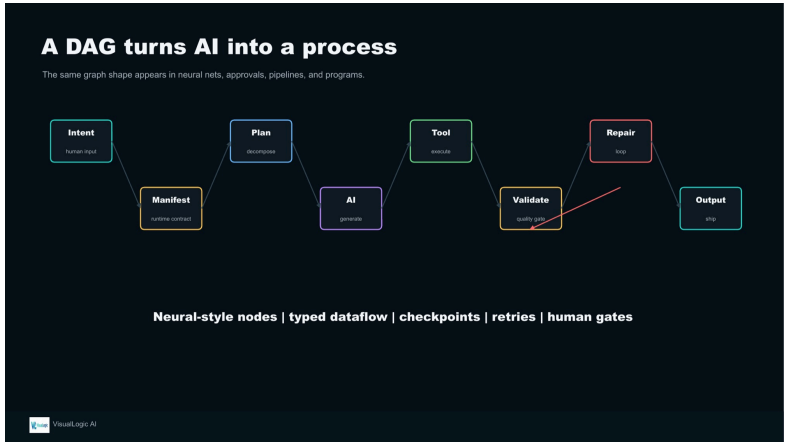
PITFALL

Adding a Section without mounting it in navigation or routing leaves preview with no entry point. After adding a page, confirm it is mounted and reachable via /route/<path>.

CHECKPOINT

You can name the preview route for a given .sc and open it successfully via /route/<path>.

Decoupled Frontend, Backend, And Theme



DAG harness: feedback loop

CORE IDEA

VL decouples three things at the file level: frontend (.sc/.cp pages and components), backend (.vs services + .vdb data), and theme (.vth visual tokens). Pages only call services and reference tokens — they neither embed database logic nor hard-code styles. This decoupling lets the three dimensions be generated and edited independently, so a change in one does not ripple into the others.

OPERATIONS

1. To change appearance, touch only .vth theme tokens, not page structure
2. To change data logic, touch only .vs services and .vdb tables, not pages
3. To change layout, touch only .sc/.cp, calling services and referencing tokens rather than inlining
4. In Map mode verify the three layers relate by call/reference, not mutual embedding

EXAMPLE / EXERCISE

For one board, three kinds of change land in three kinds of file:

```
change primary color -> Theme/TeamPM.vth   (edit colorPrimary token)
change fetch logic   -> Services/TaskService.vs (edit listTasks)
adjust card layout   -> Sections/Board.sc   (edit layout, still calls listTasks)
```

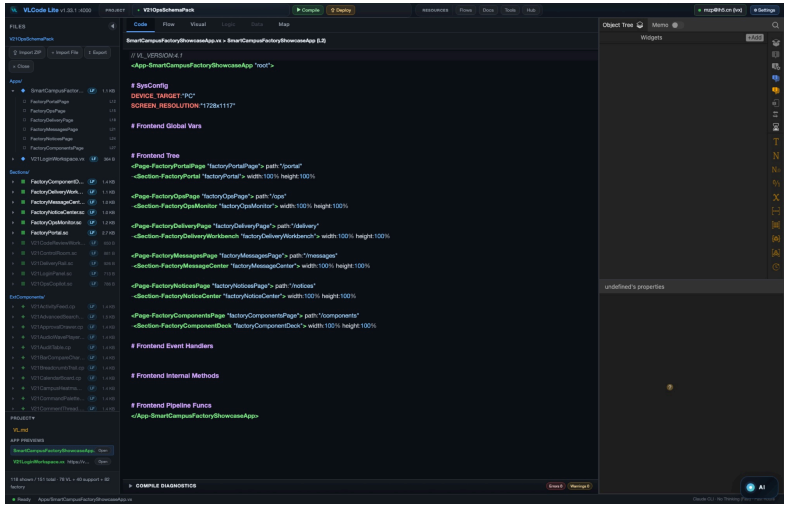
PITFALL

Hard-coding colors in pages or stuffing SQL-style data logic into a .sc re-couples the layers and loses the maintainability decoupling buys. Always keep pages calling services and referencing tokens.

CHECKPOINT

Given an appearance/data/layout change request, you can name exactly which file type to edit without touching the other two.

Small-File Strategy



Code mode for inspecting and editing VL source

CORE IDEA

The small-file strategy means splitting a project into many single-responsibility, modestly sized files instead of a few giant ones. VL's type split (.vx/.sc/.cp/.wc/.vs/.vdb/.vth) supports this naturally: smaller, more focused files let the AI generate in parallel, repair surgically, and Debug a single file precisely. This is the structural prerequisite for VLCode's "generate and repair fast."

OPERATIONS

1. When generating, prefer splitting pages, services, and components into several small files over one big file
2. When a file takes on multiple responsibilities, split it by type
3. When repairing, prefer single-file Debug to keep the change scope in the smallest file
4. Avoid broad rewrites that touch many files; follow high cohesion, low coupling

EXAMPLE / EXERCISE

Instead of one file holding all task logic, split into small files:

```
Services/TaskService.vs     # task CRUD
Services/MemberService.vs   # member management
Sections/Board.sc           # board page
Sections/TaskDetail.sc      # detail page
```

Now repairing one page only touches one .sc.

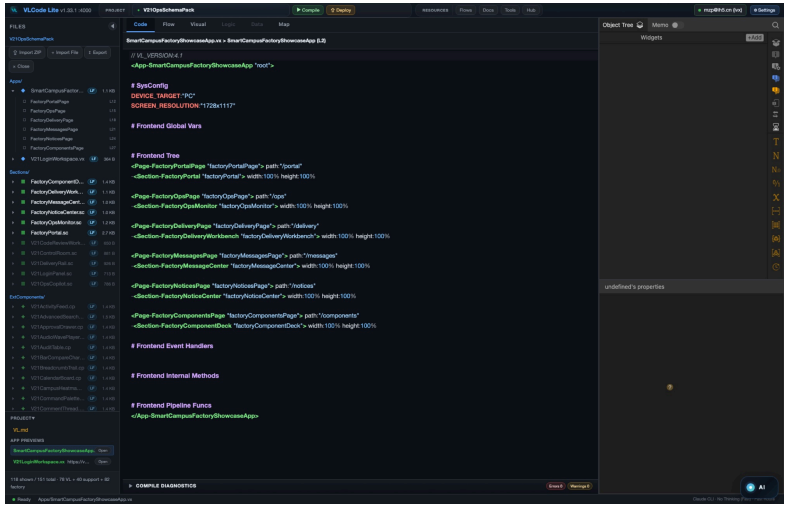
PITFALL

Merging multiple Sections into one huge .sc to "look tidy" makes any small change trigger reparsing of a large file and a potential broad rewrite — making it more fragile, not less.

CHECKPOINT

You can split a mixed-responsibility large file into several small files by type and explain why that aids surgical repair.

How To Read Generated VL



Showcase: code mode

CORE IDEA

Reading generated VL means reading top-down, map-to-detail: grasp the whole via ProjectMeta/Map first, then enter specific files. Use VLCode's multiple views — Code for source, Visual for pages, DB/ER for backend, Data for tables, THEME for tokens — to see the same code from several angles instead of grinding through source line by line. Once you know the file types and hyphen hierarchy, generated output becomes readable and verifiable.

OPERATIONS

1. Start in Map mode to survey App/Section/Service/data connections
2. Open a .sc, read structure in Code, then switch to Visual to see the actual page
3. Use DB mode for .vs and Data mode for .vdb to verify backend and data separately
4. Follow the hyphen hierarchy and service/token references to confirm pages, services, and data agree
5. Use AI Chat to ask it to "explain this file's structure and which services it calls"

EXAMPLE / EXERCISE

Ask AI Chat to guide your reading of the output:

```
"In one sentence, explain the structure of Sections/Board.sc and list which TaskService methods it calls."
```

Then switch to Visual to confirm the description matches the actual page.

PITFALL

Reading only Code mode line by line while ignoring Map and Visual makes you miss the forest for the trees and overlook cross-file dependencies. Cross-check with multiple views rather than treating VL as plain text.

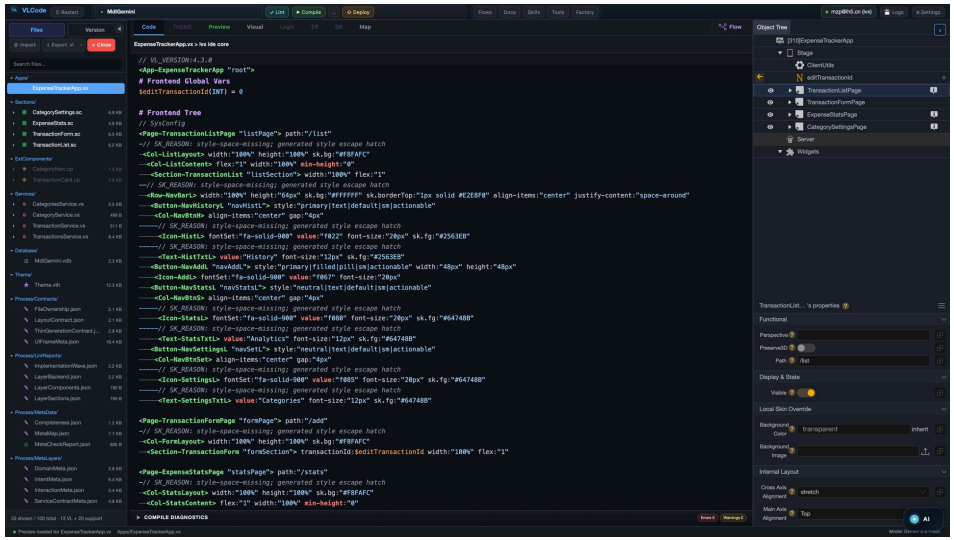
CHECKPOINT

Given a freshly generated project, you can explain within ten minutes which pages it has, which services each calls, and which tables hold the data.

CHAPTER

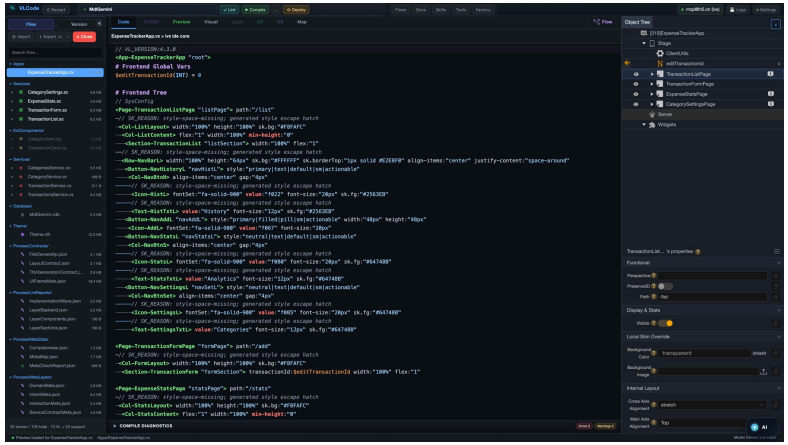
Part V: AI Chat And Workflow Control

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



AI Chat used as the command and decision surface

Write Requirements As Executable Goals



AI Chat used as the command and decision surface

CORE IDEA

AI Chat is VLCode's primary interaction entry and its workflow control console — what you type is treated as an executable goal that drives a generation or modification run, not casual chat. Writing requirements as bounded, verifiable goals directly determines output quality: name the entities (tables), pages, actions and constraints so the AI can emit compilable, previewable VL.

OPERATIONS

1. Type a one-line goal in the bottom AI Chat box naming the app type, core entities, and key pages
2. List the key fields of each entity (e.g. a task's title, owner, status, due date)
3. Describe the main actions and flow (create, assign, status transition) so the AI knows which Service methods to generate
4. Add a verifiable acceptance point (e.g. 'board has three status columns, drag to change status')
5. After sending, check Detail Log to confirm the AI split the goal into matching Section / Service / Data artifacts

EXAMPLE / EXERCISE

Instead of 'build a project tool', write a bounded goal:

```
Build a TeamPM team task board.
Data: Task(title, owner, status[todo/doing/done], priority, dueDate), Member(name, email).
Pages: board page showing task cards in three status columns; a task detail page to edit fields.
Actions: create task, assign owner, drag to switch status.
Acceptance: the three columns group correctly; changing status moves the card between columns.
```

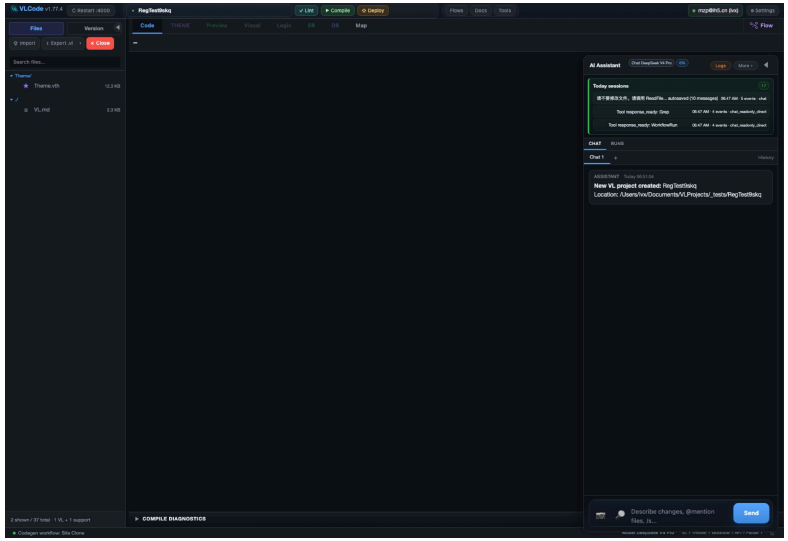
PITFALL

The most common mistake is a vague requirement ('build a CRM') — the AI has to guess fields and pages, producing something off-target and wasting tokens. Spell out entity fields and acceptance points before sending.

CHECKPOINT

You can rewrite a fuzzy request into an executable goal with entity fields, pages, actions and an acceptance point, and see it split into matching artifacts in Detail Log

Choose The Right Workflow



Workflow selector before execution

CORE IDEA

VLCode ships a set of named workflows, each tuned to a scale and scenario: meta-direct-codegen for small projects, meta-thin-fast-codegen when you want fast and stable, enterprise-grand-cascade-codegen for large systems, siteclonemax-pipeline to reverse-engineer a site. Choosing the right workflow matters more than tweaking prompts repeatedly, because it sets the layering strategy and validation depth.

OPERATIONS

1. Judge scale first: few pages (<5) → meta-direct-codegen; want fast and stable → meta-thin-fast-codegen
2. Judge source: have a design/HTML → smart-design-fullstack-codegen; cloning a site → siteclonemax-pipeline
3. Judge complexity: 100+ service methods or 30+ pages → enterprise-grand-cascade-codegen
4. Name the workflow in AI Chat in plain language, or let the AI recommend one from your goal
5. After confirming, send and watch Detail Log to see whether the workflow's node sequence matches expectations

EXAMPLE / EXERCISE

Name the workflow directly in AI Chat:

```
Use meta-thin-fast-codegen to generate the TeamPM task board — fast and stable.
```

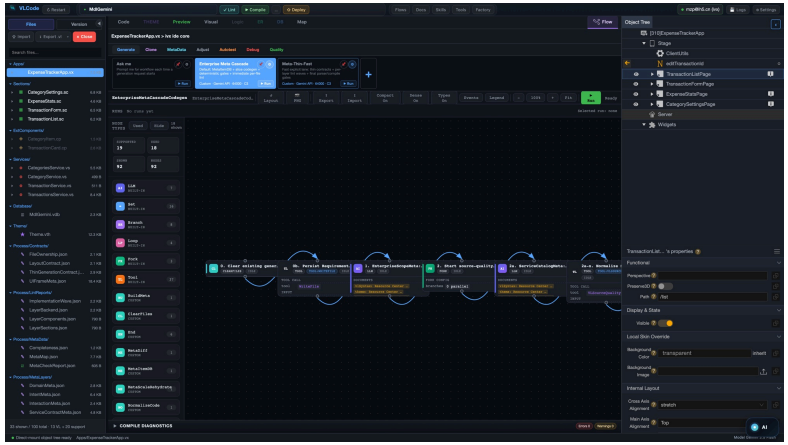
If unsure, ask the AI to choose:

```
I need a 30-page inventory system with 100+ service methods. Which generation workflow should I use?
```

The AI will suggest enterprise-grand-cascade-codegen.

PITFALL

MetaDirect For Small Projects



Code generation workflow running through the graph

CORE IDEA

meta-direct-codegen is the most direct generation path designed for small projects (under 5 pages): it derives ProjectMeta straight from the requirement, then generates Apps/Sections/Services/Database in one pass — few layers, short chain, fastest to get going. It suits prototypes and single-feature tools and is most people's first VLCode workflow.

OPERATIONS

1. Confirm the project is within 5 pages and the workspace lives under ~/Documents/VLProjects/
2. Write a clear goal in AI Chat (entities, pages, actions)
3. Specify the meta-direct-codegen workflow
4. After sending, follow the nodes in Detail Log: ProjectMeta is derived first, then the files are generated
5. When it finishes, run Lint then Compile, and finally open Preview to verify

EXAMPLE / EXERCISE

A small goal that fits meta-direct:

```
Use meta-direct-codegen to build a personal todo app MyTodo.
Data: Todo(title, done, createdAt).
Pages: one list page to add, check off, and delete items.
```

For a few-page app this workflow is the least effort.

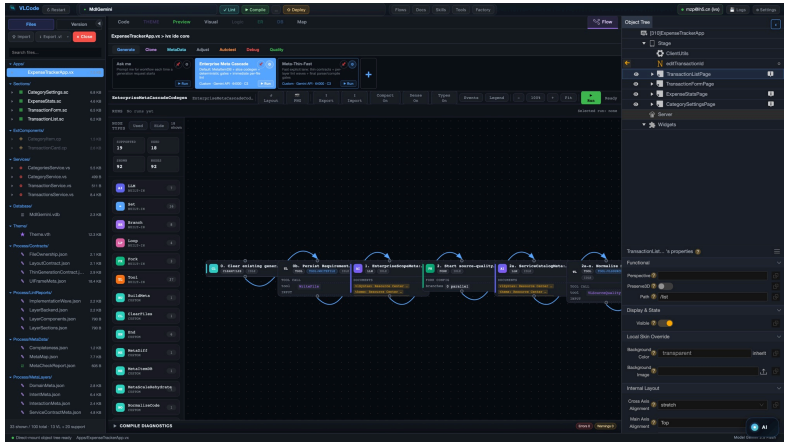
PITFALL

Feeding a large system with dozens of pages and service methods to meta-direct overloads ProjectMeta and yields incomplete output. Once you exceed ~5 pages, switch to meta-thin-fast or a layered workflow.

CHECKPOINT

You generated a complete sub-5-page app with meta-direct-codegen and passed Lint → Compile → Preview.

Meta Thin Fast Generation



Flow mode: a DAG workflow shown as executable nodes

CORE IDEA

meta-thin-fast-codegen runs a 'thin-contract + fast' route: it first establishes a lean, stable contract (the core skeleton of ProjectMeta), then generates code quickly from it, balancing speed and stability. It's steadier than meta-direct on medium projects and faster and cheaper on tokens than the large cascade workflows — a solid default for everyday generation.

OPERATIONS

1. Confirm the project is small-to-medium and you want speed with stable results
2. Write a clear goal in AI Chat and specify meta-thin-fast-codegen
3. After sending, watch Detail Log: the thin contract is produced first, then files are batch-generated from it
4. If something is missing at the contract stage, top it up in chat before continuing rather than waiting for the whole run
5. When done, verify with Lint → Compile → Preview

EXAMPLE / EXERCISE

An everyday safe default:

```
Use meta-thin-fast-codegen for the TeamPM task board — establish the thin contract first, then generate; fast and stable.
```

If Detail Log shows the contract missed the Member table, immediately add in chat: 'add a Member(name, email) table to the contract'.

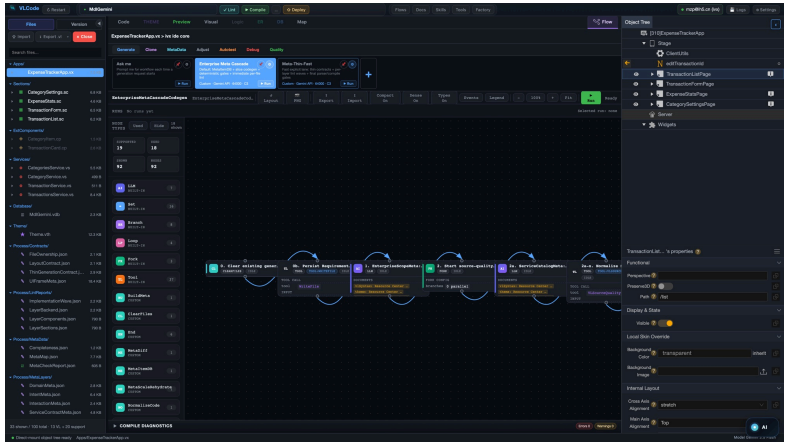
PITFALL

Rushing to wait for the final files at the thin-contract stage and ignoring missing fields in the contract. The contract is the root of every file after it — get the contract right before letting generation proceed.

CHECKPOINT

You generated a medium project with meta-thin-fast-codegen and can point to its two stages: the thin contract first, then files derived from it.

Layered And Sharded Generation (meta-layered / meta-sharded)

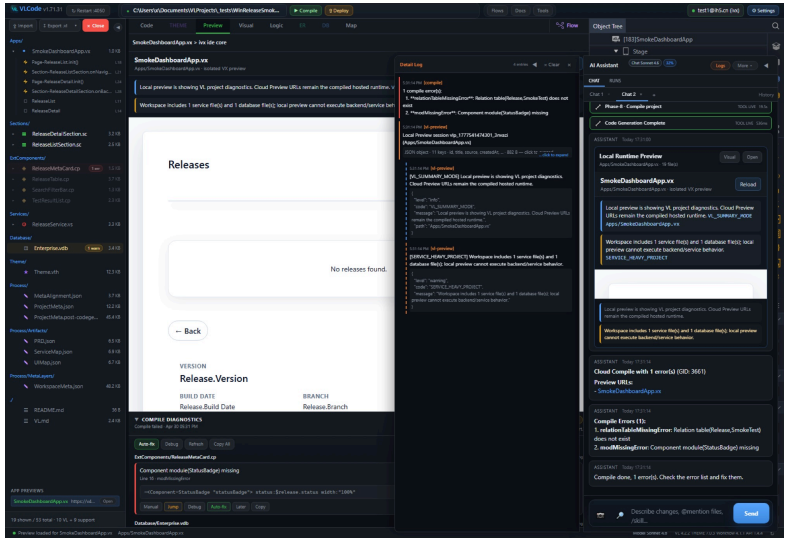


Flow mode: a DAG workflow shown as executable nodes

CORE IDEA

(Content pending.)

SmartDesign Full-Stack From Design



Compile preview inside the IDE

CORE IDEA

smart-design-fullstack-codegen runs a 'design-first (HTML-first) → code' route: it first turns design intent into an HTML design layer, then generates full-stack VL (pages + services + data) from it. It suits cases where you already have a visual design or want to lock the look before logic, making the UI match expectations instead of guessing layout from plain text.

OPERATIONS

1. Prepare the design source (a design description, HTML snippet, or screenshot)
2. Write a clear goal in AI Chat and specify smart-design-fullstack-codegen
3. In Detail Log, first confirm the HTML design layer matches the intended look
4. Once the design layer is confirmed, let the workflow generate full-stack VL (Section/Service/Data) from it
5. When done, compare against the design in the Visual canvas, then verify with Lint → Compile → Preview

EXAMPLE / EXERCISE

Design-first generation:

```
Use smart-design-fullstack-codegen for a CustomerCRM dashboard:
top row of KPI cards (new this month, revenue, conversion), below a deal funnel on the left and a recent-activity list on the right.
Show the HTML design layer first so I can confirm layout, then generate full-stack.
```

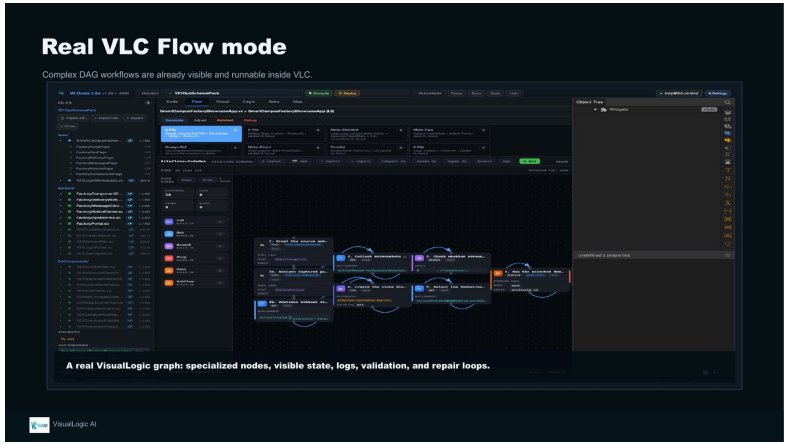
PITFALL

Skipping the HTML design-layer confirmation and jumping to final code, only to find the layout wrong and need a full rework. Lock the look at the design layer before releasing full-stack generation.

CHECKPOINT

You produced a confirmable HTML design layer with smart-design-fullstack-codegen, then generated matching full-stack VL and previewed it.

EnterpriseGrand For Large Systems



DAG harness: routing and review

CORE IDEA

enterprise-grand-cascade-codegen is a cascade generation workflow built for large systems, targeting 100+ service methods and 30+ pages. It splits a huge system into ordered generation stages via cascading layers, producing layer by layer while keeping cross-artifact references consistent — avoiding the missing files and broken references common when generating a big system in one shot.

OPERATIONS

1. Confirm this is a large system (dozens of pages / 100+ service methods); don't use it for small projects
2. In AI Chat, spell out modules, entities, and key flows as fully as possible so the cascade has enough structure
3. Specify enterprise-grand-cascade-codegen and send
4. Follow the cascade stages in Detail Log, checking each stage's output and whether service methods and pages are complete
5. When done, use the Map project view to check that dependencies close, then verify with Lint → Compile → Preview

EXAMPLE / EXERCISE

A large-system goal:

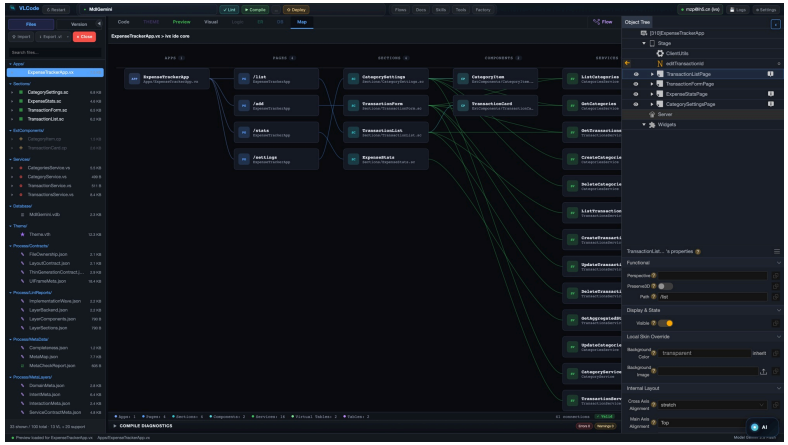
```
Use enterprise-grand-cascade-codegen to generate an ERP inventory system:
modules: purchasing, sales, stock, finance, reports, permissions.
~30 pages, 100+ service methods, sharing product and supplier master data across modules.
```

PITFALL

Using it for a small tool over-layers — slow and token-heavy; giving too little structure lets the cascade stages drift apart. Reserve it for genuinely large systems and describe modules and entities fully.

CHECKPOINT

SiteCloneMax Site Reverse-Engineering



Map mode for project structure and relationships

CORE IDEA

siteclonemax-pipeline reverse-engineers an existing website into real VL: paired with siteclonemax-explore it does authenticated deep crawling and network capture, recording page structure and API behavior, then rebuilds them as compilable VL pages and services. It suits migrating a legacy site or quickly standing up a VL version modeled on an existing product.

OPERATIONS

1. First run siteclonemax-explore to deep-crawl and capture network traffic on the target site (including auth-gated pages)
2. In AI Chat specify siteclonemax-pipeline, giving the target site and the scope to rebuild
3. In Detail Log, check that captured pages and network requests cover the key features
4. Let the pipeline rebuild the captures into VL Section/Service/Data
5. Compare against the original in Visual and Preview, confirming fidelity page by page, then verify with Lint → Compile

EXAMPLE / EXERCISE

Reverse-engineering an internal portal:

```
First run siteclonemax-explore to deep-crawl https://old-portal.example.com (logged-in), capturing pages and APIs.
Then run siteclonemax-pipeline to rebuild it as VL: list, detail, and search pages with matching service methods.
```

PITFALL

Skipping explore's authenticated deep crawl and cloning pages only visible after login loses most of the functionality. Make sure explore covered the auth-gated key paths before entering the pipeline.

CHECKPOINT

Using explore + pipeline you rebuilt an auth-gated site's key pages into compilable VL and compared them page by page against the original in Preview.

FactoryTurbo Reusing Components



DAG harness: artifact flow

CORE IDEA

factory-turbo-codegen is a speed-first workflow: it reuses pre-built factory components instead of generating UI and logic from scratch each time, so generation is faster and more consistent. It embodies VLCode's 'bank token-compute through reuse' idea — mature components are assembled rather than re-spent on tokens.

OPERATIONS

1. Confirm the app can heavily reuse existing factory components (standard tables, forms, boards, etc.)
2. Write a clear goal in AI Chat and specify factory-turbo-codegen
3. After sending, watch Detail Log to see how the workflow picks and assembles factory components
4. Against the requirement, confirm whether the assembled pages need light customization
5. When done, verify with Lint → Compile → Preview, and fine-tune with an incremental workflow if needed

EXAMPLE / EXERCISE

Speed-first, component reuse:

```
Use factory-turbo-codegen to quickly build a CustomerCRM back office:
customer list as a standard table component, add/edit as standard form components, deals as a board component.
Reuse factory components, minimal customization.
```

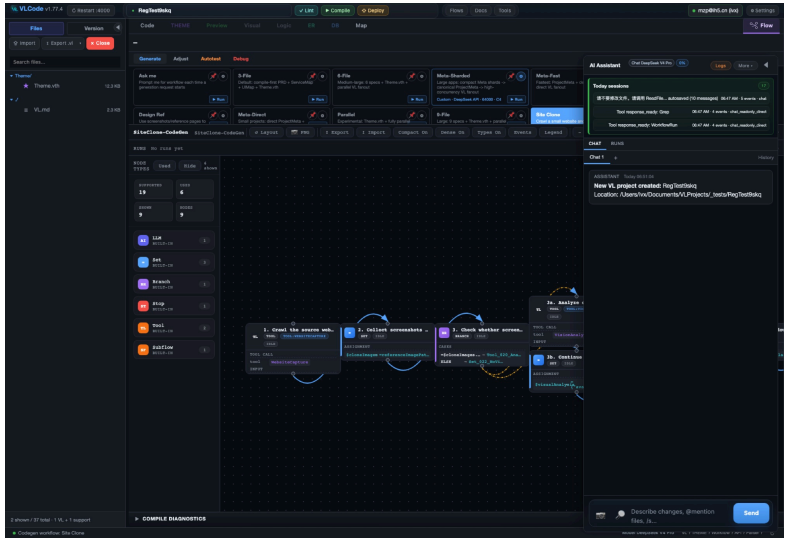
PITFALL

Forcing factory-turbo onto a highly custom, visually unique UI yields a 'close but not it' generic look. When you need unique design use smart-design-fullstack-codegen, and keep factory reuse for standardized screens.

CHECKPOINT

You quickly built a standardized back office with factory-turbo-codegen by reusing factory components and can explain why it's faster and cheaper than generating from scratch.

Monitor Progress During A Run



Workflow generation run in progress

CORE IDEA

VLCode splits run information across three places that never duplicate each other: AI Chat shows only final summaries and cards that need your decision; Detail Log (right panel) is the full run ledger — node progress, file writes, tool I/O, validation and errors; the status bar (bottom) shows the current step, LLM provider/model, compile status, and ports. Watch progress in the right place.

OPERATIONS

1. After a run starts, mainly watch the right-side Detail Log for per-node progress and file writes
2. Glance at the bottom status bar for the current step name, the model in use, and compile status
3. Wait in AI Chat for the final summary or a decision card — don't hunt for details there
4. Expand a tool node in Detail Log to see its full input/output
5. If a node stalls or errors, locate the specific node in Detail Log before acting

EXAMPLE / EXERCISE

During a TeamPM generation, each place shows its own thing:

```
Status bar:  Running generate-sections · DeepSeek V4 Pro · Compile: pending · :4000
Detail Log:  ✓ derive-meta  ✓ write Apps/TeamPM.vx  → generate-sections (writing BoardPage.sc)
AI Chat:     (quiet — awaiting final summary / decision card)
```

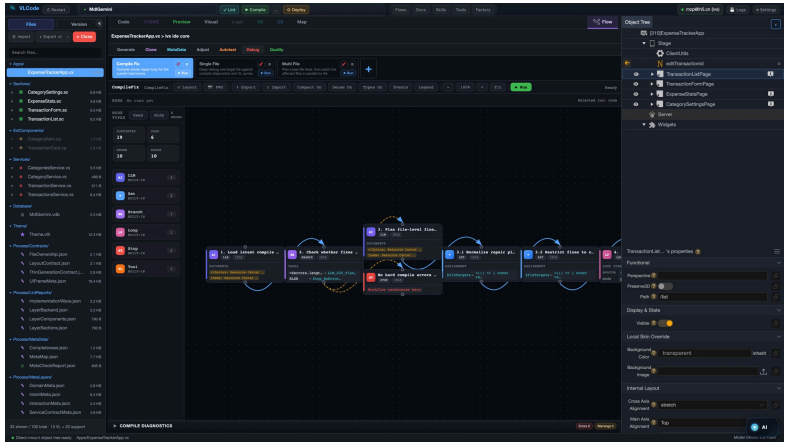
PITFALL

Digging through AI Chat for per-file, per-node detail — that's Detail Log's job, and Chat deliberately doesn't repeat it. Watching the wrong panel makes you think nothing's happening when it is.

CHECKPOINT

During a run you can say: current step → status bar, per-node and files → Detail Log, final summary and decisions → AI Chat.

Pause, Resume, And Abort



Debugging and rerun context inside the DAG

CORE IDEA

In VLCode you can run multiple chat sessions and multiple workflow runs in parallel (one IDE window = one workspace = one backend). The key to controlling a run is the abort scope: aborting at chat scope stops this chat plus all runs under it, while aborting at workflow scope stops one run only. Pick the wrong scope and you either kill other tasks or fail to stop the one you meant.

OPERATIONS

1. Pause the current run from AI Chat in plain language or via the control entry
2. Resume it from where it paused when you're ready
3. Decide the abort scope: stop one run → workflow-scope abort; stop every run in this chat → chat-scope abort
4. With multiple parallel runs, confirm in Detail Log which run you intend to stop
5. After aborting, check Detail Log to confirm the other parallel runs are unaffected

EXAMPLE / EXERCISE

Distinguish the two aborts:

```
Situation: one chat is running both 'generate TeamPM' and 'fix CustomerCRM compile errors'.
Stop only the fix → workflow-scope abort, stopping that run alone.
Stop everything in this chat → chat-scope abort.
```

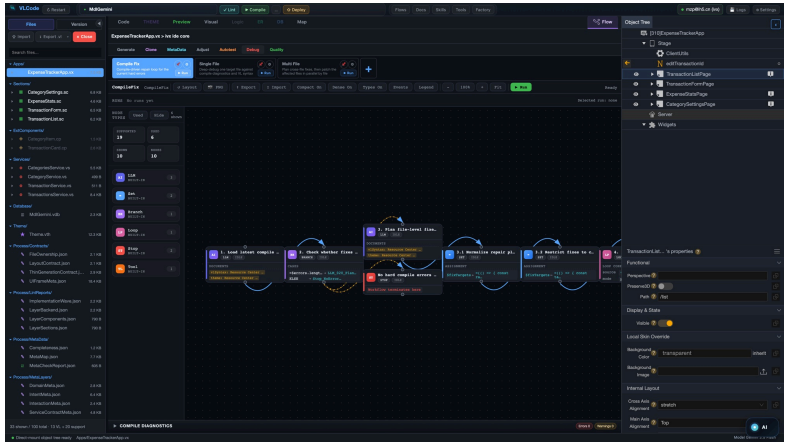
PITFALL

Trying to stop one problem run but using a chat-scope abort, which also kills the other healthy runs in the same chat. Confirm whether the scope is workflow or chat before aborting.

CHECKPOINT

With two parallel runs, you can abort just one without affecting the other and explain the difference between chat-scope and workflow-scope abort.

Rerun From A Specific Step



Debugging and rerun context inside the DAG

CORE IDEA

When a run errors or produces something wrong at a node, rerunning from a specific step avoids redoing everything: instead of regenerating from scratch, you locate that node, fix its input or source, and continue from that step. This aligns with VLCode's 'avoid broad rewrites' discipline — touch only the failing segment and keep the correct artifacts before it.

OPERATIONS

1. Locate the failing or wrong-output node in Detail Log
2. Find its root cause (inspect its input, the related source files, and ProjectMeta)
3. Fix the source file or supply the correct input in Chat
4. Rerun from that step rather than restarting the whole run
5. Confirm in Detail Log that artifacts from that node onward are fixed while earlier ones are unchanged

EXAMPLE / EXERCISE

Rerun only the failing step:

```
Detail Log shows the generate-services node errored: TaskService references a nonexistent field 'assignee'.
Fix: add an 'assignee' field to the Task table (or change the service to use 'owner'),
then rerun from the generate-services step, leaving the earlier page and data artifacts untouched.
```

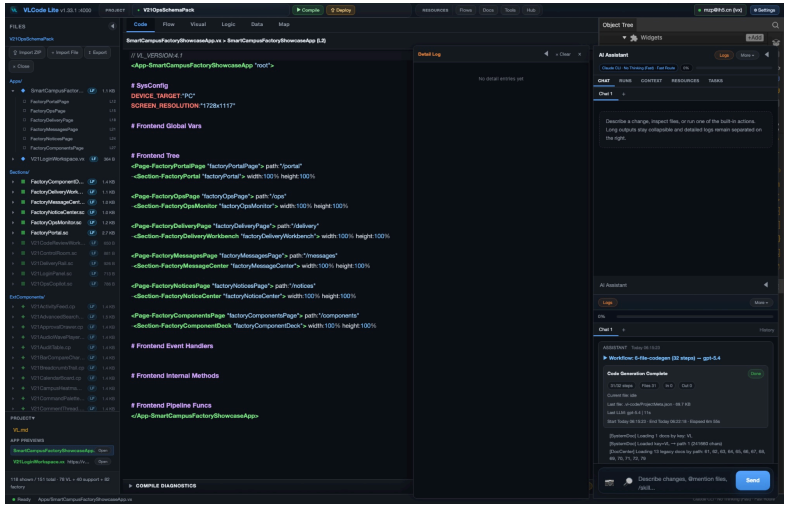
PITFALL

Restarting the whole run on any error, regenerating the already-correct pages and data and wasting time and tokens. Locate the node, fix the root cause, then rerun from that step.

CHECKPOINT

On an error you can locate the specific node, fix the root cause, and rerun from just that step while keeping the earlier correct artifacts.

Variable Injection And Human Cards



AI Chat with detailed run context

CORE IDEA

A workflow run isn't a black box that just charges through: via Interactive Cards, the run pauses at key points to ask for your input or decision. The common blocking cards are the picker-card (options, surfaced by the ask_user chat event, with a custom-answer path), the review-card (approve/reject/request_changes, with human_gate arriving as a chat event), the form-card, and action buttons (e.g. Deploy triggers a binding-card). Note ask_user / human_gate are themselves chat events (SSE), not card kinds. What you fill in is injected back into the run as variables that shape downstream nodes.

OPERATIONS

1. During a run, watch for Interactive Cards appearing in AI Chat (decision, form, option types)
2. For a picker-card, pick a preset or use the custom-answer path to enter your value (the ask_user chat event surfaces the options)
3. For a review-card, explicitly choose approve, reject, or request_changes (human_gate arrives as a chat event)
4. For a form-card, fill the fields with the variable values to inject
5. After submitting, confirm in Detail Log that your input was injected and used by downstream nodes

EXAMPLE / EXERCISE

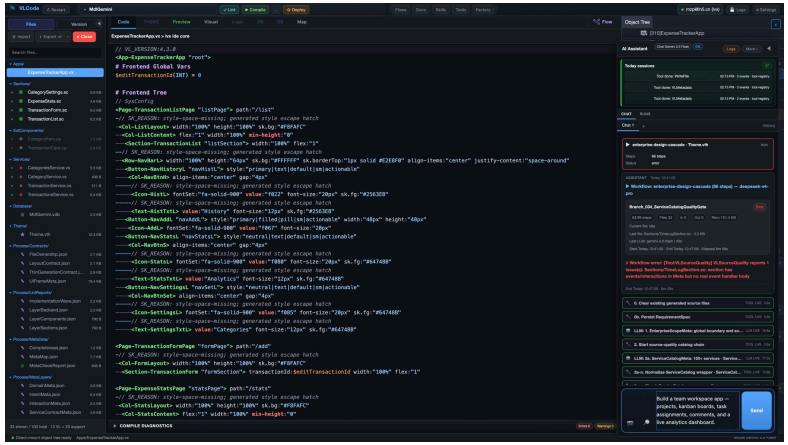
A card-driven decision segment:

```
picker-card (ask_user event): 'Which status set for TeamPM?'
  options: [todo/doing/done], [new/in-progress/resolved/closed], Custom…
  pick Custom and enter: todo / designing / building / testing / live
review-card: 'Confirm the data-table plan?' → choose 'Request changes: add storyPoints to Task'
```

PITFALL

Ignoring a blocking card as if it were a normal chat message leaves the run hanging; or entering a custom answer inconsistent with downstream contracts causes later errors. Take each card seriously and keep values aligned with the goal.

Watch Rules And Automatic Interception



Detail Log with node progress and tool output

CORE IDEA

For long unattended runs, Watch rules let you pre-register conditional monitors: when a condition is met, an action fires automatically. You register them in plain language (e.g. 'stop if a node fails'); the action can be stop, pause, or notify, and when the condition hits, a watch-fired event appears in chat to alert you — so you needn't stare at Detail Log the whole time.

OPERATIONS

1. Before or during a run, register a Watch rule in plain language
2. State the trigger condition and the action clearly (stop / pause / notify)
3. Let the run continue without watching it constantly
4. When the condition hits, receive a watch-fired alert in AI Chat
5. On the alert, open Detail Log to inspect the triggering node and act

EXAMPLE / EXERCISE

Register a stop-on-failure watch:

```
Stop the run if the generate-services node fails.
```

Or only notify me when a step retries beyond a threshold so I can decide whether to step in. When the run hits the condition, a watch-fired alert pops in chat and you go to Detail Log for the root cause.

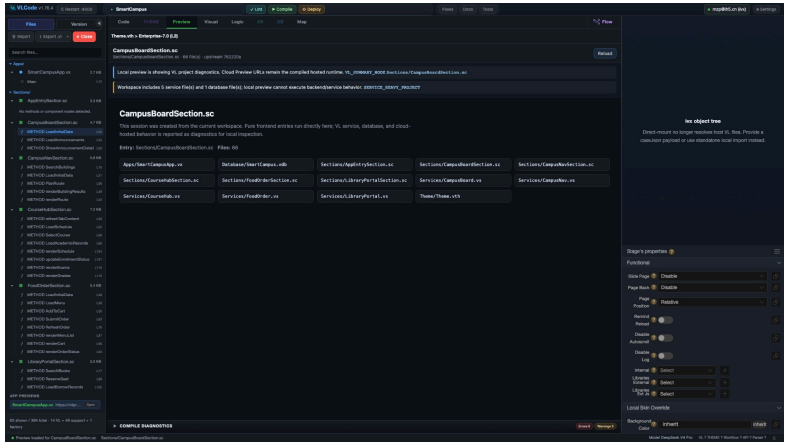
PITFALL

Treating a Watch as a 'fixer' — it only fires the stop/pause/notify action you defined; it doesn't auto-edit code. After it fires you still go to Detail Log to find and fix the root cause.

CHECKPOINT

You can register a 'stop if a node fails' Watch, and when the run hits it you see the watch-fired alert in chat and locate the problem from it.

Read Errors In Detail Log



A logic-panel failure context for debugging practice

CORE IDEA

Detail Log is the main arena for troubleshooting: it records each node's progress, file writes, tool I/O, validation results, and errors, and AI Chat doesn't repeat these. The key to reading errors is finding the failing node, examining its tool I/O and validation message, and tracing to the specific source file and ProjectMeta — not just the one-line gist in Chat.

OPERATIONS

1. After an error, find the node marked failed in the right-side Detail Log
2. Expand that node to see its full tool input/output and the error message
3. Follow the file name/field the error points to back into the source file and ProjectMeta
4. Decide whether it's a missing field, a broken reference, or a format issue like indentation or the version header
5. If the same error repeats twice, return to the source for the root cause instead of retrying or broad-rewriting

EXAMPLE / EXERCISE

Reading a compile error in Detail Log:

```
✗ compile  BoardPage.sc
  error: unknown service method TaskService.moveTask
  → root cause: the service method is named changeStatus; the page referenced moveTask
  fix: unify on changeStatus, rerun from the compile step
```

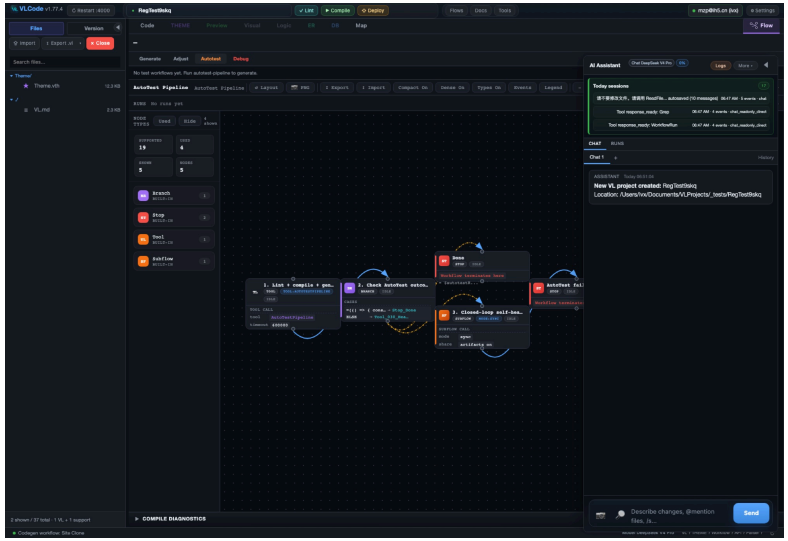
PITFALL

Guessing from the AI Chat summary alone and missing the precise node and error in Detail Log. Some also retry the same error repeatedly without returning to source — by discipline, a twice-repeating error means stop and find the root cause.

CHECKPOINT

Facing a failed run, you can locate the failing node in Detail Log, read the error, and point to the source file and root cause.

Four Checks After Completion



Logs and execution details after a run

CORE IDEA

Finishing a generation run doesn't mean it's delivered. VLCode's validation chain has a fixed order: Lint first (low cost, run earliest) → then Compile (heavier platform validation) → finally Preview (actually open at /route/<path>). This topic's 'four checks' wires the validation chain to a final read: Lint, Compile, Preview, then re-read Detail Log for any leftover errors or gaps from this run.

OPERATIONS

1. Step one: run Lint (lintPjt) for a project-level static check and clear low-cost issues first
2. Step two: run Compile (parsevl) for the heavier platform-level validation
3. Step three: open Preview at /route/<path> to actually view pages and interactions
4. Step four: re-read Detail Log to confirm no skipped error nodes or unwritten files
5. If any step fails, find the root cause and fix it rather than publishing

EXAMPLE / EXERCISE

The four checks after completion:

```
1) Lint: lintPjt → 0 errors
2) Compile: parsevl → 0 errors
3) Preview: open /route/board, verify the three columns drag to change status
4) Detail Log review: no red error nodes; Apps/Sections/Services/Database files
all present
```

PITFALL

Using Compile as the first trial loop and running it repeatedly — it's heavy and slow; and don't publish just because Preview 'looks right'. Go in Lint → Compile → Preview order and close with a Detail Log review.

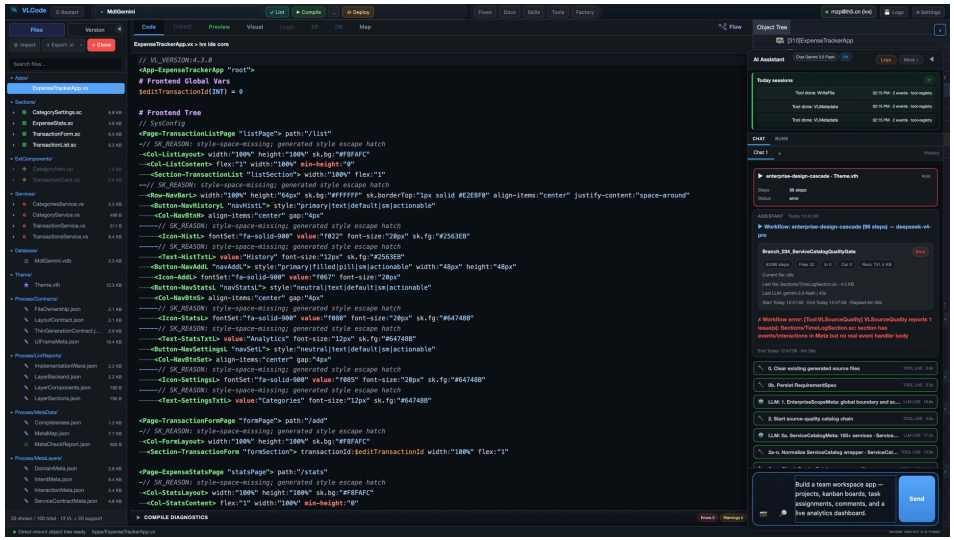
CHECKPOINT

On a just-finished generation you can run Lint, Compile, and Preview in order and only treat it as delivered after confirming no leftover errors in Detail Log.

CHAPTER

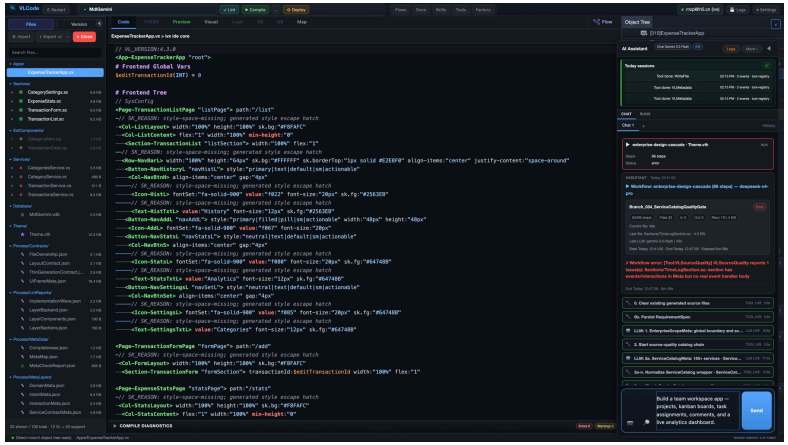
Part VI: Build The First App From Scratch

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



A concise project brief in the assistant panel

Lab Goal: Team Task Board



A concise project brief in the assistant panel

CORE IDEA

This part follows one running case — the TeamPM team task board — to take the earlier concepts through a single real end-to-end generation. The goal is a small app that creates tasks, lays them out by status, and toggles status on click — exactly the size that suits a small-project workflow like MetaDirect or Meta Thin Fast. Nail the goal first; every later step serves it.

OPERATIONS

1. Define the minimal scope for this version: a task list, new-task creation, three columns (To-Do / In-Progress / Done), and click-to-toggle status
2. Confirm it is a small project (fewer than 5 pages), fitting meta-direct-codegen or meta-thin-fast-codegen
3. Agree on the project name TeamPM (English PascalCase), living under ~/Documents/VLProjects/TeamPM/
4. Set the validation goal: Lint passes → Compile passes → Preview opens the board page

EXAMPLE / EXERCISE

The deliverable for this version fits in one sentence:

```
TeamPM: a task board where each task has title / owner / status across three
columns (To-Do, In-Progress, Done); you can create tasks and click a task to
cycle its status.
```

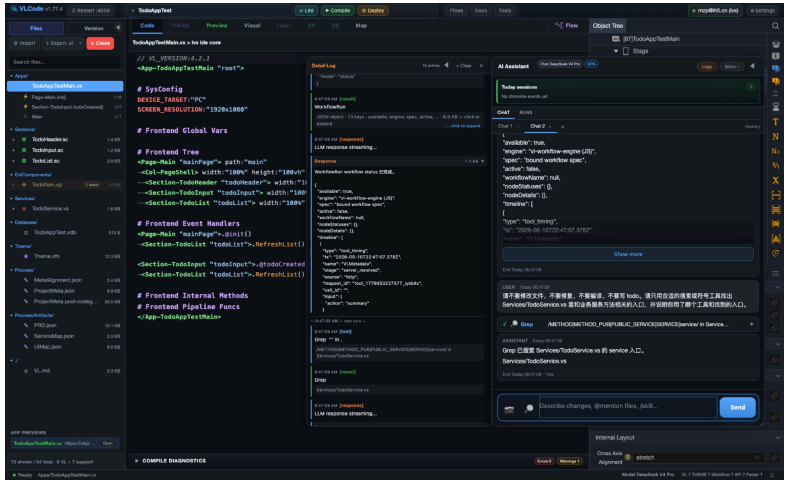
Every later step (write the requirement, pick the workflow, inspect artifacts, validate) serves only this sentence.

PITFALL

Over-scoping at the start (adding permissions, notifications, reports) overwhelms a small-project workflow, slowing generation and inviting errors. Keep version one to the board core; defer expansion to the incremental workflows in Part VII.

CHECKPOINT

Prepare The TeamPM Workspace



Final IDE state after project creation

CORE IDEA

In VLCode one IDE window equals one workspace equals one backend. Before generating, prepare the TeamPM project directory with the standard layout under the VLProjects root, so the generated .vx/.sc/.vs/.vdb files land in the right places. This is the prerequisite for everything that follows.

OPERATIONS

1. Create the project folder TeamPM/ under ~/Documents/VLProjects/ (English PascalCase name)
2. Establish the standard subdirectories: Apps/ Sections/ Services/ Database/ Theme/ ExtComponents/ Process/ Config/
3. In VLCode, switch/open the current workspace to the TeamPM folder
4. Check the bottom status bar to confirm the backend is ready and the current LLM provider/model shows correctly

EXAMPLE / EXERCISE

Preparing the directory is a one-liner:

```
mkdir -p ~/Documents/VLProjects/TeamPM/{Apps,Sections,Services,Database,Theme,ExtComponents,Process,Config}
```

Then point the VLCode workspace at this folder; the left file tree shows the empty standard structure.

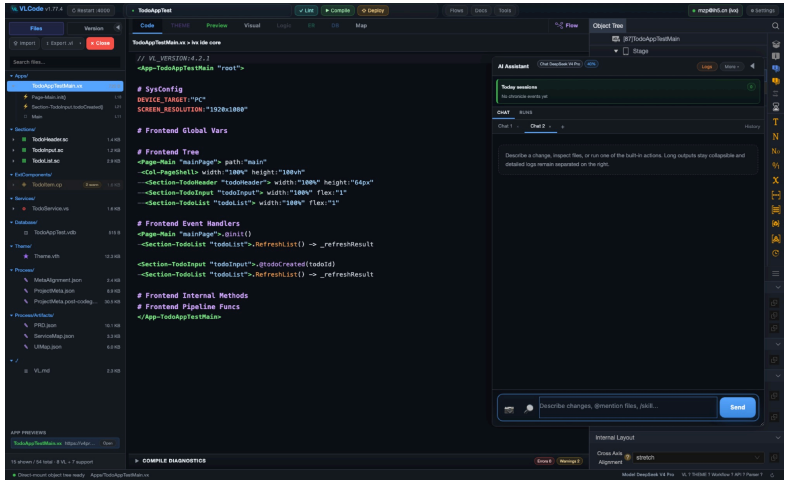
PITFALL

Generating project files into VLCode's own source tree or a stray temp folder pollutes the tool tree and breaks the project-root convention. Always create ~/Documents/VLProjects/TeamPM/ first, then generate.

CHECKPOINT

The left file tree shows TeamPM's standard directories (Apps/Sections/Services/Database/Theme...) and the status bar shows the backend ready plus the current model.

Write The First Requirement



Starting from an empty chat session

CORE IDEA

AI Chat is VLCode's primary interaction entry and its workflow control console. The first requirement must read as an executable goal — concrete about pages, data fields, and interactions, not a vague wish. The clearer the requirement, the more accurate the resulting ProjectMeta, and the less rework later.

OPERATIONS

1. Open the AI Chat main conversation and write the TeamPM goal as a natural-language requirement
2. List the data: a task has title, owner, and status (To-Do / In-Progress / Done)
3. List pages and interactions: one board page, three columns, create-new-task, click-to-toggle status
4. After submitting, wait for the AI to propose a workflow or surface an interactive card; do not pile on new requirements yet

EXAMPLE / EXERCISE

Paste a requirement like this into AI Chat:

```
Build a team task board called TeamPM. Data: a task has title, owner, and status
(To-Do / In-Progress / Done). Page: one board page that groups tasks into three
columns by status, with a "New Task" button at the top; clicking a task cycles its
status. Generate the minimal usable version first.
```

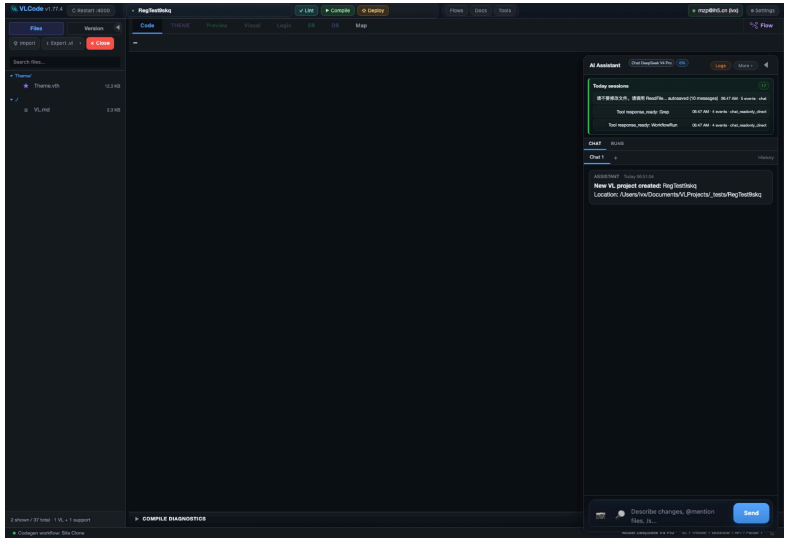
PITFALL

Writing something as broad as "build a project-management system" forces the AI to guess, drifting on fields and pages. Always spell out the three things: data fields, pages, and click behavior.

CHECKPOINT

AI Chat holds a submitted, executable requirement covering data fields, page layout, and interaction behavior, and a workflow suggestion has come back.

Select MetaDirect Or Meta Thin Fast



Workflow selector before execution

CORE IDEA

TeamPM is a small project, so prefer one of two workflows: meta-direct-codegen for the smallest, fastest direct generation, and meta-thin-fast-codegen which stays fast while running a stable thin-contract that drifts less. Both are lighter than the layered (meta-layered-codegen) or large cascade (enterprise-grand-cascade-codegen) workflows and fit this version.

OPERATIONS

1. Among the AI's workflow suggestions, choose between meta-direct-codegen and meta-thin-fast-codegen first
2. If you want maximum speed and the scope truly is tiny, pick meta-direct-codegen
3. If you want a more stable thin-contract with less rework, pick meta-thin-fast-codegen
4. Confirm you do NOT need a heavyweight workflow like enterprise-grand-cascade or site-clone

EXAMPLE / EXERCISE

Name the workflow explicitly in AI Chat:

```
This is a small project — generate TeamPM with meta-thin-fast-codegen; favor stability and minimal rework.
```

Or for raw speed:

```
Use meta-direct-codegen to generate the minimal version of TeamPM directly.
```

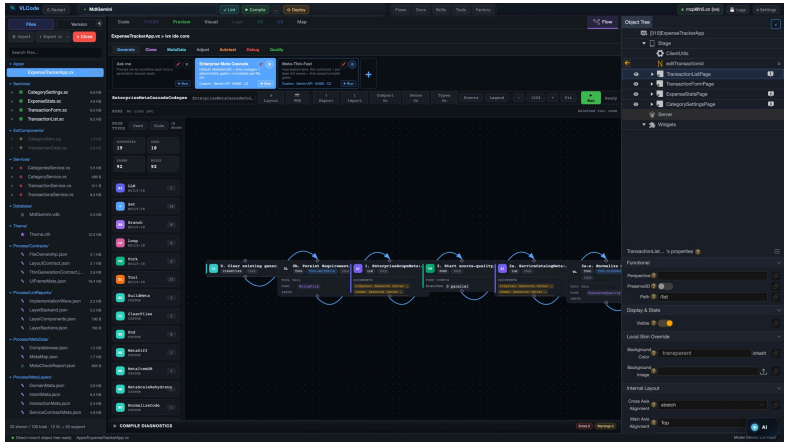
PITFALL

Putting enterprise-grand-cascade-codegen (built for 100+ service methods and 30+ pages) onto a small board over-shards it, slows it down, and produces bloated artifacts. Match the workflow weight to the project size.

CHECKPOINT

You have selected meta-direct-codegen or meta-thin-fast-codegen for TeamPM and can justify it over the layered or large-system workflows

Start The Generation Run



Code generation workflow running through the graph

CORE IDEA

With the requirement and workflow set, start the generation run from AI Chat. Chat shows final summaries and any cards that need a human decision; the real per-node progress, file writes, and tool I/O go to the Detail Log on the right. During generation, let the run finish rather than repeatedly editing the requirement mid-flight.

OPERATIONS

1. After confirming the workflow in AI Chat, start this generation run
2. If a picker-card or form-card appears (the ask_user chat event surfaces the options), answer it per the TeamPM requirement (e.g., confirm fields, page count)
3. Open the Detail Log on the right to watch node progress and file writes, rather than staring at Chat for details
4. Let the run settle to a terminal state (done/error/aborted); do not inject new requirements mid-run

EXAMPLE / EXERCISE

If, after the run starts, Chat pops an option card asking "three task states or four?", just pick "three (To-Do / In-Progress / Done)" on the card; the rest scrolls as nodes in the Detail Log. Chat finally shows a one-line summary like "Generation complete, N files written."

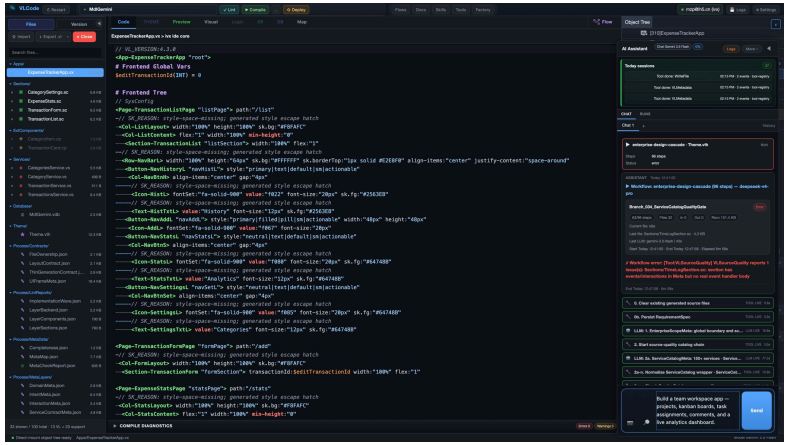
PITFALL

Expecting per-file, per-node details to appear in Chat — where they don't — means you miss them. Remember: Chat holds summaries and human decisions; the Detail Log is the full run ledger.

CHECKPOINT

The generation run has started and reached a terminal state, the Detail Log shows node progress and file writes, and Chat shows a completion summary.

Understand Intermediate Artifacts



Detail Log with node progress and tool output

CORE IDEA

Generation is not one shot of code but a chain of intermediate artifacts: first the ProjectMeta (the machine-readable map, with canonical root keys apps, sections, components, services, dataSchema, theme, dependencyGraph), then the VL files derived from it. Reading these artifacts in the Detail Log lets you pinpoint which link drifted when something breaks, instead of blindly rewriting.

OPERATIONS

1. In the Detail Log, follow the node order: ProjectMeta is generated/updated first, then the VL files
2. Confirm the pages, services, and tables ProjectMeta lists match the TeamPM requirement
3. Inspect the files each node writes (.vx/.sc/.vs/.vdb) and which node owns them
4. If an intermediate artifact mismatches the requirement, locate that node rather than waiting for the whole run to finish and then doing a big rewrite

EXAMPLE / EXERCISE

The Detail Log shows an artifact stream like:

```
Node 1: build ProjectMeta — apps:[TeamPM], sections:[Board], services:[TaskService], dataSchema:[Task]
Node 2: write Apps/TeamPM.vx
Node 3: write Sections/Board.sc
Node 4: write Services/TaskService.vs
Node 5: write Database/Task.vdb
```

Check the ProjectMeta line first — it is the source of every file after it.

PITFALL

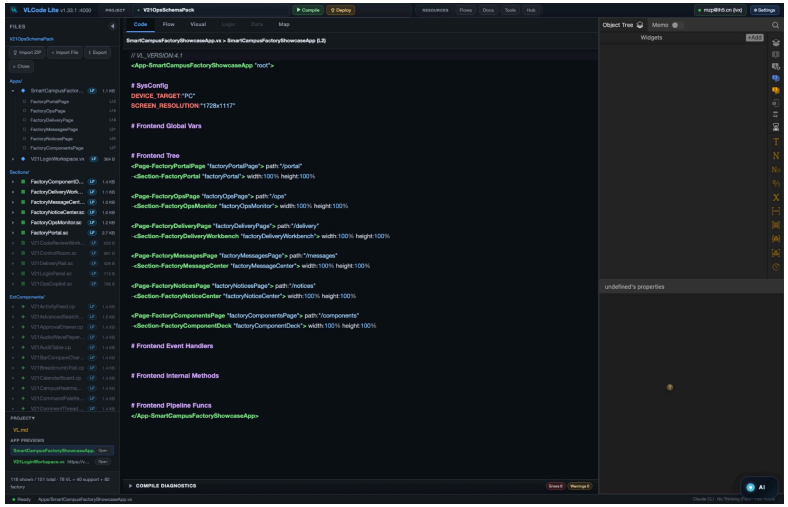
Skipping ProjectMeta and looking only at final files leaves you unable to find the root cause. If ProjectMeta is wrong, the files inherit the error; read the map first, then the files.

CHECKPOINT

You can point to the ProjectMeta node in the Detail Log and verify its listed sections/services/data match the TeamPM requirement.

PART VI: BUILD THE FIRST APP FROM SCRATCH

Inspect Apps/Sections/Services



Code mode for inspecting and editing VL source

CORE IDEA

After generation, browse the structural files in Code mode first: the .vx under Apps is the app entry, the .sc files under Sections are pages/regions, and the .vs files under Services are backend logic. Check completeness, English PascalCase names, and that VL_VERSION is the first line of each file before diving into content.

OPERATIONS

1. Expand Apps/, Sections/, and Services/ in the left file tree
2. Switch to Code mode and open Apps/TeamPM.vx, confirming it references the board Section as the entry
3. Open Sections/Board.sc and confirm the page structure and three-column layout exist
4. Open Services/TaskService.vs and confirm it has task methods like create and toggle-status
5. File by file, confirm the first line is VL_VERSION and indentation uses consecutive hyphens

EXAMPLE / EXERCISE

Opening Apps/TeamPM.vx, the top should be the version declaration:

```
// VL_VERSION:4.3.1
```

and Services/TaskService.vs should contain method names like createTask or updateStatus that correspond to the board's create and status-toggle actions.

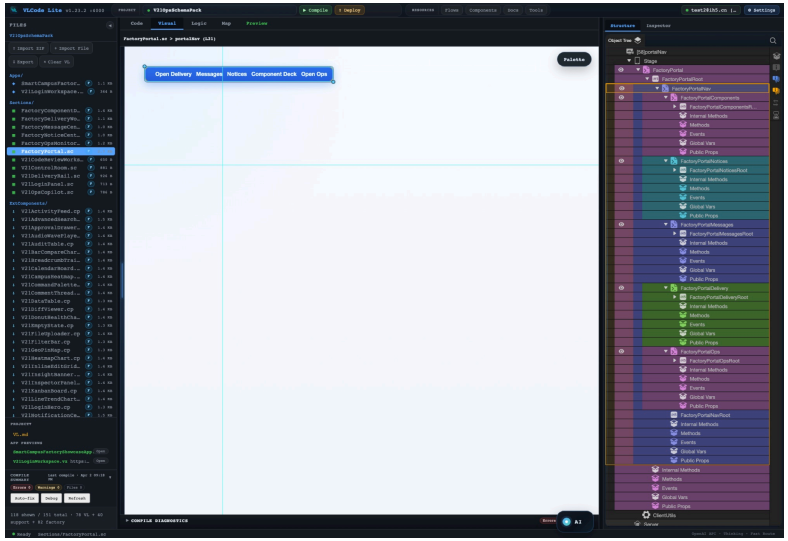
PITFALL

Inspecting Sections without checking Services lets you miss a break where "the page has a button but no matching service method." Walk all three layers — Apps → Sections → Services — during the structural check.

CHECKPOINT

All three of Apps/Sections/Services are present, the entry correctly references the board page, the service exposes methods matching create/status-toggle, and every file's first line is VL_VERSION.

Inspect Pages In Visual Mode



Visual panel with object tree and properties

CORE IDEA

Visual mode renders a generated Section onto the canvas, paired with the object tree and properties panel, so you can see what a page looks like without reading source. For the board, this step confirms the three-column layout, task cards, and the new-task button were actually generated and placed correctly.

OPERATIONS

1. Select Sections/Board.sc and switch to Visual mode to open the canvas
2. In the object tree, check the container hierarchy: board → three status columns → task cards
3. Select the "New Task" button on the canvas and confirm it exists and sits at the top
4. In the properties panel, verify key component properties (column title text, the field a card binds to)

EXAMPLE / EXERCISE

Select the "In-Progress" column among the three on the Visual canvas; the properties panel should show its title as "In-Progress" and its cards bound to the set of tasks whose status is In-Progress; the object tree shows the hierarchy Board > Column(In-Progress) > TaskCard.

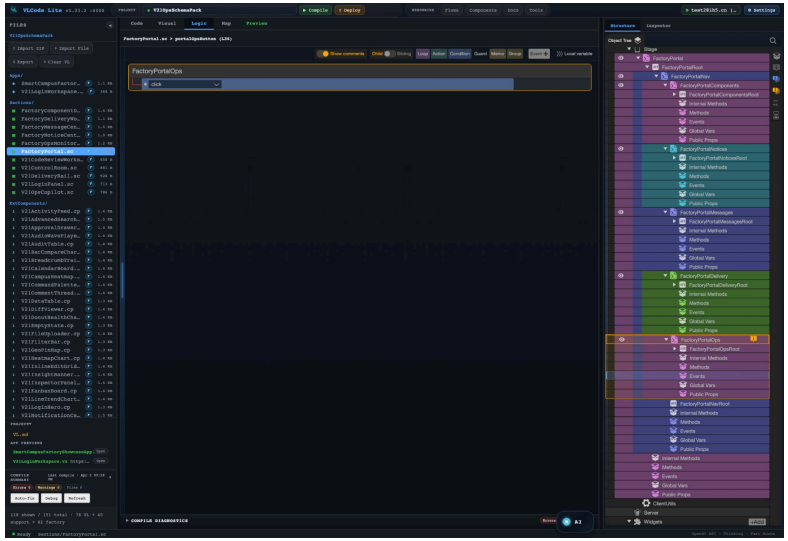
PITFALL

Checking appearance in Visual but not the object-tree bindings can fool you into thinking the page is fine when cards may not be bound to the right data field. Verify both look and binding.

CHECKPOINT

The Visual canvas shows the three-column board layout and the new-task button, the object-tree hierarchy is correct, and the properties panel confirms cards bound to the right task fields.

Check Click Behavior In Logic Mode



Logic panel for event and action blocks

CORE IDEA

Logic mode is the event/action panel, showing what runs when a component is clicked or triggered. The board's core interactions are "click a task to toggle status" and "click new-task to create a task," and this step confirms those events carry the right actions that ultimately call Service methods.

OPERATIONS

1. Select a task card in Visual/the object tree and switch to Logic mode to see its click event
2. Confirm the click event's action toggles status (and calls the matching service method)
3. Select the "New Task" button and confirm its click event triggers a create-task action
4. Follow the action to the Service method it references, then return to the DB panel to confirm the method exists

EXAMPLE / EXERCISE

Select a task card; in the Logic panel its onClick should be an action chain that toggles status, like:

```
click TaskCard → call TaskService.updateStatus(task.id, nextStatus)
```

Confirm that updateStatus is actually defined in Services/TaskService.vs.

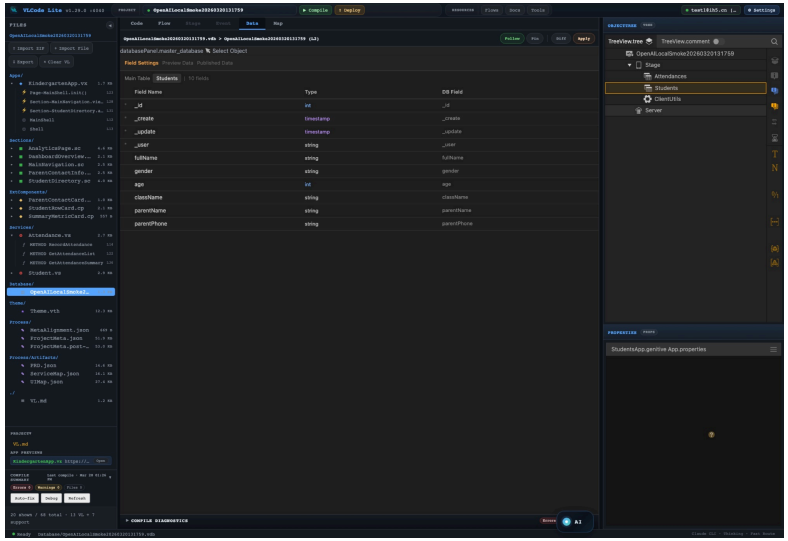
PITFALL

An event can carry an action that references a service method which doesn't exist — often invisible until Lint runs. Once you see a method name in Logic, go back to the DB panel and confirm it really exists.

CHECKPOINT

The task card's click event points to a real status-toggle service method, and the new-task button's click event points to a real create-task method.

Check Tables In Data Mode



Data panel for tables and schema

CORE IDEA

Data mode shows the project's tables and fields (corresponding to .vdb files). The board needs a task table whose fields line up across three places: the requirement, the page bindings, and the service methods. This step confirms the data layer has no missing or extra fields.

OPERATIONS

1. Switch to Data mode and open the task table behind Database/Task.vdb
2. Verify the fields: are the three core fields — title, owner, status — all present?
3. Confirm the status field's values match the three columns (To-Do / In-Progress / Done)
4. Cross-check the table fields against the card bindings in Visual and the parameters of the Service methods so all three agree

EXAMPLE / EXERCISE

The task table in the Data panel should look like:

```
Task
- title    text
- owner    text
- status   enum(todo / in_progress / done)
```

The three status enum values must map one-to-one onto the board's three columns and match the values the updateStatus service method accepts.

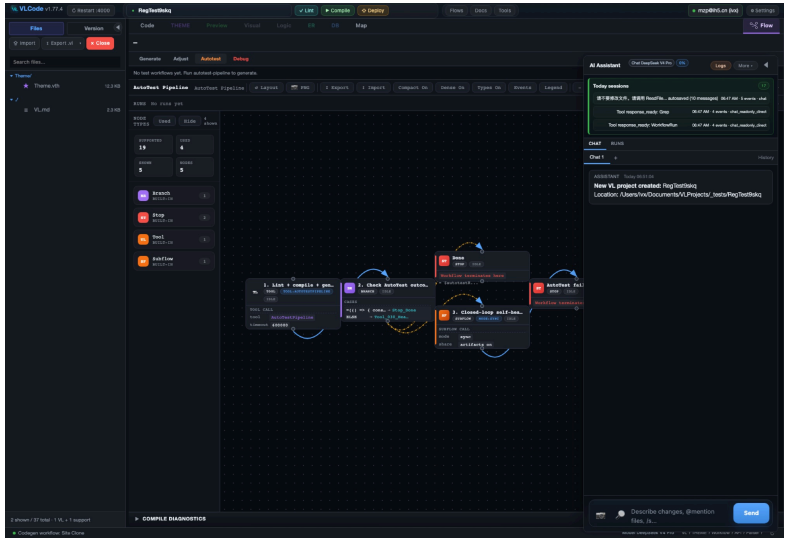
PITFALL

If the table's status field uses values different from the page columns (e.g., the table says doing while the column is labeled "In-Progress"), cards won't sort into the right column. Keep the values identical across all three places.

CHECKPOINT

The task table has title/owner/status fields, and the status values match the board's three columns and the service method parameters exactly.

Run Lint



Logs and execution details after a run

CORE IDEA

Lint is the first link of the validation chain and the cheapest — a project-level VL static check (lintPjt). Without invoking heavy platform compilation, it quickly catches missing methods, broken references, and indentation/version-line errors. The first validation pass is always Lint; do not jump straight to Compile.

OPERATIONS

1. After generation, run Lint (lintPjt) across the whole project first
2. Read the issue list in the Detail Log / report
3. Focus on: missing VL_VERSION, mixed indentation, broken service-method references, field mismatches
4. Fix the cheap issues Lint reports before considering Compile

EXAMPLE / EXERCISE

Run the project-level static check on TeamPM:

```
lintPjt
```

If it reports something like

```
Sections/Board.sc: references undefined service method TaskService.updateStatus
```

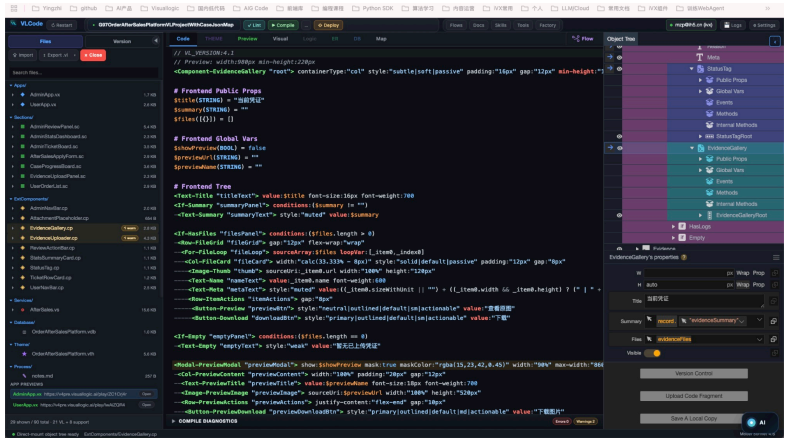
the page action and the service method don't line up — go back to the Service and add the method.

PITFALL

Skipping Lint and going straight to Compile is slower and pushes problems that Lint would catch cheaply into the heavy compile stage. Lint is always the first gate.

CHECKPOINT

Run Compile



Compile operation and platform validation

CORE IDEA

Compile is the second link of the validation chain and heavier than Lint — it runs the platform compile/parse (parsevl) for deeper validation. Use it, after Lint passes, to confirm the project actually compiles on the platform. Note: Compile is not meant to be your first trial-and-error loop.

OPERATIONS

1. Confirm Lint has passed and cheap issues are cleared, then run Compile (parsevl)
2. Read the compile result and error locations in the Detail Log
3. Pin each error to a specific file and line, then return to the source + ProjectMeta to find the root cause
4. After fixing, recompile to confirm it passes

EXAMPLE / EXERCISE

Run the platform compile on the project:

```
parsevl
```

If it reports

```
Services/TaskService.vs:12  type mismatch: status expected enum, got text
```

go back to Task.vdb, define status as an enum, align the service parameter, and recompile.

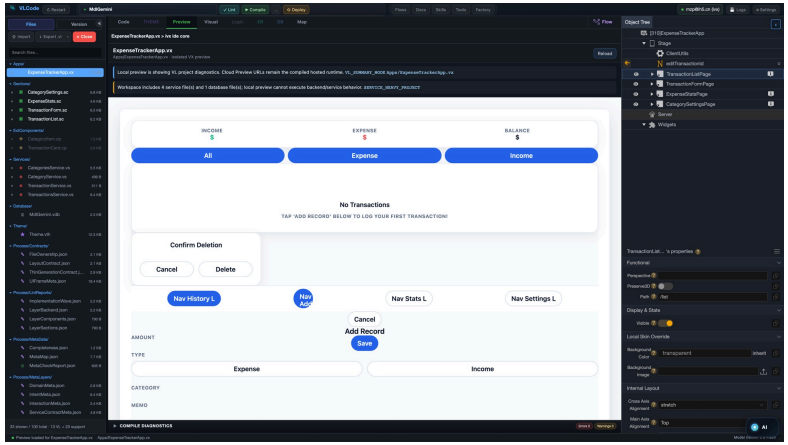
PITFALL

Treating Compile as a first repeated trial-and-error loop is slow and hides the root cause. The order must be Lint first, Compile second; on a Compile error, return to the source for the root cause rather than recompiling on luck.

CHECKPOINT

With Lint passing, Compile also passes; if errors appear, you can pin them to a file/line and fix the root cause from the source and ProjectMeta.

Open Preview



Application preview after generation

CORE IDEA

Preview is the last link of the validation chain, opening the real running page via /route/<path> once Compile passes. For the board, this is the first true user's-eye check: the three columns render, you can create a task, and clicking toggles status. Looking right isn't enough — you actually click through it.

OPERATIONS

1. After confirming Compile passes, open Preview
2. Visit the /route/<path> for the board page
3. Actually operate it: create a task and watch it appear in the To-Do column
4. Click that task and confirm its status toggles correctly across the three columns

EXAMPLE / EXERCISE

Open the board page preview (path per your project's actual route):

```
/route/board
```

Create a task titled "Prepare weekly sync" and confirm it lands first in To-Do; click it twice and watch it move to In-Progress, then Done.

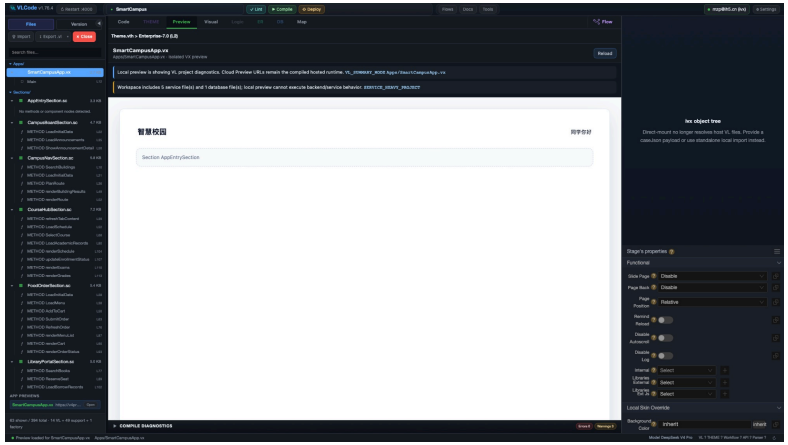
PITFALL

Only viewing the rendered Preview without actually clicking through the interaction misses "renders fine but clicks do nothing" bugs. Always create and click-toggle for real — verify behavior, not just appearance.

CHECKPOINT

Preview opens the board page, and you can create a task and toggle its status across the three columns by clicking — behavior, not just appearance, verified.

Record First-Round Issues



A visual object tree failure context for debugging practice

CORE IDEA

After the first Lint→Compile→Preview pass, there are almost always a few misalignments. Recording the issues faithfully — which file, what symptom, which link — sets up minimal, precise fixes instead of changing things from memory. The act of recording is itself the discipline that avoids broad rewrites.

OPERATIONS

1. List every issue found in Lint, Compile, and Preview, one by one
2. Tag each with: the file involved, the error symptom, and which validation link surfaced it
3. Separate "real problems" from "acceptable differences" and record only the real ones
4. Flag any "same error appearing twice" — that is the signal to return to the source + ProjectMeta for the root cause

EXAMPLE / EXERCISE

Keep a simple list:

```
1. Board.sc: In-Progress column empty — card uses status value doing, table uses in_progress (found in Preview)
2. TaskService.vs: createTask doesn't return the new task id (Compile warning)
3. New Task button does nothing — onClick has no action (found in Preview)
```

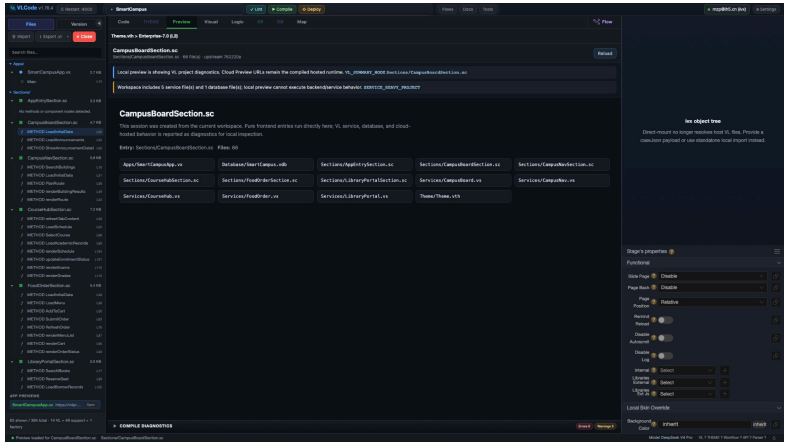
PITFALL

Not recording and acting from memory leads to changing too much at once and introducing new bugs, violating "avoid broad rewrites." List first, then fix each minimally.

CHECKPOINT

You have a first-round issue list categorized by file and validation link, with any error-that-repeated-twice flagged.

Apply Minimal Fixes With CompileFix



A logic-panel failure context for debugging practice

CORE IDEA

compile-fix is a repair workflow built for compile errors. Its principle is the minimal fix: change the file the error points to, not regenerate the whole project. Pair it with root-cause-first discipline — if the same error appears twice, return to the source + ProjectMeta for the root cause rather than recompiling on luck.

OPERATIONS

1. Hand the compile/reference issues from your first-round list to the compile-fix workflow
2. Have it make minimal changes to only the erroring files, leaving the rest untouched
3. After the change, re-run Lint + Compile to confirm the issue is gone and nothing new broke
4. If the same error has now appeared twice, stop patching and go back to the source and ProjectMeta for the root cause

EXAMPLE / EXERCISE

Specify the fix in AI Chat:

```
Use compile-fix: Board.sc card status value doing doesn't match the Task table's
in_progress; only align the value, don't rewrite other files.
```

Then re-run parsevl to confirm the compile passes.

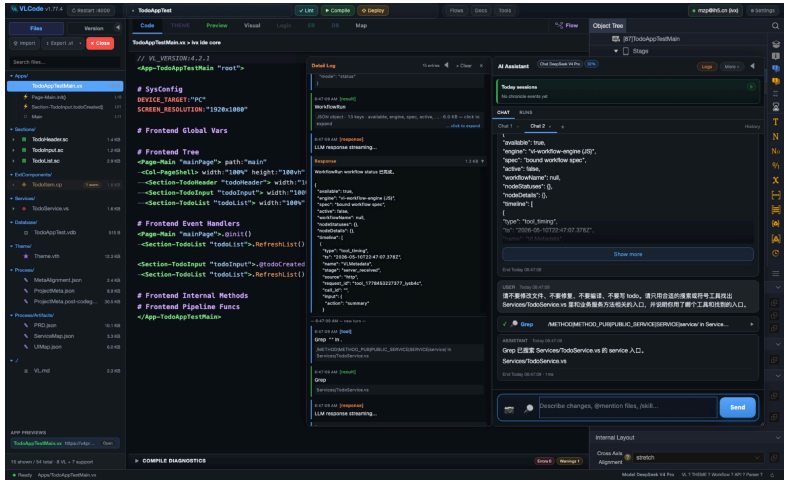
PITFALL

Using a large generation workflow to "redo" the project just to fix one small error overwrites already-verified files and introduces new problems. Repair with compile-fix as a minimal change; don't rewrite when you don't have to.

CHECKPOINT

You applied a minimal fix with compile-fix for the first-round compile/reference issues, recompiled successfully, and broke none of the already-verified files.

Finish The First Delivery



Final IDE state after project creation

CORE IDEA

"Done" for version one has a clear bar: Lint→Compile→Preview all pass, the board's core interactions actually work in Preview, and all source files sit under ~/Documents/VLProjects/TeamPM/ ready to open and export. Local-first means the deliverable is this source directory itself, with no cloud lock-in.

OPERATIONS

1. Confirm all three validation links pass: Lint, Compile, and Preview with no blocking issues
2. Walk the full core flow in Preview: create a task plus toggle status across the three columns
3. Verify the TeamPM source directory is complete, names are English PascalCase, and every file starts with VL_VERSION
4. (Optional) Bind the cloud GID and deploy as an optional target; not deploying still counts as delivered

EXAMPLE / EXERCISE

Write the delivery checklist for version one as:

```
[x] Lint passes
[x] Compile passes
[x] Preview: can create a task and click-toggle status
[x] Source complete under ~/Documents/VLProjects/TeamPM/
```

All four met means version one is delivered; further expansion goes through the incremental workflows in Part VII.

PITFALL

Treating "Compile passes" as delivered and skipping the hands-on Preview often surfaces broken interactions only after release. The minimum delivery bar is that it actually clicks through in Preview, not just that it compiled.

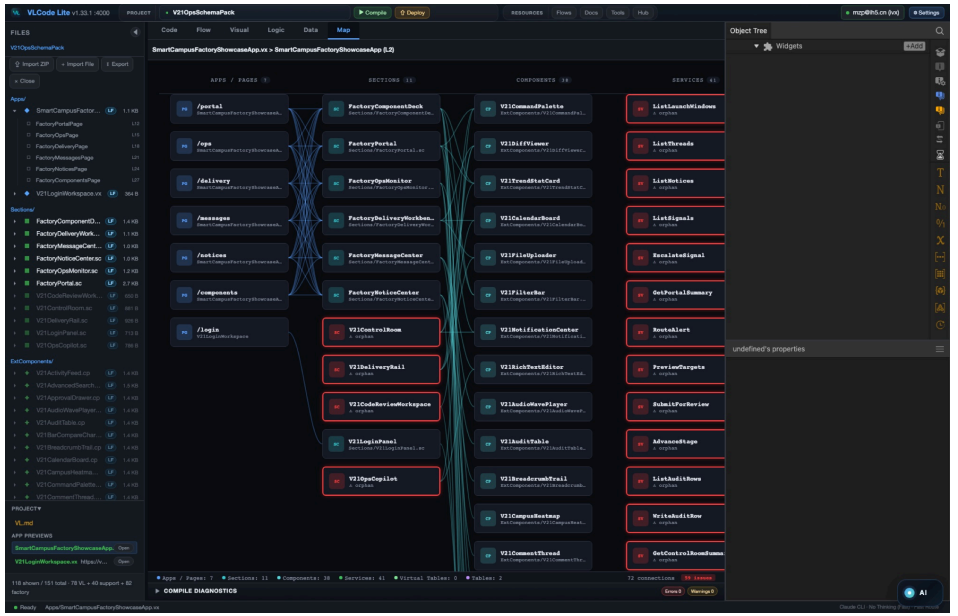
CHECKPOINT

All three of Lint/Compile/Preview pass, the board's core interactions work in Preview, and the TeamPM source directory is complete — version one is delivered.

CHAPTER

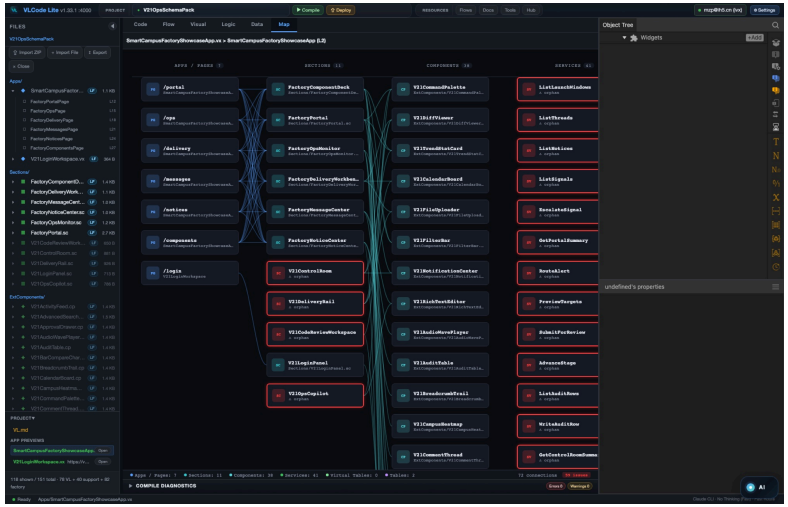
Part VII: Incremental Changes And Professional Workflows

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



Map mode for application structure

Add A Page With add-page



Map mode for application structure

CORE IDEA

add-page is an incremental workflow that adds one new page (a Section, a .sc file) to an existing app without regenerating the whole project. It reads ProjectMeta and wires the new page into routing, the object tree, and navigation instead of generating from scratch. Reach for it whenever you need one more screen on an app that already exists, rather than re-running a full generation.

OPERATIONS

1. In AI Chat describe the page you want and which app it belongs to, e.g. an Archive page in TeamPM
2. Pick add-page in the workflow selector so it runs incrementally against the current ProjectMeta
3. Watch the Detail Log right panel for the new .sc file and the new route entry it adds to ProjectMeta
4. Open the Map project view to confirm the page is wired into routing and navigation
5. Run Lint -> Compile, then open Preview at /route/<new-path> to verify it renders

EXAMPLE / EXERCISE

Type into AI Chat:

```
Add an "Archive" page to the TeamPM app that lists completed tasks sortable by completion date; wire it into the main nav with route /archive
```

After running add-page, the Detail Log shows it created `Sections/Archive.sc` and registered the `/archive` route in ProjectMeta.

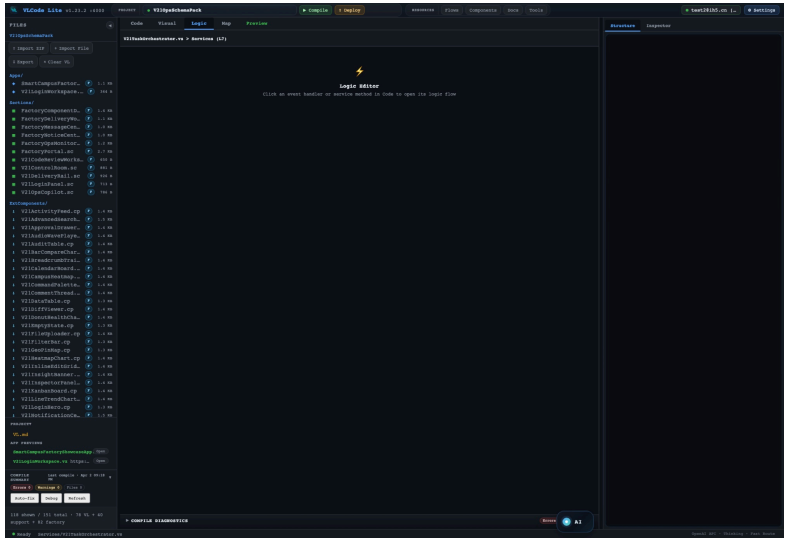
PITFALL

Using a full generation workflow to 'add a page' overwrites or rewrites existing pages and loses work. For a single page always use add-page so it edits ProjectMeta incrementally instead of re-running meta-direct-codegen.

CHECKPOINT

The Map view shows the new Section, Preview opens it at /route/<new-path>, and no existing page was changed.

Add A Service With add-service



Service logic panel for backend behavior

CORE IDEA

add-service incrementally adds a piece of backend logic (a Service, a .vs file), usually a set of new service methods. Because VLCode keeps frontend, backend, and theme decoupled, you can grow the service layer without touching pages. When a page needs a new data operation (a query, a save, an approval action), generate the methods first with add-service, then wire a button to them in Logic mode.

OPERATIONS

1. In AI Chat describe what the new service does and its inputs/outputs, e.g. archive a task, count open tasks per member
2. Pick the add-service workflow so it reads ProjectMeta and creates the matching .vs file
3. Check the Detail Log for the new service file and the names of the new methods
4. Open the DB logic panel to verify each method signature and body
5. Run Lint -> Compile to confirm there are no broken service-method references

EXAMPLE / EXERCISE

Type into AI Chat:

```
Add an ArchiveService to TeamPM with archiveTask(taskId) and countOpenByMember(); archiving writes the status field back to the tasks table
```

After add-service runs, the Detail Log shows it created `Services/ArchiveService.vs` containing both methods.

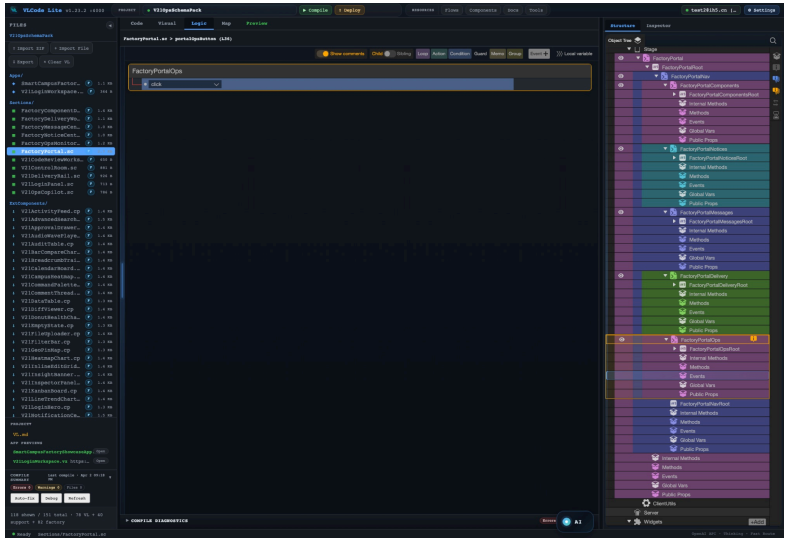
PITFALL

Wiring a page to a service method that doesn't exist yet produces a 'broken service method reference' at Compile time. Generate the method with add-service first, then bind it in Logic mode — don't reverse the order.

CHECKPOINT

The DB panel shows the new methods and Compile passes with no undefined-service-method errors.

Add A Feature With feature-adjust



Logic panel for event and action blocks

CORE IDEA

feature-adjust adds a complete small feature to an existing page or interaction — usually spanning UI, an event, and a service call, not just one spot. It reads and edits the relevant .sc / .vs files as needed and wires the new behavior into the Logic event panel. Use it when a one-line requirement implies end-to-end 'what happens when I click' logic, instead of hand-editing several files.

OPERATIONS

1. In AI Chat state the feature as one complete requirement, including the trigger and the expected result
2. Pick the feature-adjust workflow and run it
3. Track which pages, events, and services it touches in the Detail Log
4. Open the Logic event panel to confirm the new event (e.g. a button onClick) is bound to its action
5. Click through it in Preview to verify the end-to-end behavior, then run Lint -> Compile

EXAMPLE / EXERCISE

Type into AI Chat:

```
Add a "Mark Done" button to the TeamPM task card; clicking it calls the service to set the task status to done and refreshes the current list
```

feature-adjust edits the task-card Section, adds an onClick event, and wires it to the complete-task service method.

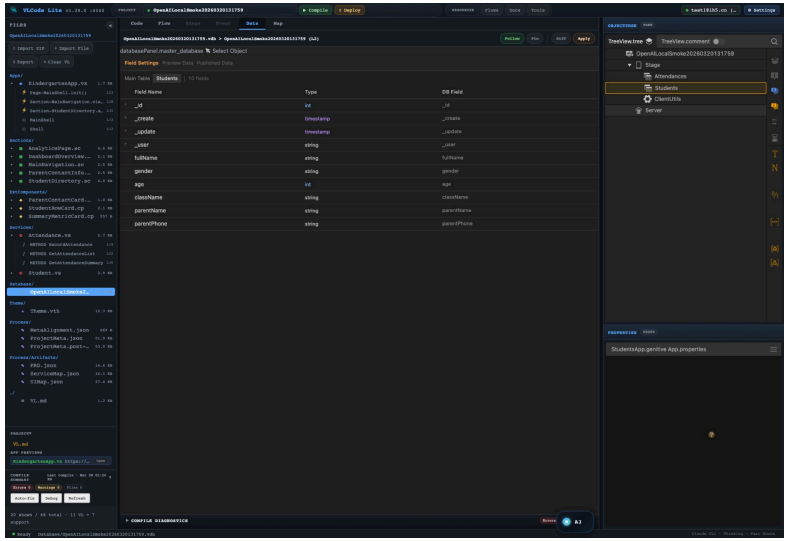
PITFALL

Cramming several unrelated features into one feature-adjust run triggers broad edits that are hard to trace. Describe one focused feature per run and split multiple features into separate runs.

CHECKPOINT

In Preview you can complete the full new feature (e.g. click the button and see the status change) and the Logic panel shows the matching event binding.

Modify Data With data-adjust



Data panel for tables and schema

CORE IDEA

data-adjust is the incremental workflow for the data layer (tables and fields in .vdb files): add a field, change a type, add a table, or adjust a relationship without rebuilding the whole database. Because tables are the contract that pages and services depend on, changing one ripples to its consumers, so the workflow must update ProjectMeta in lockstep to avoid field drift. Use it to evolve the data model instead of hand-editing the .vdb.

OPERATIONS

1. In AI Chat say exactly which table to change and which fields to add or modify, with types
2. Pick the data-adjust workflow and run it
3. Verify the schema change in the Data panel — confirm field names and types are as expected
4. Check the Detail Log to see ProjectMeta's data definition updated in sync
5. Run Lint -> Compile, watching whether services or pages that referenced the old field were updated too

EXAMPLE / EXERCISE

Type into AI Chat:

```
Add a priority field to TeamPM's tasks table (enum low/medium/high, default medium) and let the task list page filter by it
```

data-adjust edits the .vdb to add the field and updates the services and pages that reference that table.

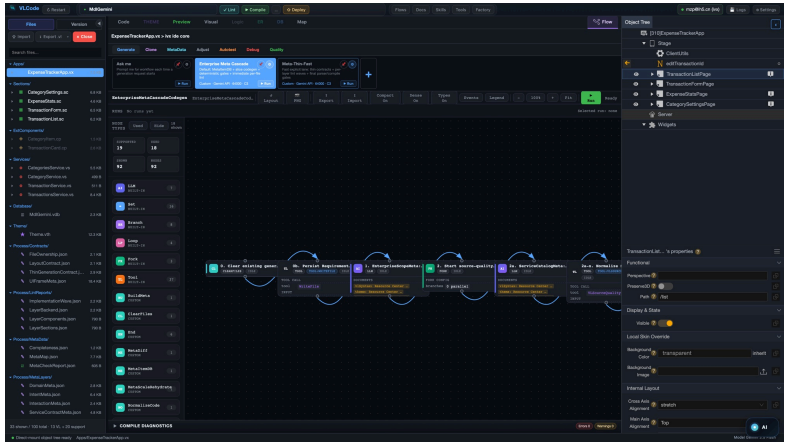
PITFALL

Editing a field in the .vdb but not the services and pages that read it causes 'data field drift' — Compile passes but runtime data doesn't line up. Make data-adjust update the consumers too and confirm in the Compile report that no stale old-field references remain.

CHECKPOINT

The Data panel shows the new field and no service or page still references a renamed or removed old field.

Batch Small Changes With batch-adjust



Flow mode: a DAG workflow shown as executable nodes

CORE IDEA

batch-adjust bundles a set of scattered small edits (reword copy, swap an icon, tweak spacing, normalize button naming) into one workflow run instead of changing and compiling them one by one. It suits a checklist of 'a dozen tiny fixes' that the AI can sweep through and apply uniformly. Combined with multi-task concurrency, it lets you peel routine touch-ups off your main line of work and process them in a batch.

OPERATIONS

1. Collect the edits into a checklist and list each one clearly in AI Chat
2. Pick the batch-adjust workflow and submit the whole batch at once
3. Verify in the Detail Log which file each individual edit landed in
4. Spot-check a few in Map or Visual to confirm nothing unrelated was touched
5. Run Lint -> Compile once for the whole batch rather than after each edit

EXAMPLE / EXERCISE

Type into AI Chat:

```
TeamPM batch tweaks: 1) rename every "Delete" button to "Remove"; 2) set task-card corner radius to 8px everywhere; 3) change the top nav title to "Team Board"; 4) standardize the empty-state text to "No tasks yet"
```

batch-adjust applies each one and lists in the Detail Log which file each change hit.

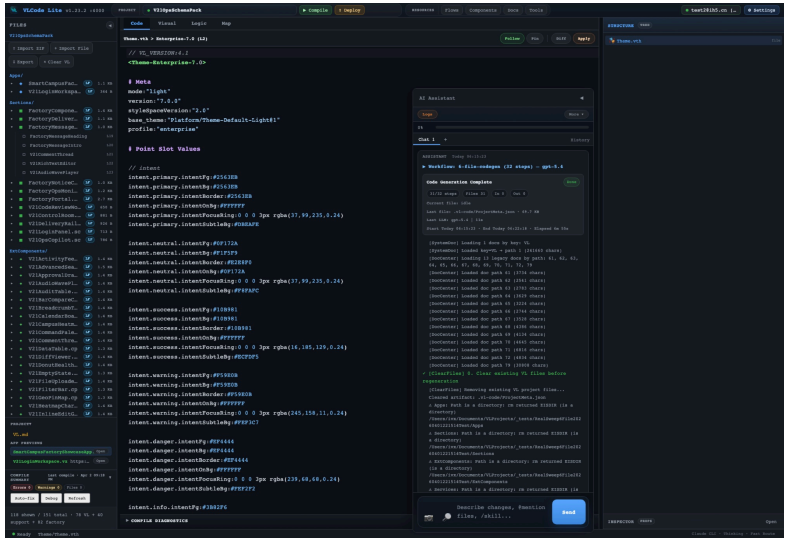
PITFALL

Slipping a functional requirement that needs real logic design into batch-adjust turns the batch into a broad rewrite. Keep batch-adjust to genuine small touch-ups; route new behavior or new data through feature-adjust / data-adjust instead.

CHECKPOINT

Every item on your checklist has a matching Detail Log entry, the whole batch passes one Compile, and unrelated content is untouched.

Customize Theme With theme-customize



Theme panel for enterprise visual tokens

CORE IDEA

theme-customize edits the theme layer (visual tokens in .vth files: colors, font sizes, radii, spacing) to change the whole app's look without touching page structure or logic. Because theme and components are decoupled, restyling won't break behavior; but components reference tokens rather than hard-coded values, so editing a token is what makes a change global. Use it to reskin or align branding instead of restyling page by page.

OPERATIONS

1. In AI Chat describe the visual change you want, e.g. primary color, corner radius, type scale
2. Pick the theme-customize workflow and run it
3. Verify the tokens updated as expected in the THEME panel
4. Look at a few pages in the Visual canvas or Preview to confirm the change is global and consistent
5. Run Lint -> Compile to confirm components still reference the theme tokens correctly

EXAMPLE / EXERCISE

Type into AI Chat:

```
Change TeamPM's primary color from blue to brand green #1B8A5A, set primary-button radius to 6px everywhere, and bump body text up one size
```

theme-customize edits the matching tokens in the .vth and every button and text block updates uniformly.

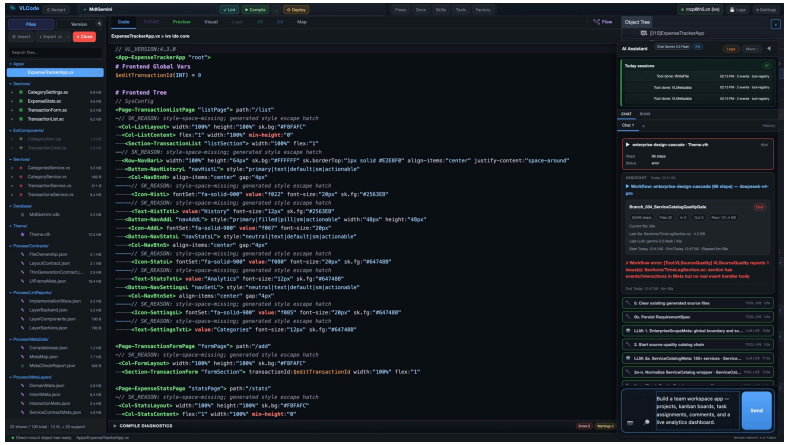
PITFALL

Hard-coding a color or size in one component instead of going through the theme causes 'theme/component mismatch' — those hard-coded spots don't follow when you edit the theme. Always change the .vth tokens and keep components referencing tokens, not literal values.

CHECKPOINT

After editing one theme token, every page that references it changes in Preview, with no page stuck on the old styling.

Recover Missing Files With complete-files



Detail Log with node progress and tool output

CORE IDEA

complete-files is a repair workflow that fills in files left missing when a generation run was interrupted (dropped connection, abort, error). It compares against ProjectMeta to find files that 'should exist but weren't written' and completes only those, rather than re-running the whole project. When a big run finished only partway, use it to resume — far cheaper than starting over.

OPERATIONS

1. In the Detail Log confirm the last run's terminal state (done / error / aborted) and which files are missing
2. In AI Chat pick the complete-files workflow so it completes against ProjectMeta
3. Watch the Detail Log list the files it found missing versus ProjectMeta and newly wrote
4. Confirm in the left file tree that Apps/Sections/Services are now complete
5. After completion run Lint -> Compile to verify the project is whole

EXAMPLE / EXERCISE

Type into AI Chat:

```
The TeamPM generation was aborted while writing Services and a few service files never got created; use complete-files to fill in the missing files against ProjectMeta and don't touch the pages already generated
```

The Detail Log shows it wrote the missing .vs files while leaving existing files alone.

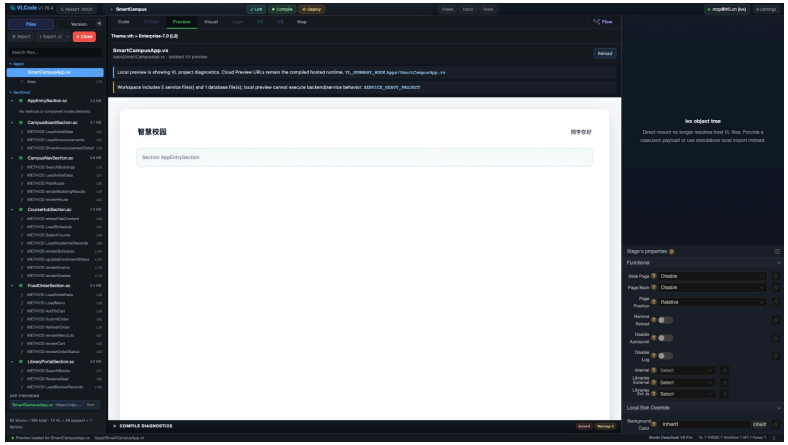
PITFALL

Re-running the full workflow after an interruption overwrites files that were already generated and wastes time and tokens. Use complete-files first to fill only the missing pieces incrementally.

CHECKPOINT

Every file declared in ProjectMeta now exists, the file tree has no gaps, and the whole project compiles.

Debug A Single File



A visual object tree failure context for debugging practice

CORE IDEA

debug-single-file narrows the repair to one file, used when an error clearly localizes to a specific .sc / .vs / .vdb and you don't want collateral edits elsewhere. It reads that file plus ProjectMeta context and makes a minimal fix. Preferring it over multi-file debug is how the 'avoid broad rewrites' principle plays out during repair.

OPERATIONS

1. Locate the failing file and the exact error in the Detail Log or Compile report
2. In AI Chat pick debug-single-file and name the target file
3. Let it produce a minimal fix using that file plus ProjectMeta
4. Review in the Code editor that the edit stays within that single file
5. Run Lint then Compile to confirm the file's error is gone

EXAMPLE / EXERCISE

Compile reports:

```
Error in Sections/TaskList.sc: unknown service method `loadTasks`
```

In AI Chat say: "Use debug-single-file to fix Sections/TaskList.sc — change the reference to loadTasks to the existing listTasks; don't touch other files."

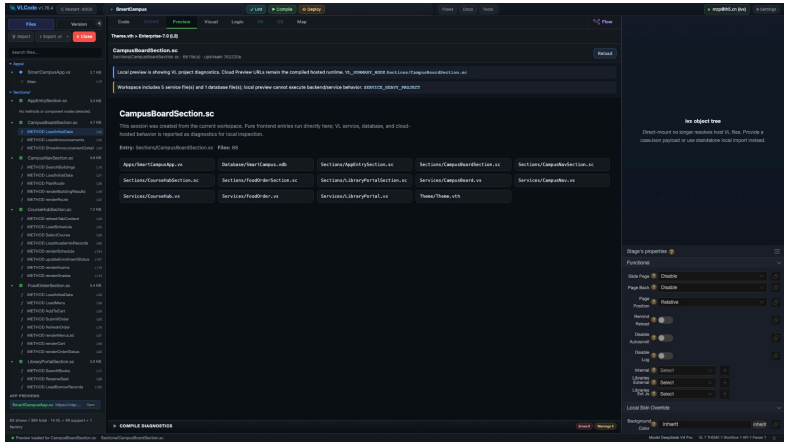
PITFALL

Letting the AI sweep the whole project for a clearly single-file problem just introduces new broad edits. When the error localizes to one file, keep the fix strictly inside that file.

CHECKPOINT

Only the target file changed, and its error no longer appears after Lint and Compile.

Debug Multiple Files



A logic-panel failure context for debugging practice

CORE IDEA

debug-multi-file handles errors that span several files — typically one contract change cascading into breakage, e.g. after renaming a service method signature the page references and data fields no longer line up. It reads the related files plus ProjectMeta together and makes a coordinated, consistent fix. Use it when the root cause is 'cross-file contract inconsistency' rather than a typo in a single file.

OPERATIONS

1. Gather the multiple errors in the Detail Log and check whether they point to one root cause
2. Return to source and ProjectMeta to confirm this is a cross-file contract issue, not a single-spot typo
3. In AI Chat pick debug-multi-file and describe the files involved and the shared root cause
4. Let it reconcile across files, keeping service methods, page references, and data fields consistent
5. Run Lint -> Compile to confirm all the related errors clear together

EXAMPLE / EXERCISE

After a service renames `getTasks()` to `listTasks()`, several errors appear:

```
Sections/TaskList.sc: unknown method getTasks
Sections/Dashboard.sc: unknown method getTasks
```

In AI Chat say: "Use debug-multi-file to update every page that references getTasks to listTasks, keeping them consistent with TaskService."

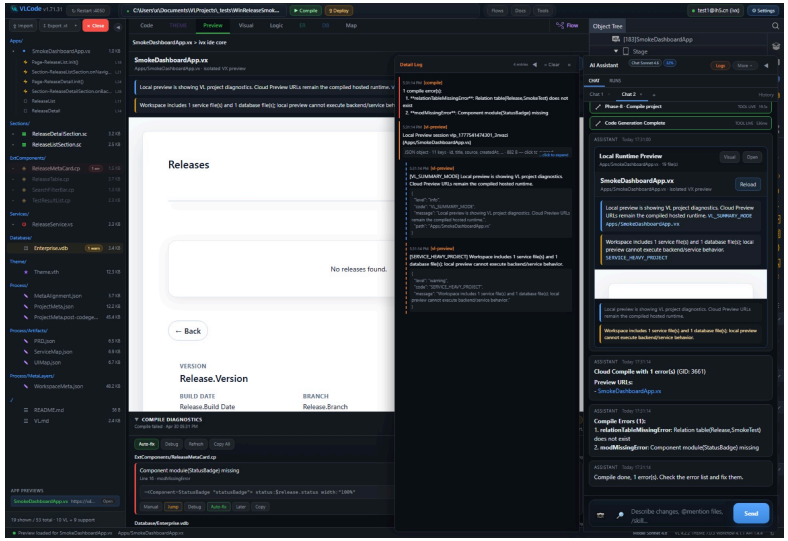
PITFALL

Patching in place when the same error appears twice usually means you skipped the root cause. The rule: if the same error repeats twice, go back to source and ProjectMeta for the root cause, then reconcile once with debug-multi-file instead of patching file by file.

CHECKPOINT

Every file referencing that contract uses the same version, and the multiple errors clear together in one Compile.

Import Design From Screenshot Or HTML



Compile preview inside the IDE

CORE IDEA

VLCode has a family of design-first workflows that turn a visual (screenshot or HTML/CSS) into a real, runnable VL app — smart-design-fullstack-codegen settles the HTML design first then generates full-stack code, and html-first-vl-codegen / site-clone-codegen take a similar HTML-first path. They let you go from a mockup straight to real pages instead of guessing layout from a text description. Ideal when you already have a visual you want faithfully reproduced.

OPERATIONS

1. In AI Chat attach the design (screenshot or HTML/CSS) and state which page and scope to reproduce
2. Pick smart-design-fullstack-codegen (or html-first-vl-codegen) so it generates code from the design
3. Track the Sections, Services, and data structures it produces in the Detail Log
4. Compare layout against the original in the Visual canvas, then open the live page in Preview
5. Run Lint -> Compile and do one human review pass against the mockup to fix deviations

EXAMPLE / EXERCISE

Type into AI Chat:

```
Here's a design screenshot of the CustomerCRM customer-detail page (attached); use smart-design-fullstack-codegen to reproduce it in VL: a customer info card on top, a contact-history timeline on the right, an action button row at the bottom
```

The workflow starts from the HTML design and generates the matching .sc plus supporting services.

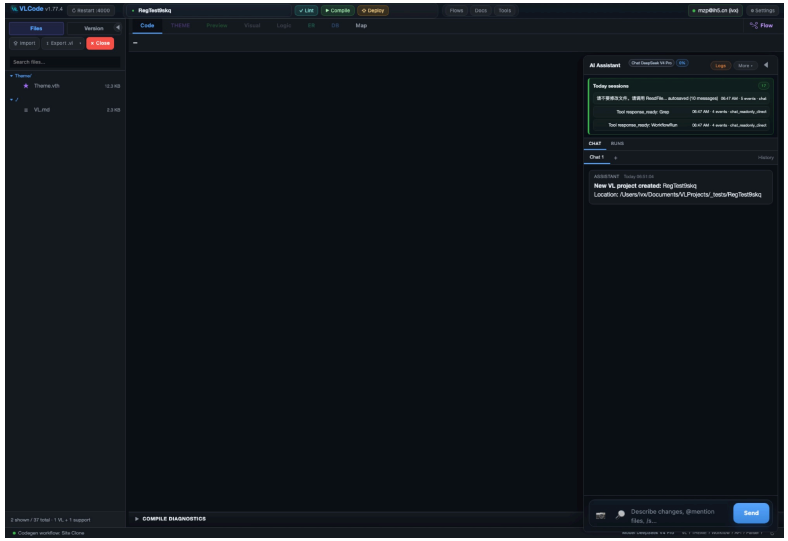
PITFALL

Expecting one run to reproduce a complex design pixel-perfect usually disappoints. Treat it as a faithful starting point: after generating, compare in Visual and fine-tune with incremental workflows rather than re-running full generation repeatedly.

CHECKPOINT

The Preview page matches the mockup's layout closely and, as real VL files, passes Compile

Upload And Categorize A .vflow



Workflow selector before execution

CORE IDEA

A .vflow is the workflow bundle that connects VL Agent to VLCode (workflow JSON + manifest, plus optional subflows/docs/prompts). In VLCode you can drag a ready-made .vflow into the Flow panel and, on import, assign it a category so it joins your list of available workflows. This is the entry point for pulling a prebuilt professional workflow into your project without authoring a DAG yourself.

OPERATIONS

1. Switch to the Flow workflow panel
2. Drag the .vflow file in, or click '+ Upload .vflow'
3. On import choose one of 8 categories: Generate / Enterprise / Clone / MetaData / Test / Adjust / Autofix / Quality
4. Confirm the manifest parsed and any subflows/docs/prompts were recognized
5. Find it afterward in the workflow selector and trial-run it against a project as needed

EXAMPLE / EXERCISE

You receive `approval-flow.vflow`, drag it into the Flow panel, and on import pick the `Adjust` category (or `Enterprise` if that fits its purpose). It then shows in the workflow list and can be selected in AI Chat:

```
Use the approval workflow I just imported to add a manager approval step to refund requests in CustomerCRM
```

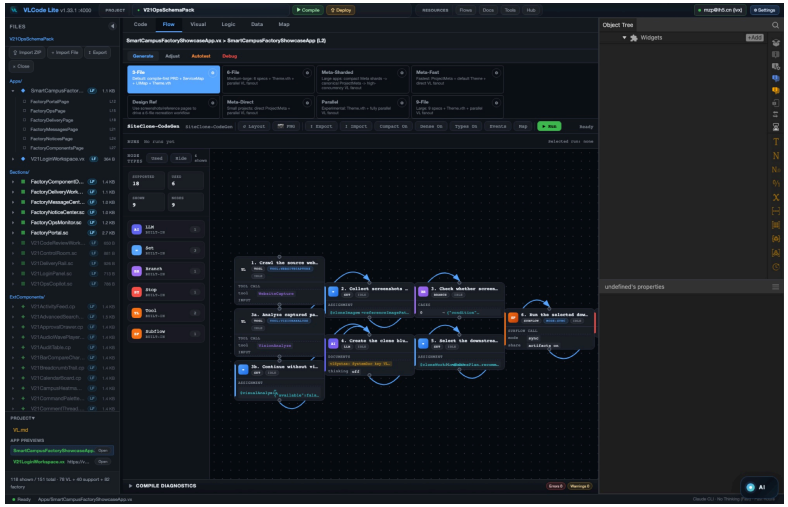
PITFALL

Picking a careless category at import time makes the .vflow hard to find later in a long workflow list. Categorize by the .vflow's real purpose (Generate for generators, Adjust for tweaks) so you can locate it again.

CHECKPOINT

The uploaded .vflow appears under the correct category in the Flow workflow list and can be selected and run.

Reuse Workflows Through Resource Center



Flow mode in the local-first IDE

CORE IDEA

The Resource Center is VL's distribution channel for workflows and resources: VL Agent authors package a workflow as a .vflow, register it, and cloud-sync it, and VLCode developers browse, import, customize, and use it on their own projects. This lets professional workflows (approval flows, specialized generators) be reused across teams instead of rebuilt each time. It is the mechanism for banking 'process intelligence' and skipping repeated setup cost.

OPERATIONS

1. Confirm Settings has the LLM provider and platform account ready so you can pull from the Resource Center
2. Browse available workflows in the Resource Center (including .vflow shipped from VL Agent)
3. Import the workflow you want so it joins your local available list
4. Customize it as needed for this project's ProjectMeta and requirements
5. Select it in AI Chat, run it against the current project, and verify the result in the Detail Log

EXAMPLE / EXERCISE

State the reuse intent in AI Chat:

```
Find an enterprise approval workflow in the Resource Center, import it, adapt it to the CustomerCRM refund process, and run it against the current project
```

On the author side a VL Agent user registers the .vflow to their local Resource Center and cloud-syncs it; on the VLCode side you can then browse and import it.

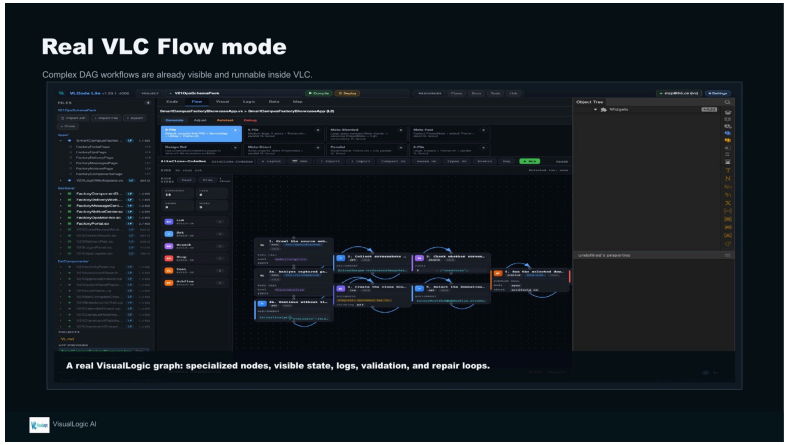
PITFALL

When the Resource Center is slow or unavailable, don't get stuck — VLCode bundles seed prompts and docs so core codegen runs offline. Don't halt your in-progress generation just to wait on the Resource Center.

CHECKPOINT

You successfully import a workflow from the Resource Center and use it to produce a run on your own project that's visible in the Detail Log.

File Ownership And Parallel Generation



DAG harness: routing and review

CORE IDEA

VLCode splits a project into small files (each Section, Service, and table is its own file) with clear ownership. One IDE window = one workspace = one backend, but multiple chat sessions and multiple workflow runs can run in parallel. Small files plus clear ownership keep parallel generation from colliding on the same file — which is exactly what lets large systems be sharded and generated fast.

OPERATIONS

1. Use the Map project view to understand file ownership and which workflow will write which files
2. Assign non-overlapping tasks to different chat sessions and run them in parallel (one backend per window)
3. Track in the Detail Log which files each run writes and confirm there's no overlapping contention
4. When needed, abort one line by chat scope or workflow scope without affecting the other runs
5. Once the lines converge, run one Compile to confirm everything fits together

EXAMPLE / EXERCISE

Drive two lines in parallel on TeamPM:

```
Session A: use add-page to generate a "Reports" page (writes Sections/Report.sc)
Session B: use add-service to generate ReportService (writes Services/ReportService.vs)
```

The two lines write different files concurrently; to stop one, choose 'abort this chat' or 'abort this workflow' as needed.

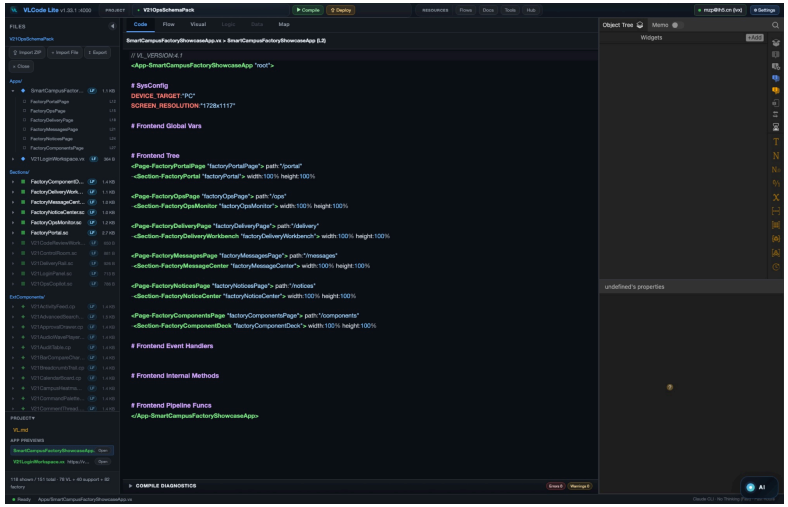
PITFALL

Pointing two parallel runs at the same file makes them overwrite each other and conflict. Before going parallel, use Map to confirm the tasks own disjoint files, then run them separately.

CHECKPOINT

Two workflows run in parallel writing their own files without conflict, and the merged project passes one Compile.

Avoid Broad Rewrites



Code mode for inspecting and editing VL source

CORE IDEA

'Avoid broad rewrites' is the core discipline behind VLCode's incremental workflows: if a problem can be solved with a minimal edit, never regenerate the whole project. Broad rewrites erase verified work, waste tokens, and can introduce new errors. The right move is to return to source and ProjectMeta for the root cause and apply targeted change with incremental workflows like add-page / add-service / *-adjust / debug-*.

OPERATIONS

1. When a problem appears, first scope it: single file, cross-file contract, or genuinely new structure
2. Pick the smallest-scope workflow: debug-single-file for one file, batch-adjust for small tweaks, add-page/add-service to add
3. If the same error repeats twice, stop and go back to source and ProjectMeta for the root cause instead of blind patching
4. Verify in the Detail Log the change surface of each run and confirm no unrelated files were touched
5. Only when the structure truly needs reworking should you consider a larger-scope workflow, after backing up/confirming

EXAMPLE / EXERCISE

The task list is missing sorting — don't regenerate all of TeamPM. In AI Chat say:

```
Just add sort-by-due-date in Sections/TaskList.sc using feature-adjust; don't touch other pages or services
```

That locks the change to one file and one behavior.

PITFALL

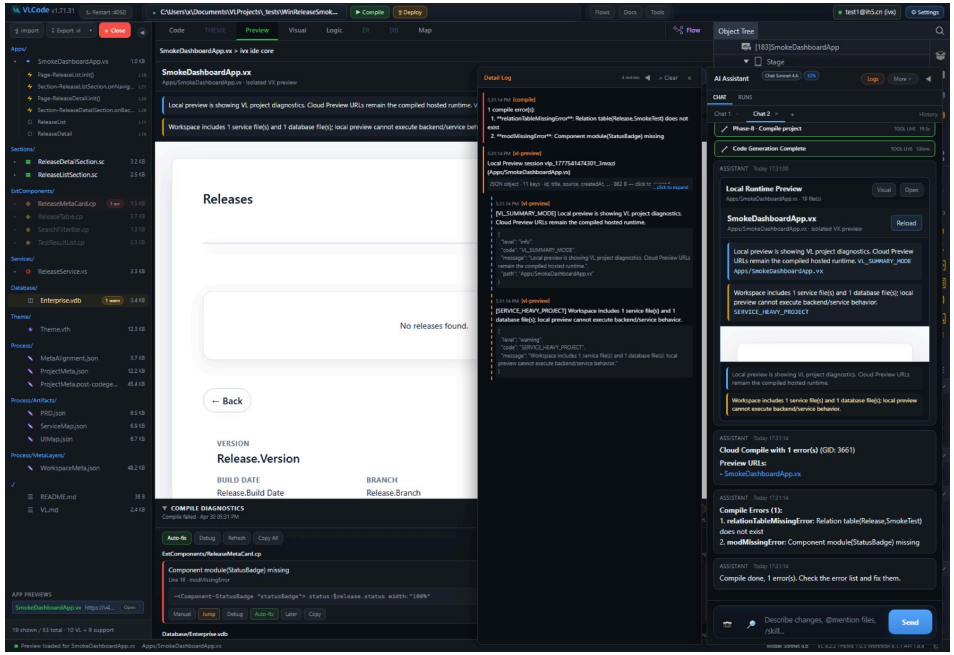
Asking the AI to 'just redo the whole thing' at the first error feels easy but discards verified content and introduces new bugs. Hold to the minimal-edit principle: find the root cause first, then fix it pointwise with an incremental workflow.

CHECKPOINT

CHAPTER

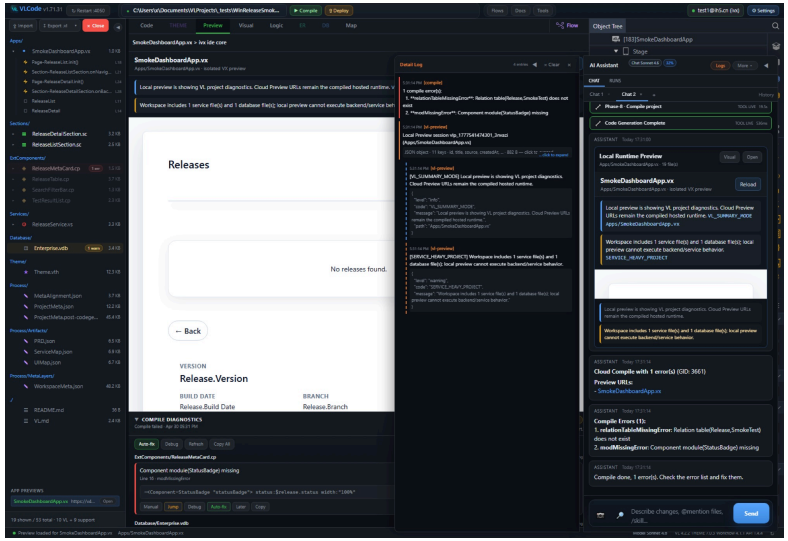
Part VIII: Validation, Debugging, Release, And Delivery

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



Compile preview inside the IDE

Validation Order: Lint → Compile → Preview



Compile preview inside the IDE

CORE IDEA

VLCode's validation chain is a fixed three-stage escalation: Lint first (lintPjt, a project-level static check, lowest cost), then Compile (parsevl, heavier platform validation), and finally Preview (actually open the page at /route/<path>). The point of the order is to let the cheapest, fastest check catch the bulk of low-level errors so you never use the heavy platform compile as your first trial loop.

OPERATIONS

1. Run Lint first from AI Chat, or let the generation workflow auto-run lintPjt at the end, and read the static-check result in Detail Log
2. Once Lint is green, trigger Compile (parsevl) for the heavier platform validation
3. After Compile passes, open the Preview route /route/<path> in the browser to verify pages and interactions for real
4. If any stage fails, stop and fix at that stage, then resume downward — never skip a stage

EXAMPLE / EXERCISE

Standard three-step rhythm (TeamPM board example):

```
1. Lint    → lintPjt reports 0 errors
2. Compile → parsevl passes
3. Preview → open /route/board, watch task cards render
```

In Chat you can simply say: "Run Lint first, only Compile when it's green, then open the board page in Preview."

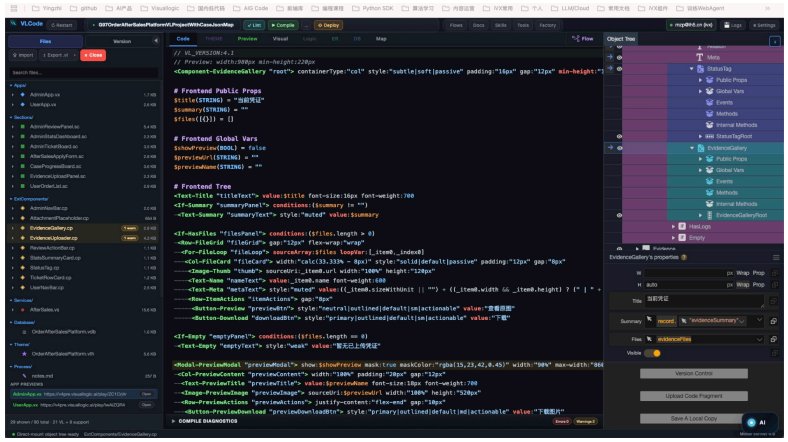
PITFALL

The most common mistake is hammering Compile as your trial loop from the start — platform compile is slow and expensive and wastes time and tokens. Always let Lint run first to catch the cheap errors.

CHECKPOINT

You can state the three-stage order Lint → Compile → Preview from memory and explain why Compile should not be the first loop.

When To Stop Recompiling



Compile operation and platform validation

CORE IDEA

More Compile is not better. VLCode's discipline: if the same error appears twice in a row, stop recompiling and go back to the source file and ProjectMeta to find the root cause, avoiding broad rewrites. Blindly re-running Compile both hides the real problem and burns tokens.

OPERATIONS

1. On the first Compile failure, read the full error in Detail Log, apply a minimal fix, and compile once more
2. If the second compile shows the same error, stop the compile loop — that is the 'same error twice' signal
3. Go back to the relevant .vx/.sc/.vs source and ProjectMeta to locate the root cause (missing method, broken reference, field drift)
4. Make a targeted minimal fix instead of letting AI rewrite the whole file set, then re-run Lint → Compile

EXAMPLE / EXERCISE

Exiting an error loop correctly:

```
Compile #1 → error: service method `createTask` not found
(minimal fix: add createTask in the Service)
Compile #2 → same error: `createTask` not found
→ STOP. Return to ProjectMeta — does the Section reference a different service name?
```

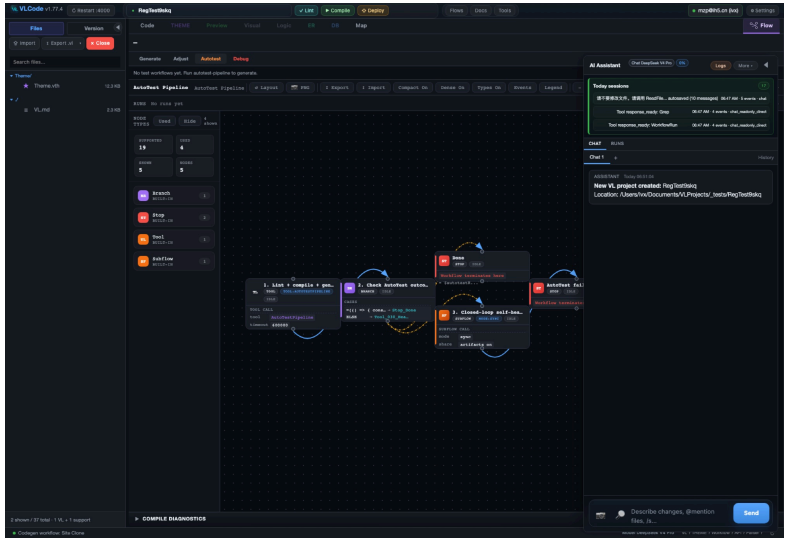
In Chat: "This error has now appeared twice — stop compiling, go back to ProjectMeta and the source for the root cause, make only a minimal fix."

PITFALL

Users fall into a 'maybe one more compile fixes it' loop and let AI do ever-larger rewrites, spreading the problem. The moment you see the same error a second time, switch to root-cause investigation.

CHECKPOINT

Read A Lint Report



Logs and execution details after a run

CORE IDEA

The Lint report comes from lintPjt, the project-level static check — the first and cheapest gate in the validation chain. Its output appears line-by-line in Detail Log, the full run ledger (not repeated in Chat); Chat shows only final summaries and human decisions. Reading the Lint report means locating each static error by file.

OPERATIONS

1. After triggering Lint, open the right-side Detail Log panel — the full run ledger of node progress, file writes, validation, and errors
2. Read each Lint entry: identify which .sc/.vs/.vdb file, which line, and what type
3. Prioritize fundamental errors first — e.g. VL_VERSION not on line 1, or indentation using spaces instead of consecutive hyphens
4. Apply a minimal fix in the source file, then check Detail Log to confirm Lint clears to zero

EXAMPLE / EXERCISE

Typical Lint entries and fixes:

```
[lint] Sections/Board.sc:1  VL_VERSION must be the first line
→ fix: put // VL_VERSION:4.3.1 as line 1, nothing before it

[lint] Services/Task.vs:12  mixed indentation
→ fix: replace space indentation with consecutive hyphens (- / -- / ---)
```

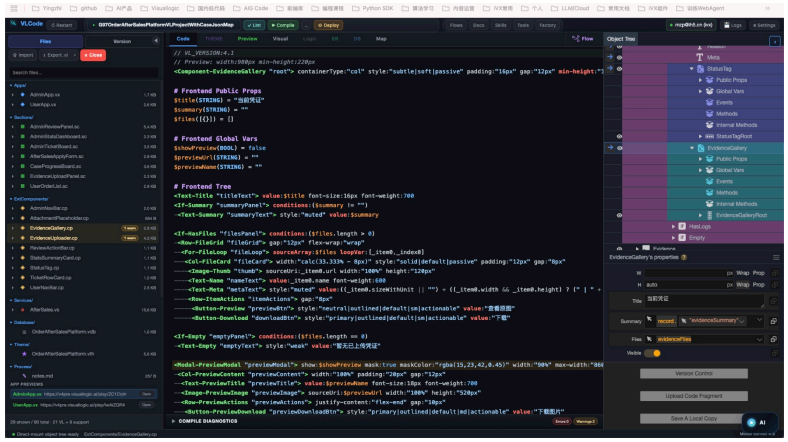
PITFALL

Beginners look for line-by-line Lint errors in Chat, but Chat only carries the summary — the full per-line record is in Detail Log. Looking in the wrong place makes you think 'no errors were reported.'

CHECKPOINT

You can find a specific Lint error in Detail Log, name its file and line, and make a minimal fix that clears that entry.

Read A Compile Report



Compile operation and platform validation

CORE IDEA

The Compile report comes from parsevl, the platform compile — a heavier layer than Lint that catches cross-file semantic errors Lint cannot: missing service methods, broken Section references, mismatched data fields. It also appears line-by-line in Detail Log and is the gate that decides whether you may move on to Preview.

OPERATIONS

1. Once Lint is green, trigger Compile and read the parsevl result in Detail Log
2. For each compile error, confirm the file and the symbol involved (method name / Section / field)
3. Cross-check against ProjectMeta — many compile errors are really a mismatch between Meta-declared relationships and the source
4. Apply a minimal fix and recompile; remember the same-error-twice rule and switch to root cause instead of rewriting

EXAMPLE / EXERCISE

Typical Compile entry:

```
[compile] Sections/Board.sc  calls TaskService.archive() — method not declared
→ root cause: the Service only has close(); the Section calls a non-existent archive()
→ minimal fix: add archive() in the Service, or have the Section call close()
```

In Chat: "Compile reports archive() is not declared — add that method to TaskService and touch no other files."

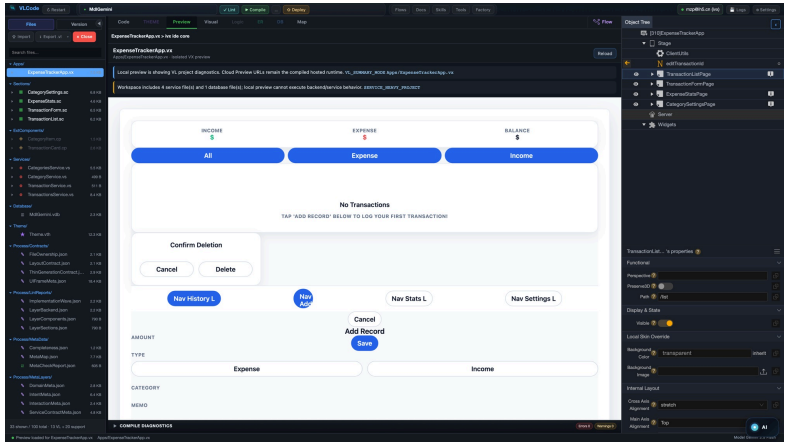
PITFALL

Treating a Compile error as a pure syntax issue and hand-editing the source without consulting ProjectMeta — many compile errors are Meta relationship drift, and source-only edits keep retriggering the same error.

CHECKPOINT

You can read a parsevl compile error, name the file and symbol involved, and clear it with a minimal fix guided by ProjectMeta.

Preview Routes And /route/path



Application preview after generation

CORE IDEA

Preview is the final stage of the validation chain — meaningful only after Lint and Compile both pass. It opens a concrete page via the /route/<path> route, where <path> maps to a Section/page in the project. This is the only stage where you actually see whether rendering, interactions, and data work as intended; the first two only prove 'it compiles', not 'it looks right'.

OPERATIONS

1. Confirm Lint and Compile have both passed before opening Preview
2. Visit the target page as /route/<path>, where <path> is that Section's route
3. Actually exercise the page: click buttons, submit forms, switch lists, and check interactions fire
4. When something is wrong, trace it back to the relevant Section/Service source rather than guessing in the preview

EXAMPLE / EXERCISE

Previewing the customer list page in CustomerCRM:

```
Preview → /route/customers
```

Open a customer, click 'Add Contact', and watch the list refresh. If the button does nothing, return to that Section's Logic panel to check the click event, then check the Service to confirm the method actually exists.

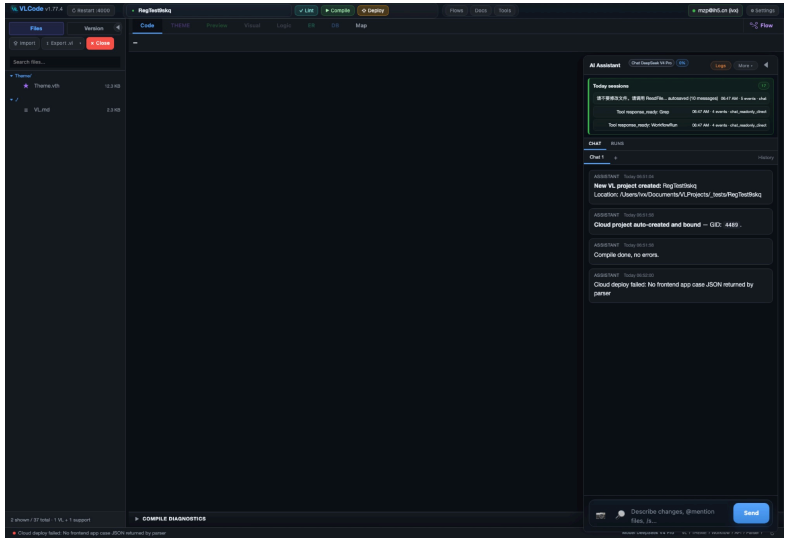
PITFALL

Treating Preview as the first check, skipping Lint/Compile and opening the page directly — you end up looking at a half-built or stale-compiled page and waste time. Preview is always the last stage.

CHECKPOINT

You can open any page via /route/<path> and exercise one interaction to confirm it behaves as intended.

Pre-Deploy Checklist



Deploy operation after validation

CORE IDEA

The pre-deploy checklist gathers scattered validation into one repeatable 'before-release self-check'. It introduces no new mechanism — it walks through the Lint → Compile → Preview chain, file-naming discipline, ProjectMeta consistency, and GID binding status item by item, so what you deliver is verified, not hopeful.

OPERATIONS

1. Confirm Lint (lintPjt) is zero-error, Compile (parsevl) passes, and key pages preview correctly for real
2. Confirm every VL source file name is English PascalCase and line 1 of each file is VL_VERSION
3. Cross-check ProjectMeta so apps/sections/services/dataSchema dependencies have no broken references or field drift
4. Confirm Theme matches components; for cloud release, confirm the GID is bound correctly and has permission
5. Check the Status Bar shows a healthy LLM provider/model, compile status, and ports

EXAMPLE / EXERCISE

TeamPM before-release self-check:

```
[ ] Lint 0 errors        [ ] Compile passes
[ ] /route/board preview OK   [ ] file names English PascalCase
[ ] VL_VERSION on line 1     [ ] ProjectMeta deps intact
[ ] Theme matches components  [ ] GID bound (if cloud release)
```

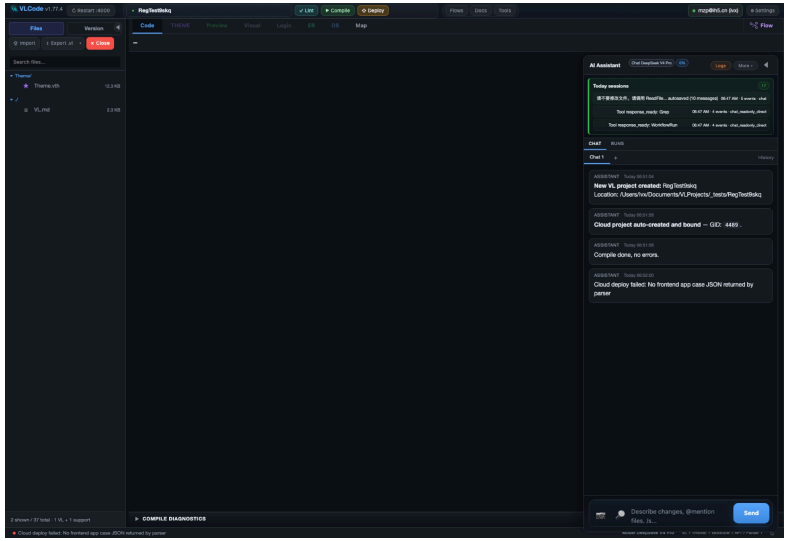
PITFALL

Treating a passing Compile as 'done' and shipping without the Preview test and the name/Meta checks — the breakage only surfaces once it runs in the cloud. The checklist's value is forcing every item to be reviewed.

CHECKPOINT

You can complete a pre-deploy checklist without skipping items and give a clear pass/fail judgment for each one.

Cloud Deployment As An Optional Target



Deploy operation after validation

CORE IDEA

VLCode is a local-first IDE: local read/write plus AI collaboration is the main line, and the cloud is an optional delivery target, not the only surface. You can do generation, validation, preview, and delivery entirely locally; you only take the cloud step when you need to publish the app to a team or externally, done through a bound GID.

OPERATIONS

1. Do all development locally by default: generate, Lint, Compile, and local Preview
2. Consider cloud deployment only when you actually need an external or team-facing release
3. Before a cloud release, use the GID binding button in the sidebar header to confirm binding to the correct cloud project GID
4. After release, return to local iteration — the cloud is a delivery target, not the development surface

EXAMPLE / EXERCISE

Keeping the main line and the optional step straight:

```
Local (main line): generate → Lint → Compile → /route/ preview → deliver source
Cloud (optional): bind GID → deploy for the team to access
```

If you only use it yourself or hand over the source directory, you never need to touch the cloud step.

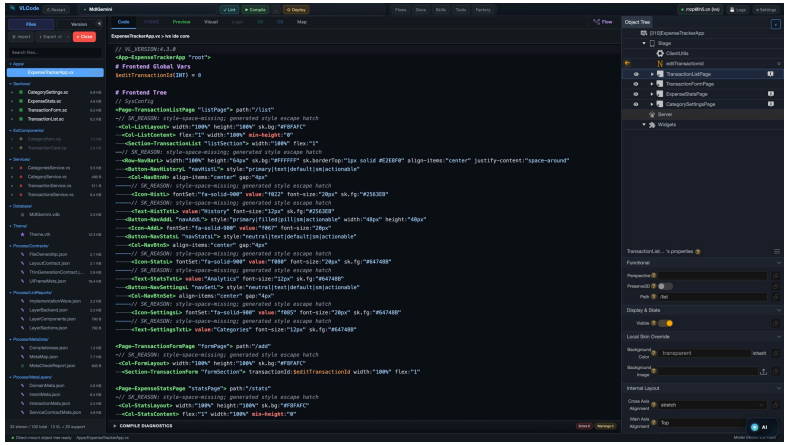
PITFALL

Assuming 'it isn't done until it's in the cloud' and agonizing over deployment too early. Local-first means: no publish need, no cloud — don't turn an optional target into a mandatory step.

CHECKPOINT

You can explain that cloud deployment is an optional delivery target rather than a required step, and judge whether a given project truly needs it.

Export Source And Avoid Lock-In



Code and Visual editing modes side by side

CORE IDEA

Because VLCode is local-first, a project's real artifact is the plain VL source (.vx/.sc/.cp/.wc/.vs/.vdb/.vth) plus ProjectMeta (.vl-code/ProjectMeta.json) under ~/Documents/VLProjects/<Name>/ — sitting on your local disk, readable, versionable, and transferable. Delivery is the source directory itself, with no dependency on a cloud account or proprietary runtime, which fundamentally avoids vendor lock-in.

OPERATIONS

1. Find the full project directory under ~/Documents/VLProjects/<Name>/ (Config/Apps/Sections/Services/Database/Theme/Process)
2. Confirm all source files use English PascalCase names with VL_VERSION on line 1, so others can pick it up
3. Put the directory under version control or simply archive it as the deliverable itself
4. Confirm it opens and continues in VLCode on another machine with no cloud dependency

EXAMPLE / EXERCISE

Delivering a CustomerCRM source directory:

```
~/Documents/VLProjects/CustomerCRM/
├── Apps/CustomerCRM.vx   Sections/      Services/
├── Database/             Theme/         ExtComponents/
└── Process/              Config/
```

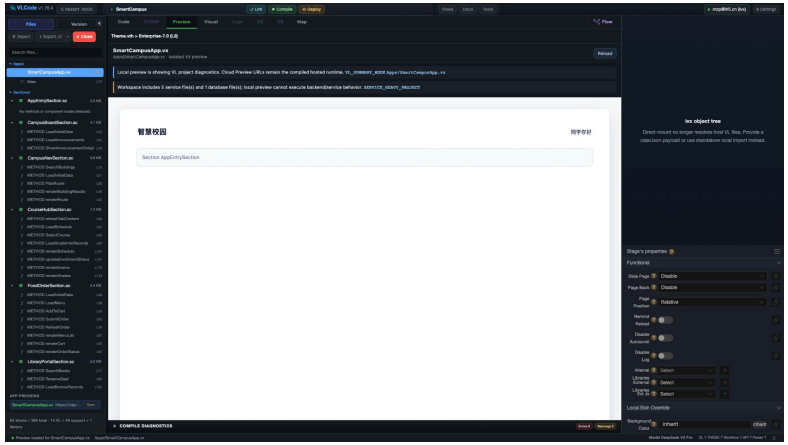
Hand over the whole directory; the recipient opens it in their own VLCode directly, no cloud account of yours required.

PITFALL

Assuming the app 'lives in the cloud' and delivering only a cloud link, leaving the recipient unable to take over offline or migrate. The real deliverable is the local source directory.

CHECKPOINT

Common Error: Non-English File Names



A visual object tree failure context for debugging practice

CORE IDEA

VL projects have a hard rule: file names must be English PascalCase. Giving a .vx/.sc/.vs file a Chinese or non-ASCII name causes errors during generation, compile, or import. This discipline ranks with VL_VERSION on line 1 and consecutive-hyphen indentation as a non-negotiable basic convention.

OPERATIONS

1. Check Sections/, Services/, Apps/ and similar folders for any Chinese or non-ASCII file names
2. Rename such files to English PascalCase (e.g. '任务看板.sc' → Board.sc)
3. Update ProjectMeta and every reference to that file so nothing breaks
4. Re-run Lint → Compile to confirm the error is gone

EXAMPLE / EXERCISE

Wrong vs fixed:

```
✗ Sections/任务看板.sc        ✗ Services/客户服务.vs
✓ Sections/Board.sc           ✓ Services/CustomerService.vs
```

When issuing a generation prompt in Chat, state it too: "Name all files in English PascalCase, no Chinese names."

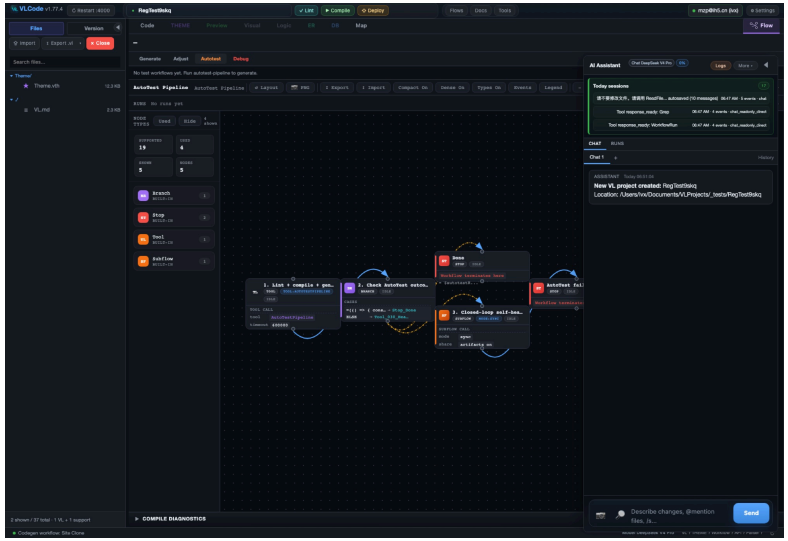
PITFALL

Writing requirements in Chinese and casually letting AI name files in Chinese, which yields non-English file names. Constrain the naming rule in the prompt up front to prevent it at the source.

CHECKPOINT

Your project directory has no Chinese/non-ASCII file names — all VL files are English PascalCase and Lint/Compile no longer error on this.

Common Error: Old GID Permission



Logs and execution details after a run

CORE IDEA

When syncing or publishing a project to the cloud, you bind it to a cloud project GID. Binding to an old GID you have no permission for makes cloud operations fail. The GID binding button now lives in the sidebar header, dedicated to binding or rebinding the cloud project GID.

OPERATIONS

1. Find the GID binding button in the sidebar header and see the currently bound GID
2. If a cloud operation reports a permission error, first check whether that GID is old or not yours
3. Use the binding button to rebind to a correct GID you have permission for
4. Retry the cloud sync/publish and confirm via Status Bar and Detail Log that the permission error is gone

EXAMPLE / EXERCISE

Troubleshooting path:

```
Cloud publish → error: permission denied (gid 4836)
→ check whether the sidebar-header GID is stale or someone else's
→ use the GID binding button to rebind to a GID you own
→ retry publish
```

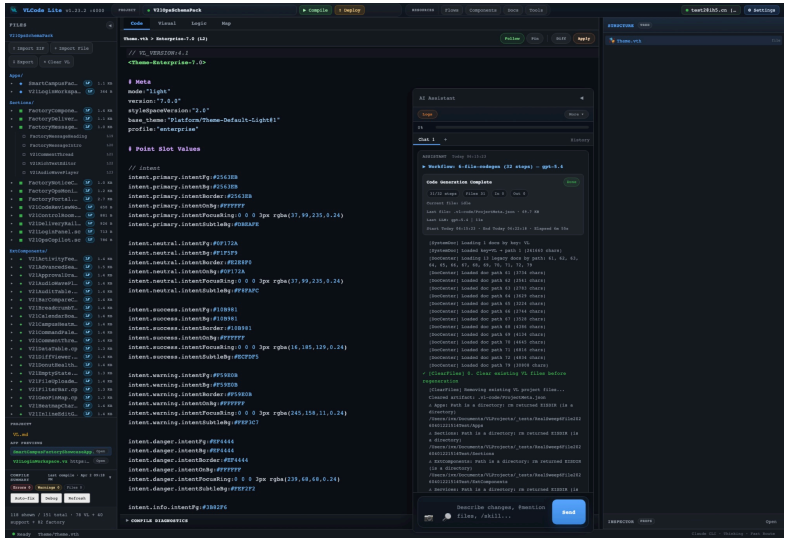
PITFALL

Reusing someone else's or an old project's GID to publish, then repeatedly failing on permissions while blaming the network. The first step is always to verify the currently bound GID in the sidebar header.

CHECKPOINT

You can view and rebind the GID in the sidebar header so a cloud operation that failed on an old GID succeeds after the rebind.

Common Error: Theme/Component Mismatch



Theme panel for enterprise visual tokens

CORE IDEA

VL decouples frontend, backend, and theme: components (properties/methods/events) pull visual tokens (colors, spacing) from the .vth theme. When a component references a token that doesn't exist or was renamed in the current Theme, you get a Theme/component mismatch — broken styling, missing-token errors. The THEME panel is the workbench for diagnosing this.

OPERATIONS

1. In the THEME panel, review which visual tokens the current .vth defines
2. Cross-check the affected Section/Component to confirm the token it references actually exists in the Theme
3. If a Theme rename/deletion caused it, either restore the token in the Theme or point the component at an existing token
4. Recompile and check in Preview that styling is consistent again

EXAMPLE / EXERCISE

Typical mismatch:

```
Component references token: color.brandPrimary
THEME panel only has:        color.primary
→ mismatch: the component cannot resolve brandPrimary
→ fix: add brandPrimary in the Theme, or switch the component to color.primary
```

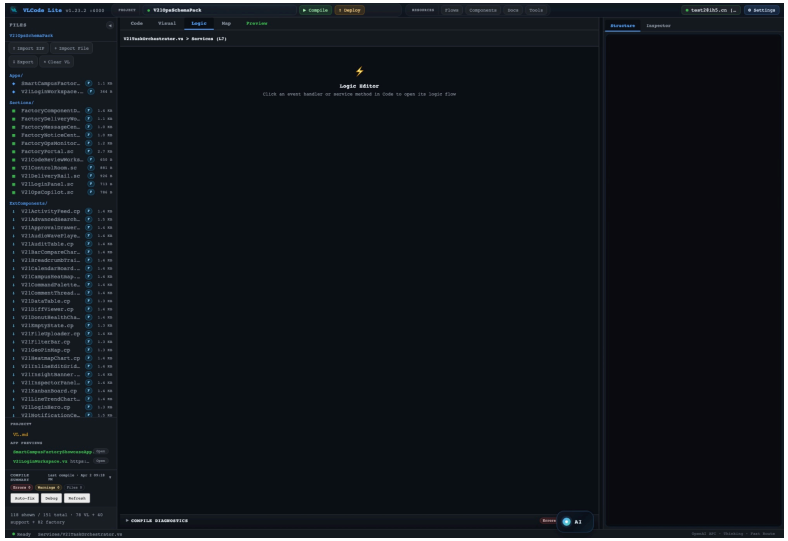
PITFALL

Renaming a Theme token but forgetting to update the components that reference it (or vice versa) — the price of decoupling is keeping token names aligned on both sides. When you change one side, always check the other.

CHECKPOINT

You can locate a token a component references but the Theme lacks, realign Theme and component, and confirm correct styling in Preview.

Common Error: Broken Service Method Reference



Service logic panel for backend behavior

CORE IDEA

In VL, a Section's click/submit behavior typically calls a backend method in a Service (.vs). When the method name the Section calls doesn't exist in the Service (renamed, deleted, or never generated), you get a broken service method reference — usually surfacing at Compile. The DB logic panel is where you verify which methods actually exist.

OPERATIONS

1. In the Compile report, locate a 'method not found / not declared' error and note the calling Section and the missing method name
2. Open the DB logic panel and confirm which methods that .vs actually defines
3. Decide whether the Service is missing the method (add it) or the Section called the wrong name (fix the call)
4. Cross-check ProjectMeta for consistent service dependencies, then recompile to verify

EXAMPLE / EXERCISE

Break and minimal fix:

```
[compile] Sections/Customers.sc → CustomerService.deleteAll() not declared
DB panel actually has: remove(id)
→ root cause: the Section calls a non-existent deleteAll()
→ fix: change the Section to call remove(id), or add deleteAll() in the Service
```

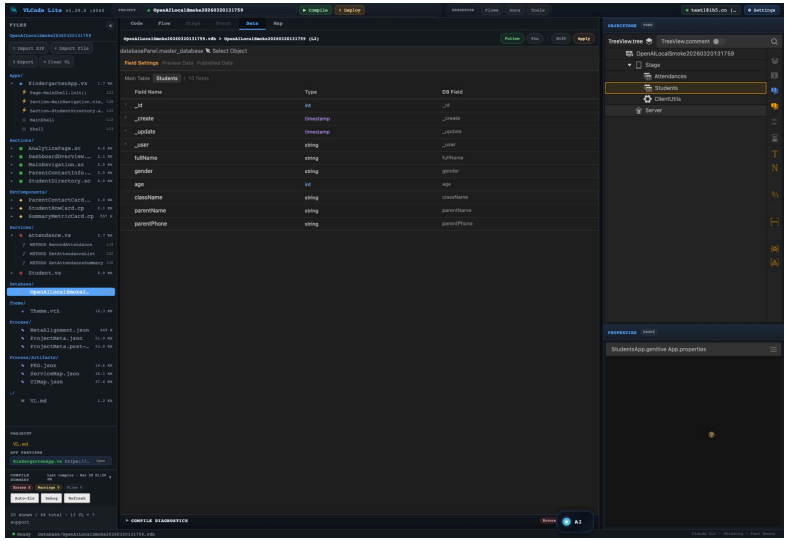
Chat: "The Customers page calls a non-existent deleteAll() — switch it to the existing remove(id) in the Service, don't create a new method."

PITFALL

On a 'method not found', blindly letting AI create a new same-named method in the Service, producing duplicates or empty stubs. Read the existing methods in the DB panel first, then decide to add or to re-point the call.

CHECKPOINT

Common Error: Data Field Drift



Data panel for tables and schema

CORE IDEA

Data tables (.vdb) define tables and fields, and Sections, Services, and ProjectMeta all reference those fields. When somewhere uses a field name the table doesn't actually have (spelling mismatch, renamed/deleted field), you get data field drift — a field-not-found error at Compile or runtime. The Data panel is the authoritative source for the real table schema.

OPERATIONS

1. Open the relevant .vdb in the Data panel and see exactly which fields and types the table defines
2. Compare against the field name referenced at the error site and find the one that doesn't match the schema
3. Decide whether it is a misspelled/renamed field (fix the reference to the real field) or a genuinely missing field (add it to the .vdb)
4. Sync ProjectMeta's data declarations, then recompile to verify alignment

EXAMPLE / EXERCISE

Pinpointing the drift:

```
Data panel, table Task fields: id, title, status, ownerId
Section binds to:              assignee   ← not a field on the table
→ root cause: field drift (should be ownerId)
→ fix: change the binding to ownerId, or add an assignee field to Task
```

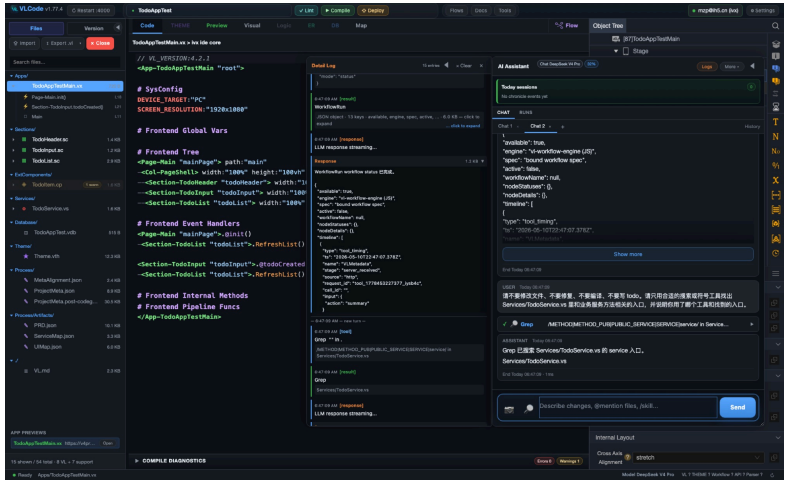
PITFALL

Writing field names from memory in a Section without checking the real schema in the Data panel, causing field-name drift. Confirm the field exists in the Data panel before binding to it.

CHECKPOINT

You can compare against the real schema in the Data panel, find a referenced-but-missing field, and eliminate the drift by fixing the reference or adding the field.

Delivery Package And Source Directory



Final IDE state after project creation

CORE IDEA

VLCode's deliverable is the project's own source directory: ~/Documents/VLProjects/<PascalCaseName>/, with the standard layout Apps/ Sections/ Services/ Database/ Theme/ ExtComponents/ Process/ Config/. This directory is self-contained — all VL source plus ProjectMeta — and local-first means delivery needs no cloud; whoever receives the directory can continue in their own VLCode.

OPERATIONS

1. Confirm the project lives under ~/Documents/VLProjects/<Name>/ with all standard subdirectories present
2. Complete the pre-deploy checklist: Lint/Compile/Preview pass, English PascalCase names, consistent ProjectMeta
3. Archive the entire project directory as the delivery package, preserving the structure
4. Include startup instructions (open in the desktop app, or run from source with npm run dev to http://localhost:4000)

EXAMPLE / EXERCISE

Delivering a TeamPM package:

```
~/Documents/VLProjects/TeamPM/
├── Apps/TeamPM.vx   ├── Sections/   ├── Services/      ├── Database/
├── Theme/           ├── ExtComponents/  ├── Process/   └── Config/
```

Delivery note: open this directory in VLCode; for source run, `npm run dev` then visit http://localhost:4000.

PITFALL

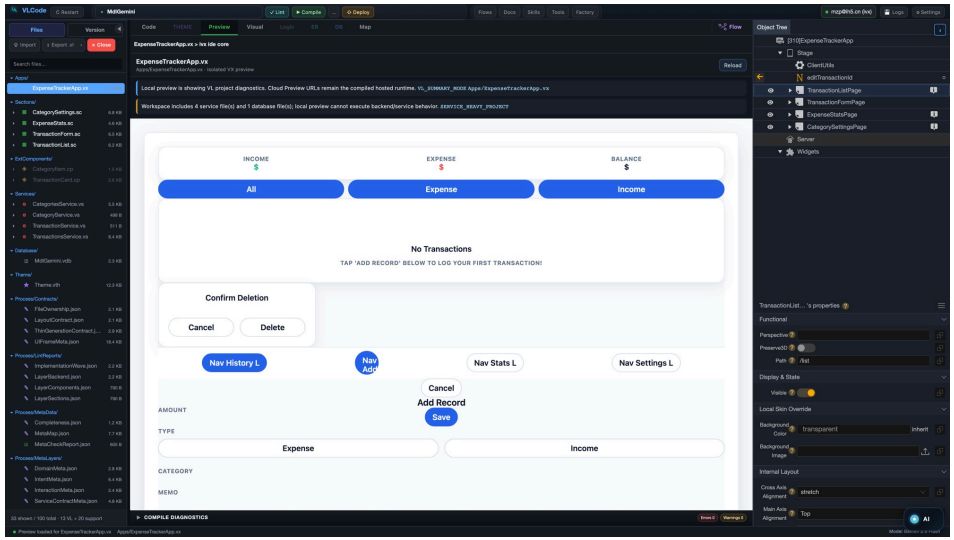
Packaging only Sections or a few files and dropping Config/ProjectMeta or Theme, so the recipient hits broken dependencies on open. The deliverable is the complete standard directory, not scattered files.

CHECKPOINT

CHAPTER

Part IX: Complete Case Studies

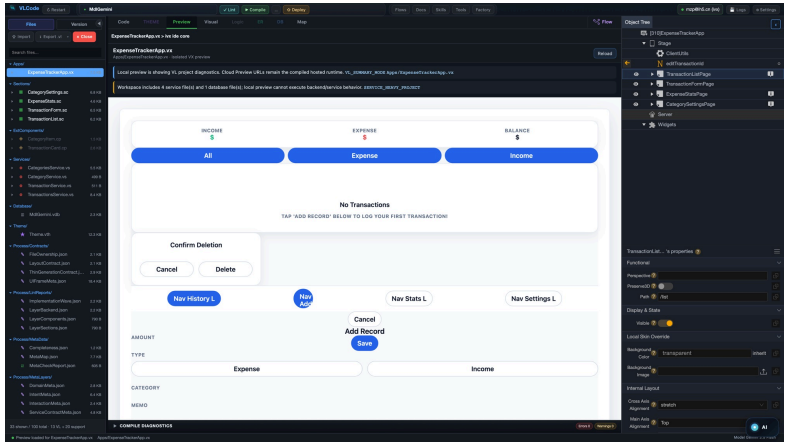
- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



Application preview after generation

PART IX: COMPLETE CASE STUDIES

Case 1: Todo Application



Application preview after generation

CORE IDEA

Case 1 is the beginner end-to-end drill: generate a minimal working Todo app from scratch in VLCode, covering one data table, one page, and basic create/check/delete interactions. It is small enough for the meta-direct-codegen workflow, so you watch the whole chain from requirement to Preview in a single run and build the mental model.

OPERATIONS

1. Pick an English PascalCase project name under ~/Documents/VLProjects/, e.g. TodoApp (Chinese/non-ASCII file names error out immediately)
2. Open or create that workspace from the sidebar and confirm the Status Bar shows your LLM provider and port 4000
3. In AI Chat, describe the Todo app goal in one sentence as an executable requirement and name the meta-direct-codegen workflow
4. Start the run and track node progress and per-file writes in the right-side Detail Log (meta-direct is the single-stream small-project workflow — one pass for frontend + backend + data + theme)
5. After generation, walk Lint (lintPjt) → Compile (parsevl) → Preview, and open /route/todo-list to add and check items

EXAMPLE / EXERCISE

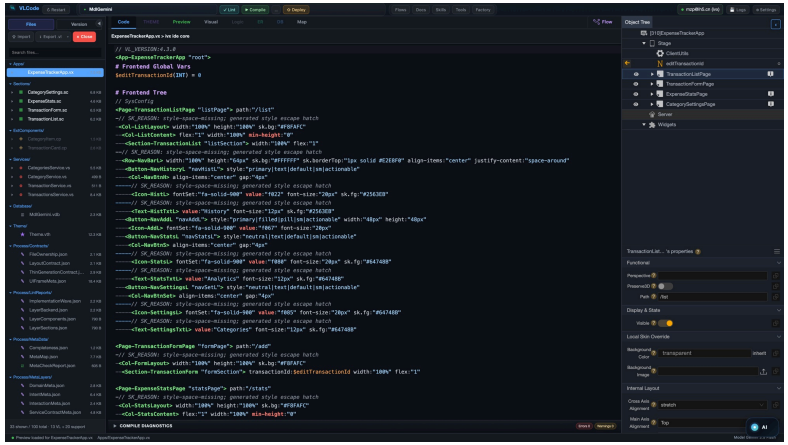
Type this requirement verbatim into AI Chat and name the workflow:

```
Use meta-direct-codegen to generate a single-user todo app TodoApp:
one todos table (title text, done bool default false, created time default now);
one list page TodoList — top input to add a todo, each row has a done checkbox and a delete button, completed rows show strikethrough.
```

When it finishes, ~/Documents/VLProjects/TodoApp/ should contain these files (Detail Log lists each write):

```
Apps/      TodoAppApp.vx              # app entry (frontend App)
           TodoAppBackendMainApp.vx   # backend main app
Sections/  TodoList.sc                # list page (add/check/delete)
Services/  TodoService.vs             # CRUD service methods
Database/  TodoApp.vdb                # todos table
Theme/     ThemeDefault.vth           # default theme (StyleSpace 2.0)
```

Case 1: Requirement Prompt



AI Chat used as the command and decision surface

CORE IDEA

The requirement prompt is your contract with AI Chat: it decides what gets generated. A good prompt states the goal as an executable, verifiable description — naming entities, fields, pages, and interactions rather than vague wishes. AI Chat is VLCode's primary interaction entry, where you issue natural-language commands and final summaries return.

OPERATIONS

1. In the AI Chat input, name the app (English PascalCase) and its purpose in one line
2. List the data entities and their key fields in short phrases
3. List the pages and the core interactions on each
4. If clarification is needed, the AI asks via an ask_user chat event (SSE) inline in the conversation — just reply, or use the custom-answer path (this is a chat event, not an interaction card)
5. After sending, watch Detail Log show how the workflow splits the requirement into nodes

EXAMPLE / EXERCISE

Write the Todo requirement as a structured prompt:

```
App TodoApp. Data: todos(title text, done bool, created time). Page: TodoList — top input to add a todo; each row has a done checkbox and a delete button; completed rows show strikethrough.
```

Every term maps to a verifiable artifact: `todos` →the table in Database/TodoApp.vdb; `TodoList` →Sections/TodoList.sc; the three interactions→three event chains in TodoList.sc plus methods in TodoService.vs. After generation, check them one by one to sign off.

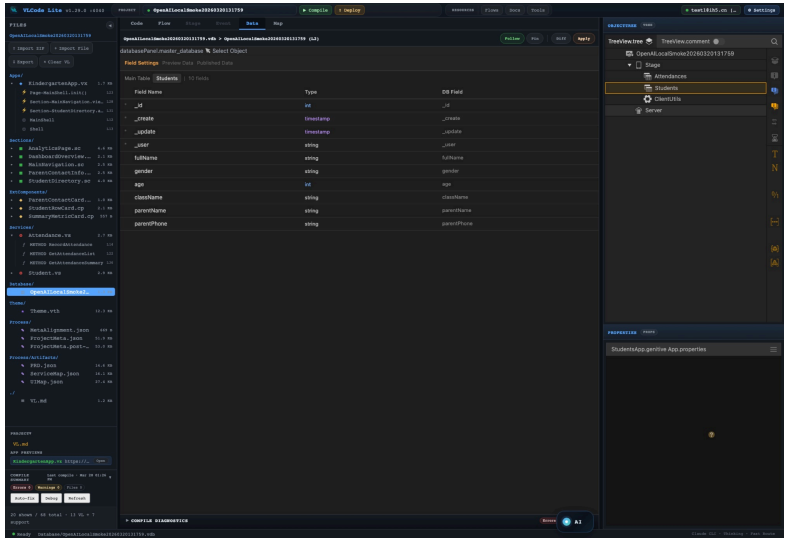
PITFALL

Writing the requirement as a marketing one-liner ("make a nice todo tool") lets the AI improvise and drift off target. Spell out field names and interaction verbs so generation and later verification have concrete anchors to match against.

CHECKPOINT

Every field, page, and interaction in your prompt can be located one-for-one in the generated files

Case 1: Data Design



Data panel for tables and schema

CORE IDEA

Data design defines the tables and fields behind the app. In VLCode, data lives in .vdb files and is shown visually in the Data panel (tables/schema). For the Todo app, data design means pinning down the todos table's fields, types, and constraints — the foundation that pages and service methods reference.

OPERATIONS

1. Switch to the Data panel to view the generated todos table and field types
2. Verify the field names match the requirement prompt (title, done, created)
3. Check the .vdb file starts with VL_VERSION on line one and uses consecutive hyphens for indentation
4. Confirm a primary key and a time field exist for list ordering and deletion references
5. If fields need adjusting, note the issue for a later incremental data-adjust

EXAMPLE / EXERCISE

An ideal todos schema (visible in the Data panel):

```
todos
  id        primary key
  title     text
  done      bool, default false
  created   time, default now
```

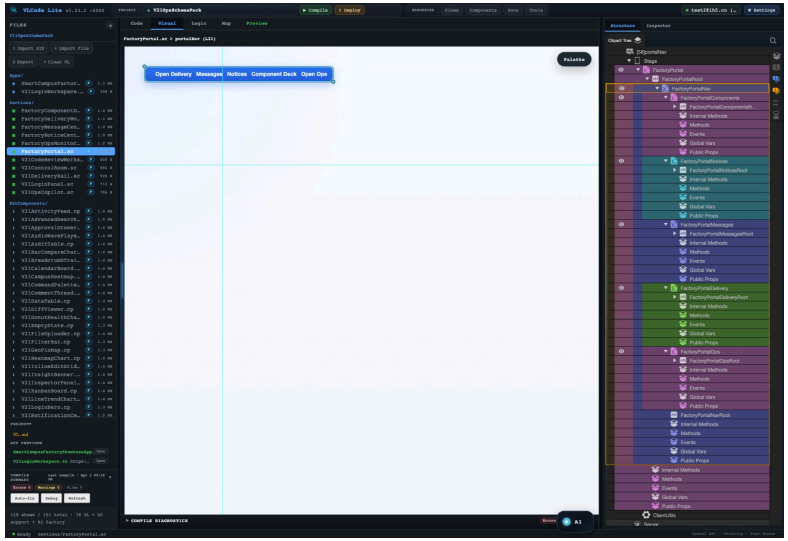
The .vdb source must start with `// VL_VERSION:4.3.1` on line one, with nothing before it.

PITFALL

Seeing fields in the Data panel and assuming all is well, while missing that the .vdb first line lacks VL_VERSION or mixes spaces for indentation — this triggers hard-to-locate "strange" errors. Always check the source file's first line and indentation.

CHECKPOINT

Case 1: Pages And Interactions



Visual panel with object tree and properties

CORE IDEA

Pages and interactions present data to users and accept actions. In VLCode, pages/regions are .sc (Section) files; you observe layout and the object tree on the Visual canvas, and click behavior in the Logic event panel. The Todo app's core interactions are three event chains: add, check-done, and delete.

OPERATIONS

1. Switch to Visual mode and confirm the TodoList page's input, list, checkbox, and delete button all exist in the canvas and object tree
2. In the properties panel, verify each object's bound field points to the todos table
3. Switch to the Logic event panel and inspect the add, check, and delete event chains one by one
4. Confirm checking done writes back the done field and delete removes the matching row
5. Use Visual's selection sync to compare object tree and canvas and locate any missing interaction

EXAMPLE / EXERCISE

In the Logic panel you expect three interactions bound:

```
Add button  → write input text to todos.title, done=false
Checkbox    → toggle todos.done for that row
Delete btn  → delete that row
```

An empty event chain means that interaction was not generated and must be added.

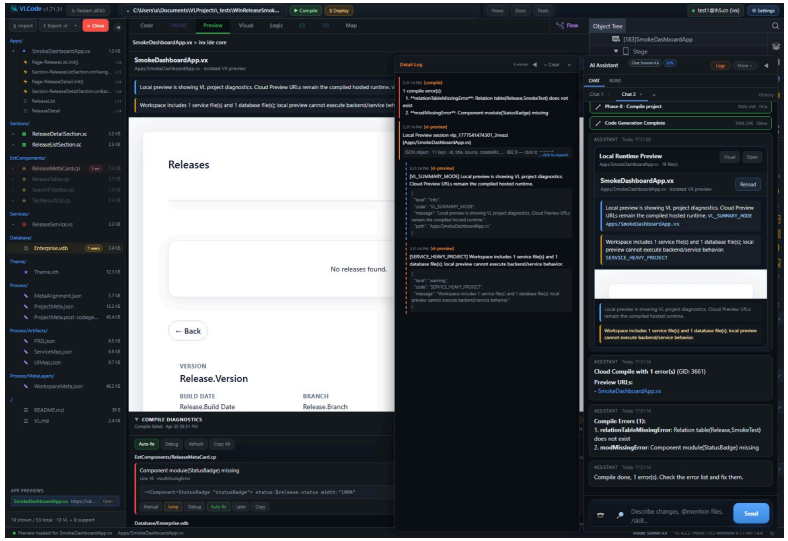
PITFALL

Seeing the buttons in Visual and assuming interactions are complete, without confirming the event chains in Logic — the buttons appear but do nothing when clicked. Always verify each action is actually bound in the Logic panel.

CHECKPOINT

All three controls are present in Visual, and the Logic panel shows real bound logic for the add, check, and delete event chains.

Case 1: Validation And Fixes



Compile preview inside the IDE

CORE IDEA

Validation and fixes are the last mile that turns generated output into a deliverable app. VLCode's validation chain is fixed: Lint (low cost, first) → Compile (heavier platform validation) → Preview. Do not use Compile as the first trial loop. When something breaks, prefer the minimal-fix workflow (compile-fix) over broad rewrites.

OPERATIONS

1. Run Lint first for project-level VL static checking (lintPjt) and read the report in Detail Log
2. After Lint passes, run Compile (parsevl) for the heavier platform validation
3. Locate the first real error in Detail Log and return to the source file plus ProjectMeta to find the root cause
4. If the same error repeats twice, stop trial-and-erroring and analyze the source instead of recompiling
5. Apply a minimal fix with the compile-fix workflow, then rerun Lint → Compile → Preview to verify

EXAMPLE / EXERCISE

A typical error and minimal fix:

```
Error: Section TodoList references undefined field todos.completed
Fix:   the field is actually 'done' — change the .sc binding to todos.done (one edit, do not rewrite the page)
```

Then rerun the validation chain once to confirm it passes.

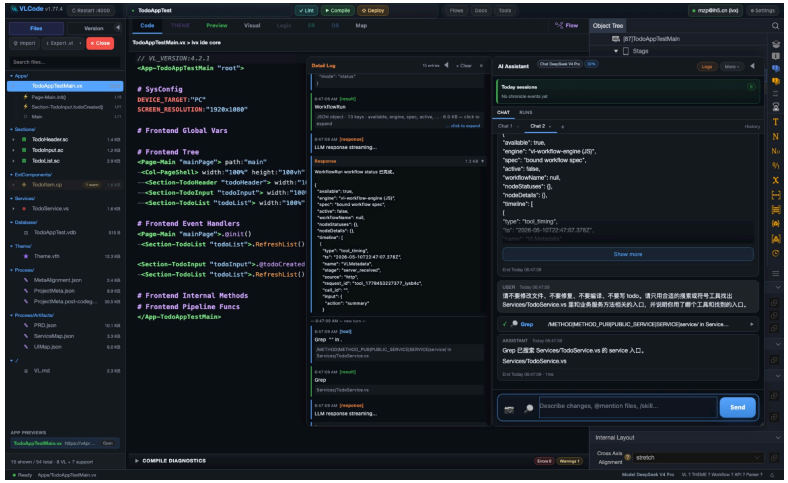
PITFALL

Letting the AI rewrite the whole file at the first error introduces new problems and deepens the loop. Locate the root cause and make a single-point minimal fix; if the same error appears twice, you must stop and return to the source and ProjectMeta.

CHECKPOINT

TodoApp passes the full Lint → Compile → Preview chain, and every fix was a minimal root-cause change rather than a whole-file rewrite.

Case 2: Campus Operations Dashboard



Final IDE state after project creation

CORE IDEA

Case 2 steps up to a mid-size system: a campus operations dashboard with multiple tables, several pages, and a number of backend service methods. It is more complex than the Todo app and suits the layered workflow meta-layered-codegen (use meta-sharded-codegen to shard when larger), so frontend, backend, and theme are generated decoupled and stay maintainable. (The legacy 6-file/9-file workflows are archived and no longer selectable — use the meta-* family instead.)

OPERATIONS

1. Create a PascalCase workspace CampusOps under ~/Documents/VLProjects/
2. In AI Chat, write the dashboard requirement as a structured goal — entities, pages, service methods — and name meta-layered-codegen
3. Choose the layered workflow meta-layered-codegen (meta-sharded-codegen when larger) for decoupled generation
4. During the run, track node progress and file writes in Detail Log
5. After generation, use the Map tab for a structural overview before verifying tab by tab (Visual / Logic / ER / DB / Data)

EXAMPLE / EXERCISE

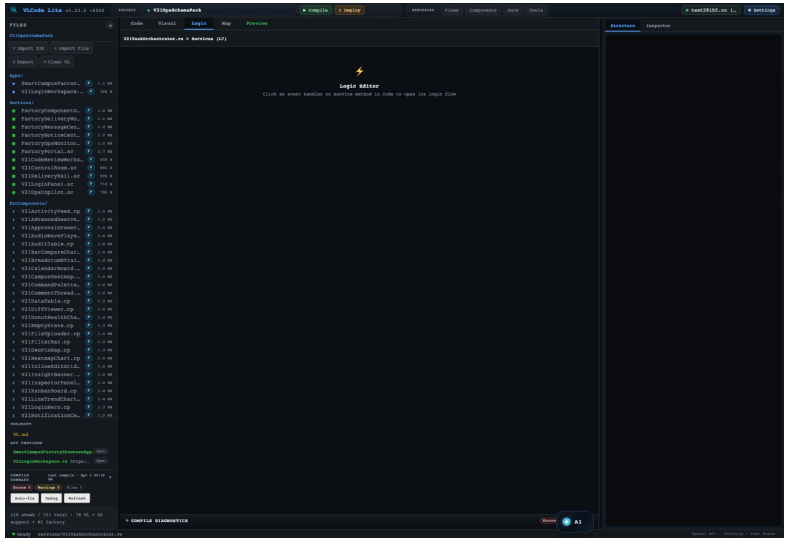
In AI Chat, write the dashboard goal as an executable requirement and name the workflow:

```
Use meta-layered-codegen to generate campus ops dashboard CampusOps:
entities buildings, incidents, staff;
pages Overview (KPI cards), Incident list, Staff schedule;
services count today's incidents, summarize by building.
```

When done, ~/Documents/VLProjects/CampusOps/ should contain:

```
Apps/      CampusOpsApp.vx · CampusOpsBackendMainApp.vx
Sections/  Overview.sc · IncidentList.sc · StaffSchedule.sc
Services/  IncidentService.vs · StaffService.vs
Database/  CampusOps.vdb            # buildings / incidents / staff tables
Theme/     ThemeDefault.vth
```

Case 2: Service Planning



Service logic panel for backend behavior

CORE IDEA

Service planning designs the app's backend logic: which service methods exist, their inputs/outputs, and responsibilities. In VLCode, backend logic lives in .vs (Service) files — you view and edit the .vs source in Code mode, while the data/service relationships are inspected in the ER and DB tabs (VLCode has no separate Service tab). The dashboard's statistics and summary logic should converge into clear service methods that pages call.

OPERATIONS

1. List service methods in the requirement up front: name + inputs + return
2. After generation, open Services/*.vs in Code mode to view the method implementations, and cross-check which tables they read/write in the ER/DB tabs
3. Verify each service method has a single responsibility and a clear name (English PascalCase file names)
4. Confirm page references to service methods match the names defined in the .vs files
5. If a service method is missing, note it and add it incrementally with add-service

EXAMPLE / EXERCISE

Plan the dashboard's service methods as contracts:

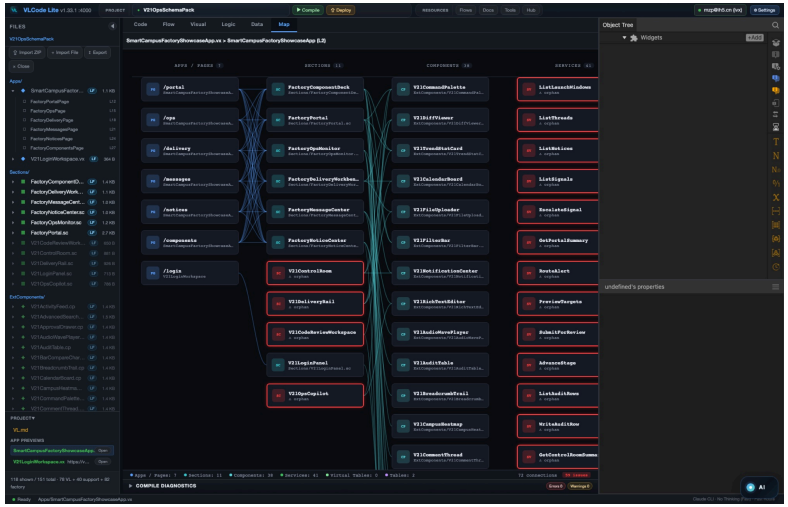
```
CountTodayIncidents()        → today's total incident count
SummarizeByBuilding()        → [{building, count}]
ListStaffOnDuty(date)        → on-duty staff list
```

Each method does one thing with a clear name, and pages reference it by name.

PITFALL

Cramming several responsibilities into one catch-all service method makes page references messy and changes painful later. During planning, give each service method a single job and a self-explanatory name.

Case 2: Map And Dependencies



Map mode for application structure

CORE IDEA

The Map project view renders the whole project's structure — apps, pages, services, data, theme, and their dependencies — as a readable graph. For a multi-module system like the campus dashboard, the Map panel is the best vantage point to check completeness and dependency coherence; behind it sits the machine-readable ProjectMeta.

OPERATIONS

1. Switch to the Map panel for an overview of CampusOps's app, Section, Service, Database, and Theme nodes
2. Follow the edges to check that page-to-service and service-to-table dependencies are coherent
3. Spot isolated nodes (tables/methods not referenced by any page or service)
4. Spot broken references (a page referencing a non-existent service method or field)
5. Cross-check the dependency issues against ProjectMeta to locate the source files to fix

EXAMPLE / EXERCISE

On the Map you should be able to read the dependency chain:

```
Overview page → CountTodayIncidents() → incidents table
Incident list → SummarizeByBuilding() → incidents / buildings tables
```

A break in any chain means a broken reference to fix in the source.

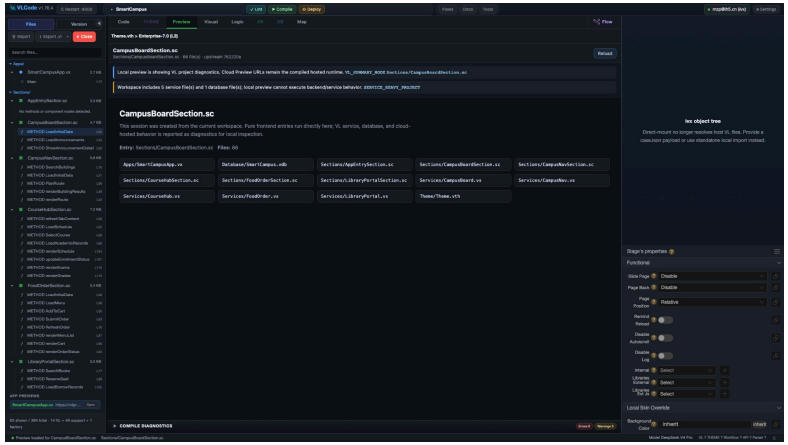
PITFALL

Looking only at individual panels instead of the global Map view misses cross-module broken references (e.g. a page calling a renamed service method). For mid-size systems, always do a dependency sweep in Map.

CHECKPOINT

You can point to each page's dependent service methods and tables on the Map panel and confirm there are no isolated nodes or broken references.

Case 2: Compile Error Handling



A logic-panel failure context for debugging practice

CORE IDEA

Compile error handling means locating and rooting out real errors in a multi-module system, not papering over them. VLCode's root-cause-first principle requires you to reproduce the error, capture the real message, inspect the call path and ProjectMeta, then fix the root cause. Compile uses parsevl for platform-grade validation, safer than fast local mode.

OPERATIONS

1. After running Compile, find the first real error and its file location in Detail Log
2. Return to the matching .sc/.vs/.vdb source and ProjectMeta to understand the root cause
3. Decide whether it's field drift, a broken service-method reference, or an indentation/version-header issue
4. Make a minimal root-cause change — never write error-swallowing or silent-fallback compatibility code
5. If the same error appears twice, stop recompiling and re-analyze from the source

EXAMPLE / EXERCISE

A real compile error and root-cause fix:

```
Error: Service references incidents.building_id, but the Data field is buildingId
Root: field-name drift (snake_case vs camelCase)
Fix:  standardize on buildingId (one edit at the .vs reference to match the .vdb definition)
```

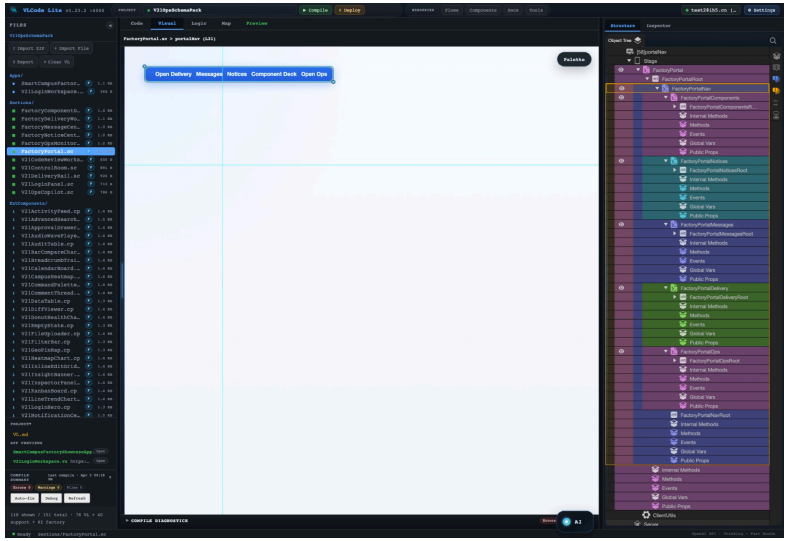
Then rerun the validation chain once to confirm.

PITFALL

Adding default values, swallowing errors with try-catch, or bypassing validation just to make Compile "look" green hides the primary error and degrades quality. Every error must surface and be fixed at the root — no fallback.

CHECKPOINT

Case 3: Customer CRM



Visual panel with object tree and properties

CORE IDEA

Case 3 is a business-facing Customer CRM: entities like customers, contacts, and deals, plus list, detail, and follow-up pages. Its value is practicing incremental evolution on an existing app — generate a base version with the right workflow, then add pages, add services, and align the theme step by step via incremental workflows.

OPERATIONS

1. Create a workspace CustomerCRM under ~/Documents/VLProjects/
2. In AI Chat, write the CRM's core entities and pages and choose a workflow to generate the base version
3. After generation, use the Visual panel to check the customer list and detail page layout and object tree
4. Use the Data panel to confirm the customers / contacts / deals schemas
5. Once the base opens in Preview, move on to the later incremental changes

EXAMPLE / EXERCISE

Base CRM requirement example:

```
Generate a customer CRM: entities customers(name, industry, owner), contacts(name, customerId, phone), deals(title, customerId, amount, stage). Pages: Customer list, Customer detail (with contacts and deals).
```

Once the base runs, later Case 3 topics add a page, add a service, and align the theme on top of it.

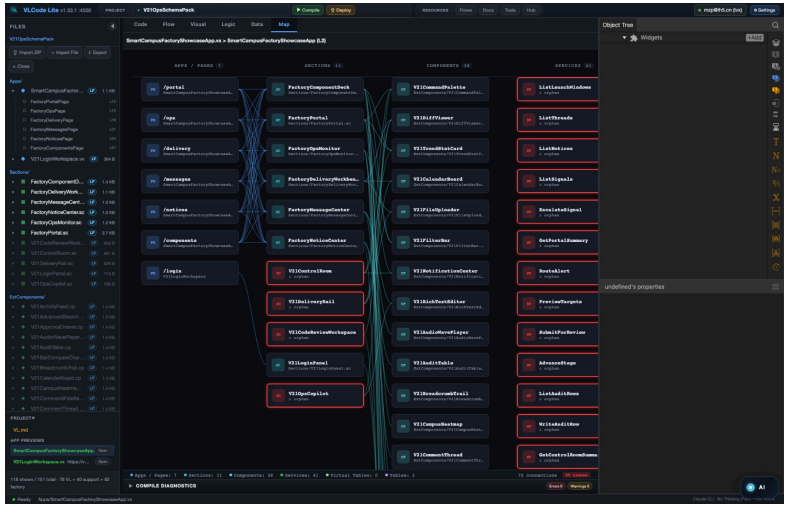
PITFALL

Stuffing every page and feature into the first generation makes verification and maintenance harder. Generate a runnable base first, then evolve in steps with incremental workflows, verifying at each step.

CHECKPOINT

The CustomerCRM base is generated, the customer list and detail pages open in Preview, and the three core tables are present in the Data panel.

Case 3: Add A Page



Map mode for application structure

CORE IDEA

Adding a page means using the add-page incremental workflow to add one page to the existing CRM without rewriting the whole project. The incremental workflow touches only the files it needs, honoring VLCode's small-scope, high-cohesion principle. For the CRM, a typical addition is a deals board or a customer follow-up page.

OPERATIONS

1. In AI Chat, describe the new page and its data source in natural language
2. Explicitly select the add-page incremental workflow rather than a full regeneration
3. After the run, confirm in Detail Log that only files related to the new page were added or changed
4. Use the Visual and Map panels to verify the new page is wired into routing and references the correct tables
5. Run Lint → Compile → Preview to validate the new page and confirm existing pages are unaffected

EXAMPLE / EXERCISE

Add a deals board to the CRM with add-page:

```
Add a deals board page DealsBoard to CustomerCRM: show deals in columns by stage, each card shows title and amount, click through to customer detail. Use the add-page workflow and don't touch other pages.
```

Detail Log should show only DealsBoard-related file writes.

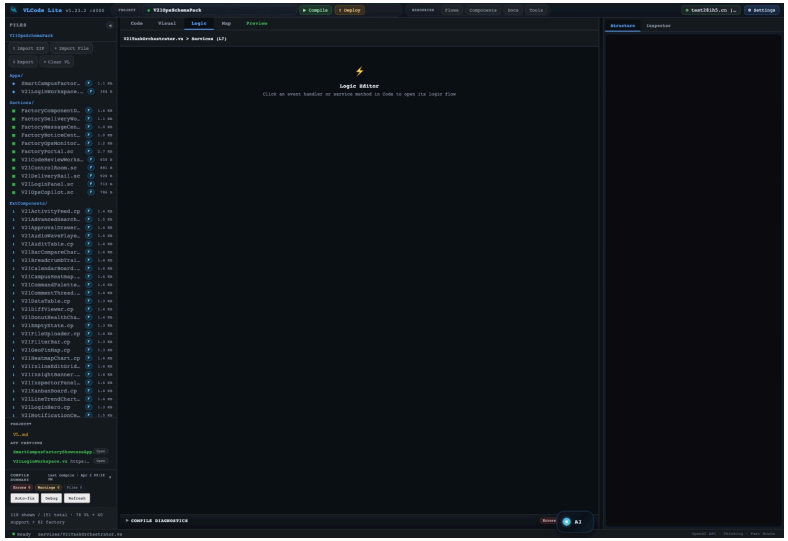
PITFALL

Triggering a full regeneration just to add one page wipes out other already-verified pages. Always choose the add-page incremental workflow and confirm in Detail Log that the change scope is limited to the new page.

CHECKPOINT

The new DealsBoard page appears in the Map routing and opens in Preview, while the original customer list/detail pages remain fully intact.

Case 3: Add A Service



Service logic panel for backend behavior

CORE IDEA

Adding a service means using the add-service incremental workflow to add one backend service method to the CRM, such as summing deal amounts by stage. Service logic goes into .vs files — view the source in Code mode and inspect which tables it touches in the ER/DB tabs. Adding a service incrementally avoids touching the frontend and other services, keeping coupling low.

OPERATIONS

1. In AI Chat, describe the new service method's name, inputs, and return, and name add-service
2. Select and run the add-service incremental workflow
3. Open Services/*.vs in Code mode to view the newly generated method implementation
4. In the Map tab, confirm the new service is referenced by the right page, and in the ER/DB tabs confirm it is wired to the right table
5. Run the validation chain to confirm the new method compiles and existing service methods are untouched

EXAMPLE / EXERCISE

Add an amount summary to the CRM with add-service:

```
Use add-service to add service method SumDealsByStage() to CustomerCRM:
return [{stage, totalAmount}] for the deals board header. Don't touch other services.
```

Code mode should show SumDealsByStage's implementation, and on the Map it wires to the deals table; Detail Log should show only service-related file writes.

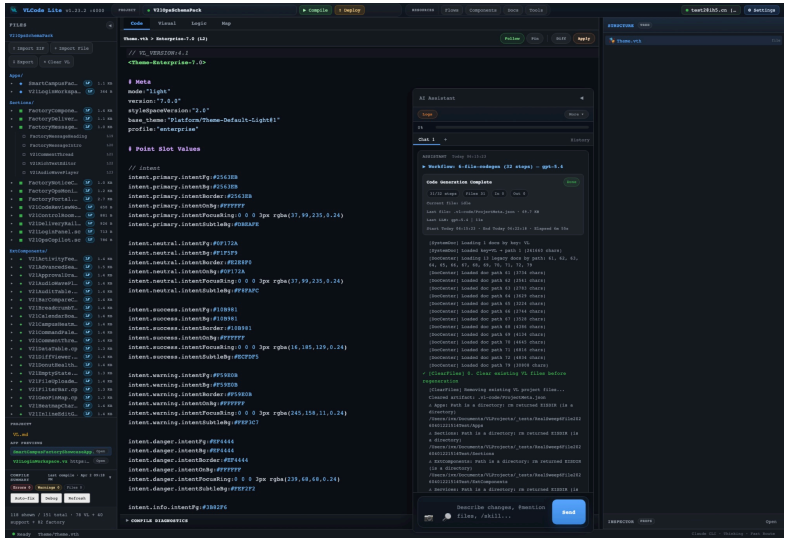
PITFALL

Naming the new service method to collide with an existing method or field, or not aligning it to the table fields, causing broken references. Before adding, check naming and dependencies in the Map tab and the ER/DB tabs to avoid drift.

CHECKPOINT

The new service method has an implementation in Code mode, wires to the correct table on the Map, is successfully referenced by the target page, and compiles

Case 3: Theme Alignment



Theme panel for enterprise visual tokens

CORE IDEA

Theme alignment makes the CRM's pages share one set of visual tokens, avoiding a style split between new and old pages. In VLCode, the theme is a .vth file edited in the THEME token workbench; frontend, backend, and theme are decoupled, so changing the theme shouldn't break page logic. Use the theme-customize incremental workflow to adjust.

OPERATIONS

1. Switch to the THEME panel to view the current .vth color, spacing, and font tokens
2. In AI Chat, use the theme-customize workflow to describe the visual rules to unify
3. Confirm new pages (like DealsBoard) reference the same theme tokens rather than hard-coded styles
4. Compare pages visually in Visual/Preview for consistency
5. Run the validation chain to confirm the theme change didn't affect any page logic or service

EXAMPLE / EXERCISE

Unify the CRM theme with theme-customize:

```
Standardize CustomerCRM's primary color to deep blue, card radius to 8, and table row height; make the deals board and customer list share the same THEME tokens. Use the theme-customize workflow.
```

After the change, all pages should look consistent in Preview.

PITFALL

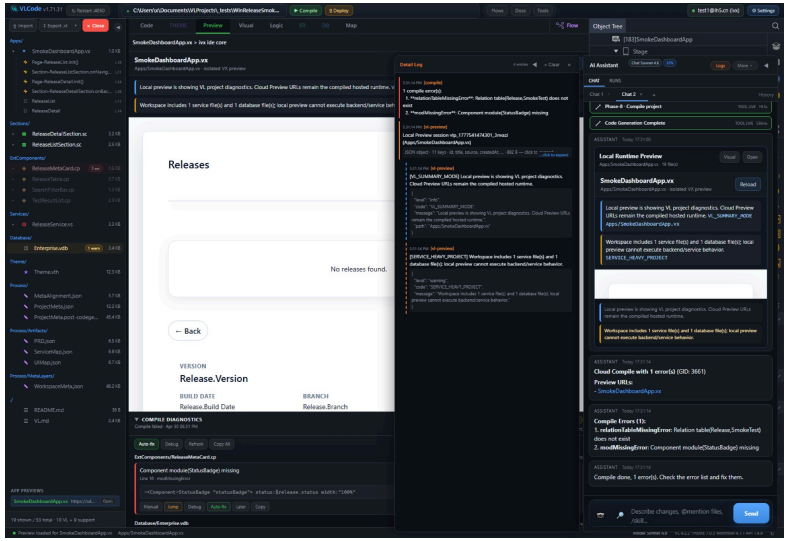
Hard-coding colors and spacing on a single page, bypassing .vth tokens, leaves new pages inconsistent and hard to unify later. Route all visuals through THEME tokens and adjust centrally with theme-customize.

CHECKPOINT

Every CRM page looks consistent in Preview and references the same .vth theme tokens, with no page-level hard-coded styling.

PART IX: COMPLETE CASE STUDIES

Case 4: Design-To-App



Compile preview inside the IDE

CORE IDEA

Case 4 practices generating a real app from a design: use smart-design-fullstack-codegen (HTML-first design → code) or html-first-vl-codegen to turn a UI design into runnable VL frontend and backend. The key is to start from a faithful visual, then have the generated pages, data, and services all align to that design.

OPERATIONS

1. Create a workspace SalesDash under ~/Documents/VLProjects/ and place the design assets (screenshot or HTML) in it
2. In AI Chat, choose and name the smart-design-fullstack-codegen (design → full stack) or html-first-vl-codegen workflow
3. In the prompt, describe region by region: what each visual area does and which table it reads/writes
4. During the run, track the 'read design → decompose regions → generate Section/Service/Database' nodes in Detail Log; at the human gate, check fidelity on the review-card
5. Run the validation chain lintPjt → parsevl and check in Preview at /route/sales-dash that visuals and interactions stay faithful to the design

EXAMPLE / EXERCISE

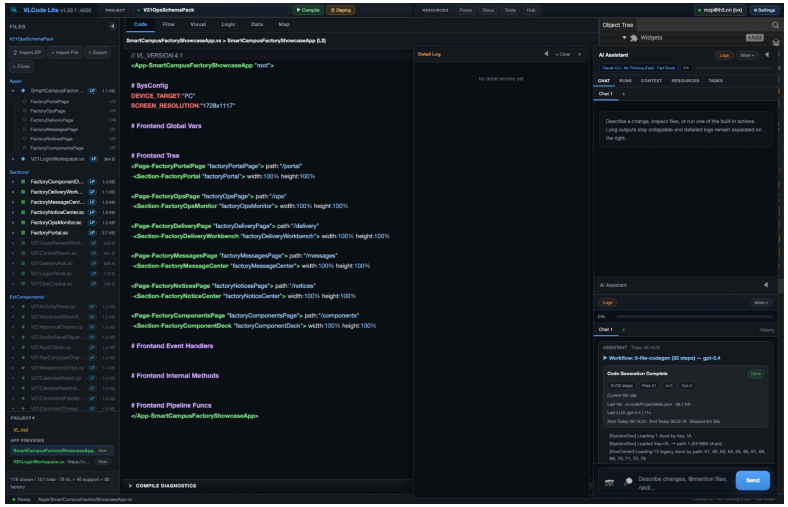
Full Design-To-App walkthrough (copy-pasteable, step by step):

```
1. Workspace: ~/Documents/VLProjects/SalesDash/, drop the dashboard screenshot/HTML in
2. In AI Chat, type:
   Use smart-design-fullstack-codegen to generate app SalesDash from my uploaded dashboard design:
   top KPI card row (today's orders, sales, refund rate);
   middle trend chart area (last-30-days sales line chart);
   bottom orders table (filter by status), data from an orders table.
3. During generation: Detail Log shows region-by-region parsing; at the review-card, compare to the design and approve / request_changes
```

When done, ~/Documents/VLProjects/SalesDash/ should contain:

```
Apps/      SalesDashApp.vx · SalesDashBackendMainApp.vx
Sections/  SalesDashboard.sc          # KPI cards + trend chart + orders table
```

Case 4: Upload Design Assets



AI Chat with detailed run context

CORE IDEA

Uploading design assets provides screenshots or HTML to the design-driven workflow as a visual baseline. VLCode's AI Chat is the primary interaction entry, where design assets come in with the conversation, and Detail Log records how the workflow reads and decomposes them. The more faithful the assets, the closer the generated pages match the design.

OPERATIONS

1. In AI Chat, provide the design screenshot/HTML as this round's input to the html-first/smart-design workflow
2. In the prompt, state which function and data each visual region maps to
3. After the run, watch Detail Log for the nodes reading the design and decomposing regions
4. Watch Chat for clarifications: region-mapping questions arrive as an ask_user chat event, while gates needing a decision arrive as a review-card — answer accordingly
5. Compare the generated Sections to the original design regions for one-to-one correspondence

EXAMPLE / EXERCISE

Hand the assets and intent to the workflow together:

```
Here is the order management page design HTML; generate the page from it: the top filter bar maps to orders status filtering, the middle table to the orders list, the right drawer to order detail. Use html-first-vl-codegen.
```

Detail Log shows the workflow parsing the design region by region.

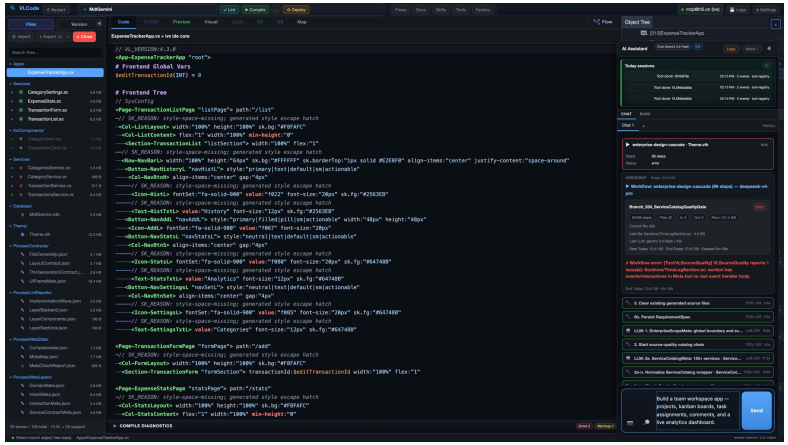
PITFALL

Uploading the design but not explaining each region's functional mapping forces the workflow to guess interactions and data. Always spell out the region-to-data/interaction mapping in the prompt.

CHECKPOINT

Detail Log shows the workflow successfully read the design assets, and the generated Sections correspond one-to-one with the design regions

Case 4: Human Review



Detail Log with node progress and tool output

CORE IDEA

Human review puts a person in charge at the key node of turning a design into an app, ensuring the output is faithful to the design rather than merely "close enough." When the workflow reaches a human gate (human_gate is a chat event / SSE), VLCode surfaces a blocking review-card (approve / reject / request_changes), letting you make explicit decisions on intermediate output in Chat, with the record kept in Detail Log. The review-card gates your input until you decide; the live cards beside it (llm-live-card / node-result-card, etc.) are just a passive feed.

OPERATIONS

1. When the workflow reaches a human gate (a human_gate event fires), watch for the review-card in Chat (approve/reject/request_changes)
2. Compare Visual/Preview against the original design, checking fidelity region by region
3. If there's a discrepancy, choose request_changes on the review-card and state which region to fix
4. Approve to let the workflow continue; reject to stop this round and redo per feedback
5. Review each human decision and its subsequent node action in Detail Log

EXAMPLE / EXERCISE

Give explicit feedback on the review-card:

```
request_changes: KPI card count is right, but the trend area was generated as a table — it should be a line-chart region; reorder the orders table columns to match the design.
```

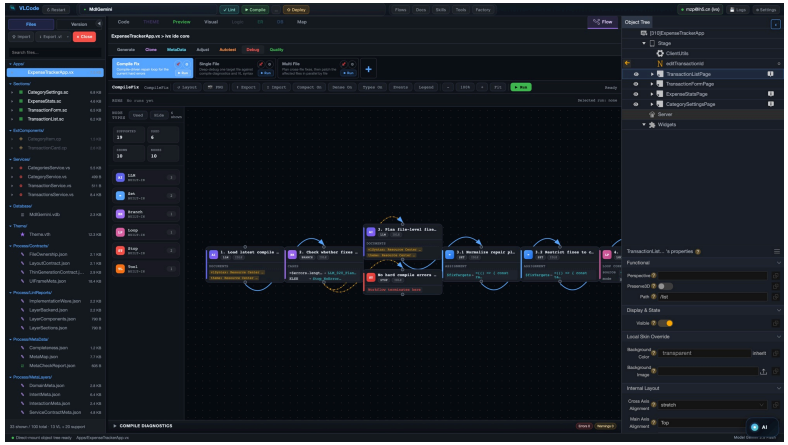
After request_changes the workflow corrects accordingly and returns to the review-card for your approval.

PITFALL

Approving everything without real comparison abandons human gating, letting design-divergent output slip through. At every review-card, verify against the design region by region — prefer request_changes over a blind approve.

CHECKPOINT

Case 5: Workflow Debug Drill



Debugging and rerun context inside the DAG

CORE IDEA

Case 5 is a dedicated workflow debug drill: when a generation or repair workflow errors midway, how to use pause, resume, abort, and Detail Log to pin down the problem. VLCode's run states settle reliably to done/error/aborted, and abort scope has two levels — chat (stops this conversation and its runs) and workflow (stops a single run).

OPERATIONS

1. During a workflow run, track node progress in Detail Log and spot the erroring node
2. Pause to halt the run at the failure so you can inspect context instead of immediately rerunning
3. Read that node's real error in Detail Log (tool I/O, validation, error stack)
4. Decide whether to stop the chat (and all its runs) or abort only this one workflow run
5. Return to the matching source file and ProjectMeta to find the root cause, ready to rerun from the node

EXAMPLE / EXERCISE

Reading a failed node in Detail Log:

```
Node compile validation failed: Section CustomerDetail references non-existent service method GetCustomerDeals()
Root: the service method is actually named ListDealsByCustomer()
Action: abort the current workflow run → fix the .sc reference name → rerun from that node
```

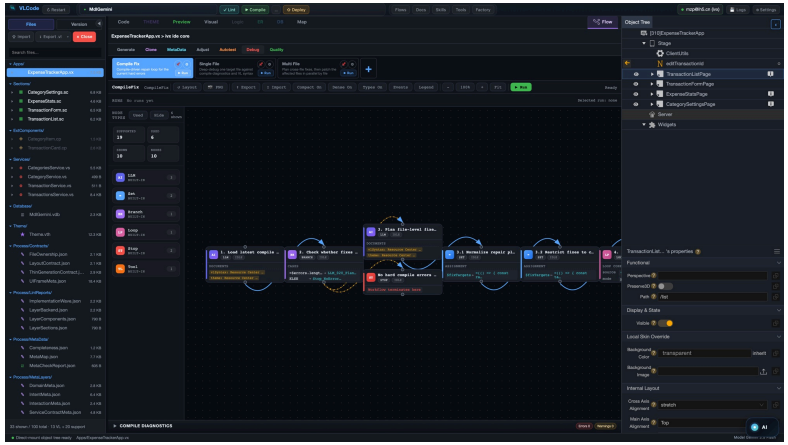
PITFALL

Aborting the whole chat and starting over at the first error wastes already-completed nodes. First use pause and Detail Log to see which node and what root cause, then decide whether the abort scope is chat or a single workflow.

CHECKPOINT

You can pinpoint the failed node and its real error in Detail Log and correctly choose between chat-level and workflow-level abort.

Case 5: Rerun From Node



Debugging and rerun context inside the DAG

CORE IDEA

Rerun from node means, after locating and fixing the root cause, continuing execution from the failed step rather than restarting — reusing the already-successful upstream nodes and saving tokens and time. This matches VLCode's philosophy of avoiding broad rewrites and banking compute through reuse: fix one spot, resume one stretch, rather than regenerating the whole run.

OPERATIONS

1. Confirm the failed node's root cause is fixed in the source file/ProjectMeta
2. In Chat/Detail Log, choose to rerun from that failed step (rerun from step)
3. Watch the upstream successful nodes be reused rather than re-executed
4. Track the new run from that node in Detail Log until it reaches done
5. After the rerun, run the validation chain to confirm the overall result is correct

EXAMPLE / EXERCISE

Resume from the failed node after fixing the root cause:

```
The compile node errored last run due to the GetCustomerDeals() reference.
CustomerDetail.sc now references ListDealsByCustomer().
Now rerun from the compile node; do not redo the earlier successful generation nodes.
```

Detail Log should show upstream nodes skipped and execution continuing from compile.

PITFALL

Rushing to rerun from a node before truly fixing the root cause makes the same node fail again, wasting a loop. Confirm the root cause is fixed first; if the same node fails twice, stop and re-analyze the source.

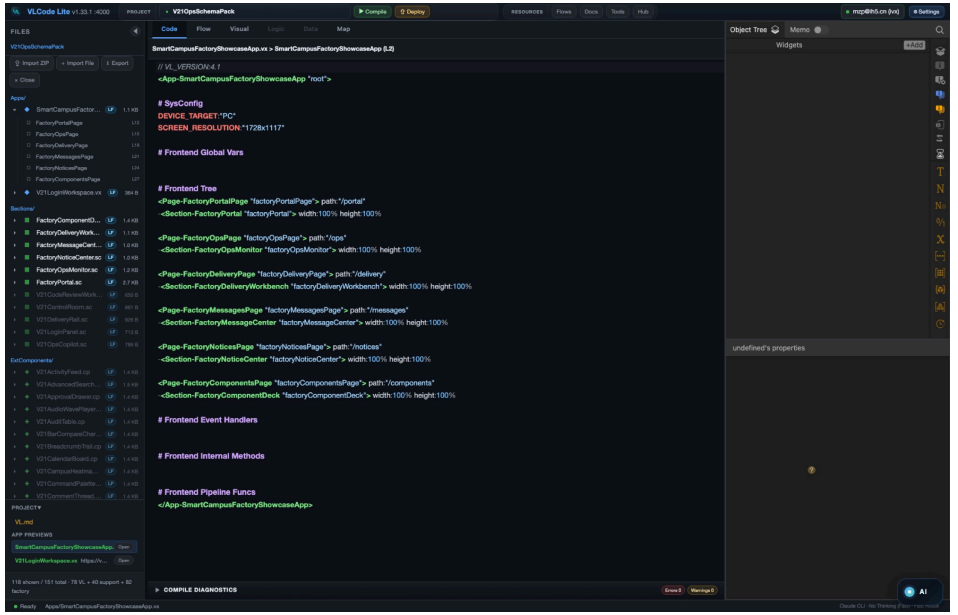
CHECKPOINT

You successfully rerun from the failed node to done, with upstream successful nodes reused rather than redone, and the overall validation chain passes.

CHAPTER

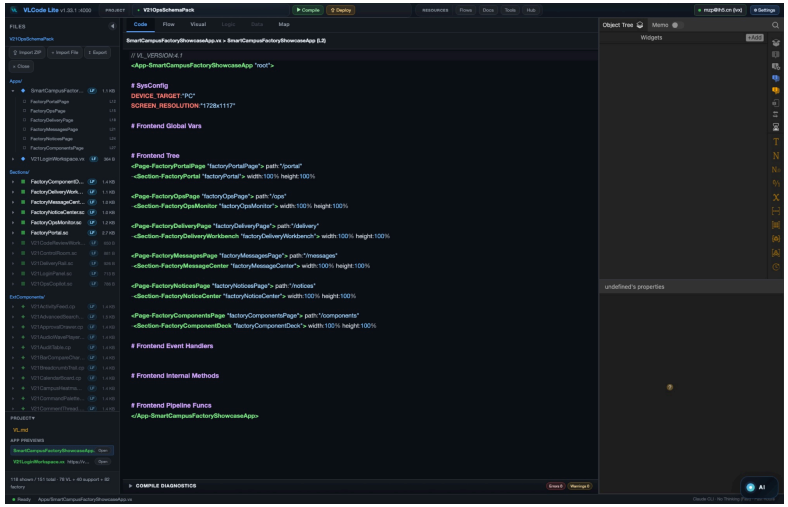
Part X: Appendices And Quick Reference

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



Code mode for inspecting and editing VL source

Common Commands



Code mode for inspecting and editing VL source

CORE IDEA

Daily VLCode work runs on a small command set: npm scripts to launch locally, and the platform-level lintPjt and parsevl for validation. Remember the split — npm scripts make it run, platform commands check whether it is correct — so you never make Compile your first trial loop.

OPERATIONS

1. Run npm run dev to start the browser Web IDE, then open http://localhost:4000 (port range 4000-4010).
2. When you need the desktop shell from source, run npm run electron.
3. After editing code, run lintPjt first for a low-cost project-level VL static check.
4. Once Lint passes, run parsevl for the heavier platform compile/parse validation.
5. Run npm run test when you want to exercise the test suite.

EXAMPLE / EXERCISE

A typical local startup:

```
npm run dev      # Web IDE on http://localhost:4000
npm run electron # or launch the desktop shell from source
npm run test     # run tests
```

The platform validation chain is ordered: lintPjt first, then parsevl, then Preview at /route/<path>.

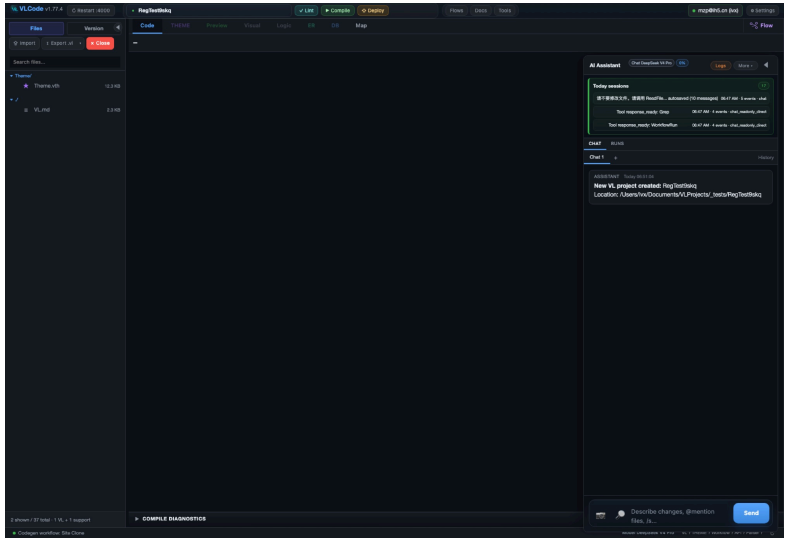
PITFALL

Treating parsevl (Compile) as the first trial loop. Recompiling repeatedly is slow and burns tokens; the correct order starts with the cheap lintPjt check and saves Compile for later.

CHECKPOINT

You can state, without looking it up, which port npm run dev opens and the order and purpose of lintPjt versus parsevl.

Common Workflows



Workflow selector before execution

CORE IDEA

Generation and modification in VLCode are driven by named workflows in three families: generation (build from zero), incremental (touch an existing project), and repair (rescue after errors). Picking the right workflow chooses the right method up front and lets the DAG orchestrate the rest.

OPERATIONS

1. Generate small projects from scratch with meta-direct-codegen; use meta-thin-fast-codegen / meta-stream-fast-codegen for a fast stable thin contract; use meta-layered-codegen for layered builds, meta-sharded-codegen when larger.
2. Use smart-design-fullstack-codegen / html-first-vl-codegen when you have a design; use enterprise-grand-cascade-codegen for large systems (100+ service methods / 30 pages); use factory-turbo-codegen to reuse components for speed.
3. Make incremental changes with add-page, add-service, feature-adjust, data-adjust, batch-adjust, or theme-customize.
4. Repair with compile-fix, complete-files (recover files dropped mid-run), debug-single-file / debug-multi-file, or heal-from-meta to self-heal against ProjectMeta.
5. Name the workflow in AI Chat with natural language and watch run progress in Detail Log. (The legacy 3-file/6-file/9-file workflows are archived and no longer selectable — use the meta-* family instead.)

EXAMPLE / EXERCISE

Name a workflow in AI Chat:

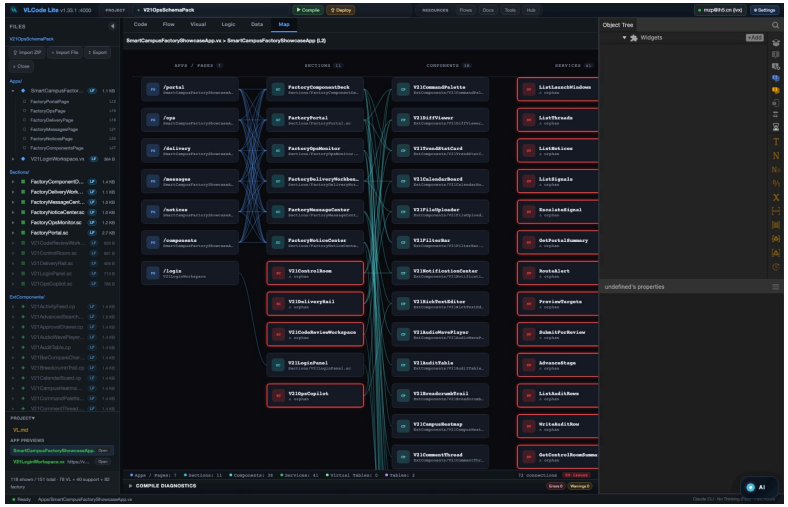
```
Use meta-direct-codegen to generate a TeamPM task board
```

Add a page after the first version lands:

```
Use add-page to add a "My Tasks" page filtered by the logged-in user
```

Fix a compile error:

Common Directories



Map mode for application structure

CORE IDEA

VL projects follow a fixed directory convention rooted at ~/Documents/VLProjects/<PascalCaseName>/, with subfolders split by file type. A consistent layout lets workflows write files to the right place and lets ProjectMeta map the project correctly.

OPERATIONS

1. Keep all projects under ~/Documents/VLProjects/, one PascalCase-named folder per project.
2. Config/ holds project.state.json (gid binding lives here), project.settings.json, and secrets.json; Apps/ holds .vx app entries (including the backend <ProjectName>BackendMainApp).
3. Sections/ holds .sc pages/regions; Services/ holds .vs service logic.
4. Database/ holds .vdb tables and schema; Theme/ holds one .vth theme (default ThemeDefault_7.0.3.vth, StyleSpace 2.0); ExtComponents/ holds project-local .cp (pure UI components) and .wc (project-local native web extensions); Process/ holds process scripts.
5. Browse this structure in the left file tree and jump to the matching subfolder to find any file type.

EXAMPLE / EXERCISE

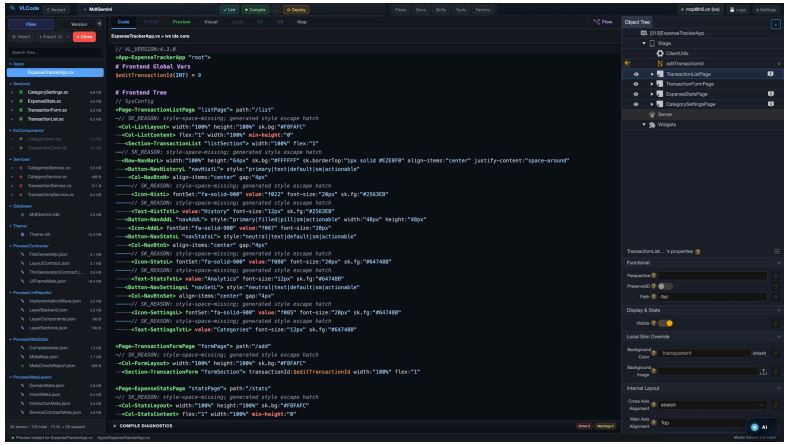
Standard project structure (full subfolder set):

```
~/Documents/VLProjects/TeamPM/
├── Apps/          # TeamPMApp.vx · TeamPMBackendMainApp.vx
├── Sections/      # TaskBoard.sc (page)
├── Services/      # TaskService.vs
├── Database/      # Tasks.vdb
├── Theme/         # ThemeDefault.vth (one .vth)
├── ExtComponents/ # TaskTile.cp (pure UI component) · TaskGantt.wc (native web extension)
├── Process/       # process scripts
├── Config/        # project.state.json (gid binding) · project.settings.json · secrets.json
└── .vl-code/      # ProjectMeta.json (project map)
```

PITFALL

Naming projects or files with Chinese / non-ASCII characters, or scattering projects into Downloads or source trees. File names must be English PascalCase, and the project root must live under ~/Documents/VLProjects/.

AI Prompt Templates



AI Chat used as the command and decision surface

CORE IDEA

AI Chat is VLCode's primary interaction entry and its workflow control console. A good prompt turns a requirement into an executable goal: name the workflow, spell out the entities and pages, and state the acceptance bar — so the AI guesses less and reworks less.

OPERATIONS

1. Generation prompts: name the workflow + project name + core entities + key pages, e.g. "Use meta-direct-codegen to generate X".
2. Incremental prompts: name add-page / add-service / feature-adjust and say which Section/Service it lands in.
3. Repair prompts: name compile-fix / debug-single-file and paste the real error from Detail Log.
4. Issue control commands in natural language: pause, resume, rerun from a step.
5. Final summaries and decisions you must approve appear in AI Chat; the full run ledger lives in Detail Log.

EXAMPLE / EXERCISE

Generation template:

```
Use meta-thin-fast-codegen to generate a CustomerCRM:
entities Customer/Contact/Deal; pages Customer List, Customer Detail, Deal Board;
each page needs full CRUD, use a blue theme.
```

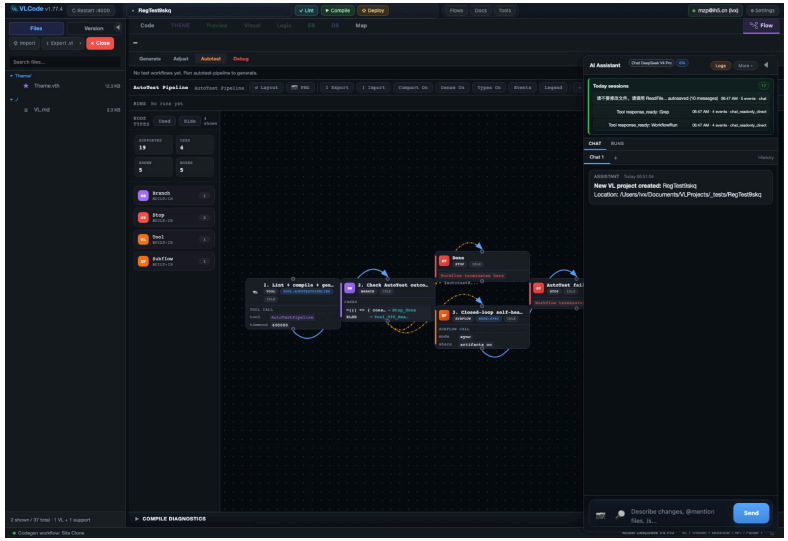
Incremental template:

```
Use add-service to add a DealService to CustomerCRM
with listByStage and advanceStage methods.
```

PITFALL

Writing a vague goal like "build a CRM" forces the AI to guess entities and pages, causing rework. State entities, pages, and acceptance criteria up front and name the workflow.

Post-Generation Checklist



Logs and execution details after a run

CORE IDEA

Don't ship straight after generation — self-check in a fixed order: first confirm the artifacts are right, then run the validation chain (Lint → Compile → Preview). This checklist turns "it looks generated" into "it actually works".

OPERATIONS

1. In the left file tree, confirm Apps/, Sections/, Services/, and Database/ all contain the expected files with nothing dropped mid-run.
2. Use Visual to inspect page layout, Logic to inspect click/event behavior, and Data to inspect table schema.
3. Run lintPjt for a low-cost static check to catch obvious problems first.
4. Once Lint is clean, run parsevl (Compile) for platform validation.
5. Open Preview at /route/<path> and actually click through the main flow to confirm it works.

EXAMPLE / EXERCISE

Post-generation self-check order:

```
1. File tree: Apps/Sections/Services/Database all present
2. Visual / Logic / Data: pages, events, schema correct
3. lintPjt: low-cost static check
4. parsevl: platform compile
5. Preview /route/task-board: click through it
```

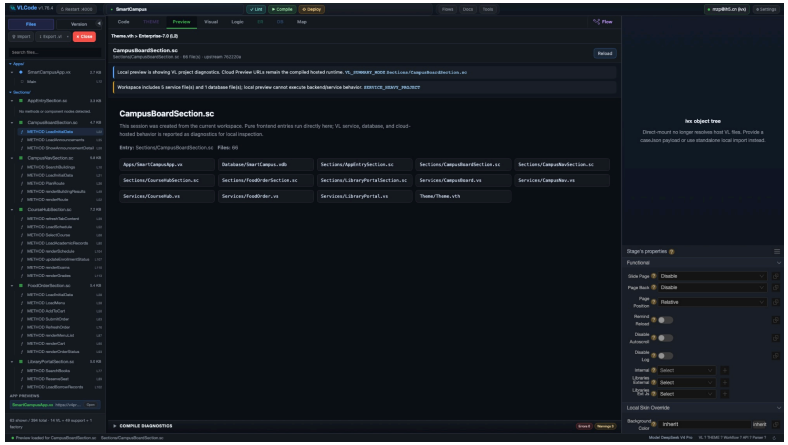
If complete-files reports files dropped mid-run, recover them before continuing.

PITFALL

Skipping Lint and compiling first, or treating a clean Compile as done without clicking through Preview. All three chain steps matter, and Preview is the real use-case validation.

CHECKPOINT

Debug Conversation Templates



A logic-panel failure context for debugging practice

CORE IDEA

Debugging must be grounded in the real error, not guesses. VLCode's root-cause-first principle says: pull the actual error stack from Detail Log, paste it into AI Chat, and name a repair workflow for a minimal fix — instead of letting the AI rewrite broadly.

OPERATIONS

1. Locate the failing node and the real error message in Detail Log (the right-side run ledger).
2. Copy the verbatim error into AI Chat along with the failing file/Section.
3. Name debug-single-file for one file, debug-multi-file when several are entangled, or compile-fix for a compile error.
4. Explicitly ask for the smallest change and the root cause, not a broad rewrite.
5. If the same error repeats twice, return to the source and ProjectMeta to find the root cause instead of blindly patching again.

EXAMPLE / EXERCISE

Debug template:

```
Detail Log error: advanceStage in TaskService.vs references an undefined method nextStage.
Use debug-single-file to fix TaskService.vs at the root cause with the smallest change;
do not touch other files.
```

Anti-pattern (don't write this): "This project has bugs, rewrite the whole thing."

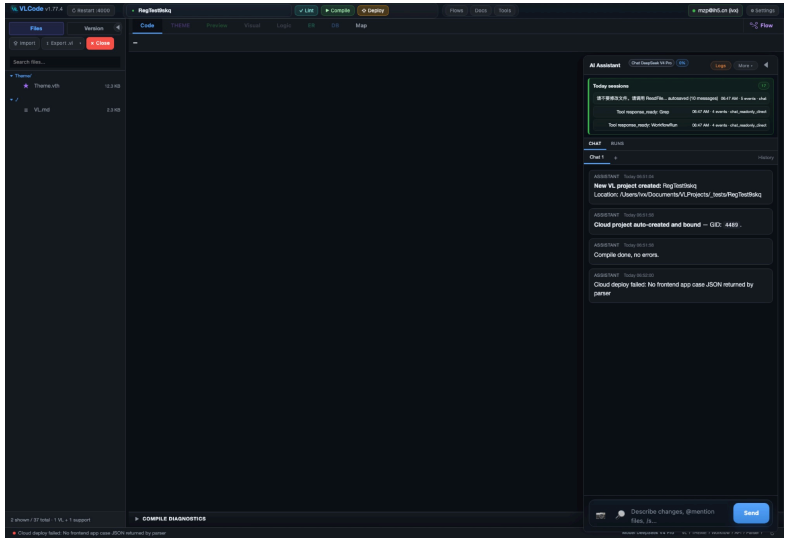
PITFALL

Asking the AI to "just fix it" without pasting the real error, or blindly re-patching the same error more than twice. If the second attempt still fails, stop and return to ProjectMeta to find the root cause.

CHECKPOINT

You can take one real error from Detail Log and write a debug prompt that names debug-single-file or compile-fix and demands a minimal change.

Pre-Release Checklist



Deploy operation after validation

CORE IDEA

The pre-release checklist makes the deliverable verifiable and portable. VLCode is local-first and cloud deployment is only an optional target — so before release you pass the full validation chain and confirm the error-prone points: binding, file names, and theme.

OPERATIONS

1. Confirm the whole chain passes: lintPjt clean, parsevl compiled, Preview /route/<path> verified working.
2. Check that all file names are English PascalCase with no Chinese/non-ASCII characters.
3. If going to cloud, use the GID binding button in the sidebar header to confirm you bound a cloud project GID you have permission for.
4. Confirm the Theme (.vth) tokens match what components use, and that service method references and data fields aren't broken or drifting.
5. Cloud deployment is only an optional target — the local source directory is the deliverable, so exporting it hands over with zero lock-in.

EXAMPLE / EXERCISE

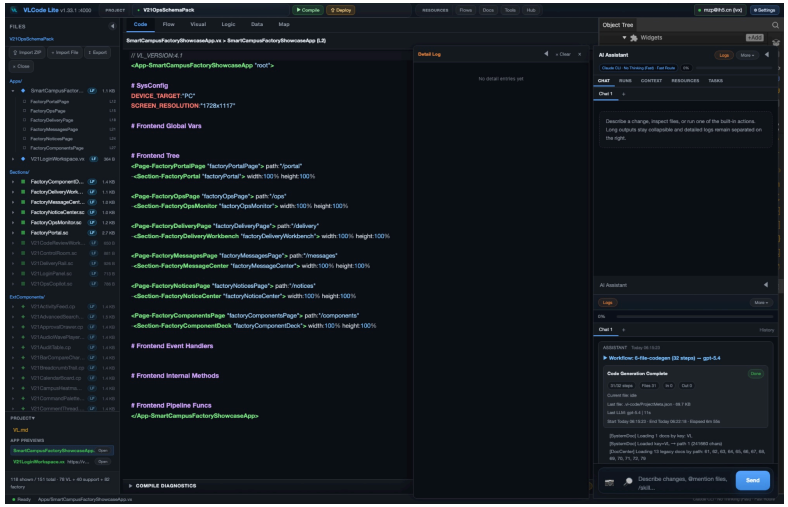
Pre-release checklist:

```
[ ] lintPjt error-free
[ ] parsevl compiles
[ ] Preview /route/... main flow verified
[ ] all file names English PascalCase
[ ] GID bound to a project you have permission for (sidebar header button)
[ ] Theme tokens match components; service/data references intact
```

PITFALL

Deploying against an old GID you have no permission for, or keeping Chinese file names until release surfaces the error. Confirm GID permission before binding and fix naming during generation.

Provider And Model Cheat Sheet



AI Chat with detailed run context

CORE IDEA

VLCode picks Provider and model in Settings → LLM. It supports eight providers and distinguishes the Chat model from the Workflow model — they can differ, but the provider/company usually stays consistent. The cost-sensitive defaults are DeepSeek V4 Pro (chat) + Kimi 2.5 (workflow + fanout).

OPERATIONS

1. Open Settings → LLM and pick a provider from anthropic-api / openai-api / deepseek-api / kimi-api / glm-api / gemini-api / minimax-api / qwen-api.
2. Set chatModelPolicy to auto (upgrades the model for long/complex prompts or multi-round tool use), fast, or strong.
3. Tune workflowMaxConcurrency (default 5), reasoningLevel (basic/standard/high), thinkingEnabled, and set workflowFanoutProvider as needed.
4. Enter keys in Settings (masked), or use the SecretRoot paste-in vault (paste KEY=VALUE; it auto-detects sk-ant-/sk-proj-/AIza... and shows a masked inventory).
5. The bottom status bar shows the current LLM provider/model — cross-check it anytime.

EXAMPLE / EXERCISE

Cost-sensitive default setup:

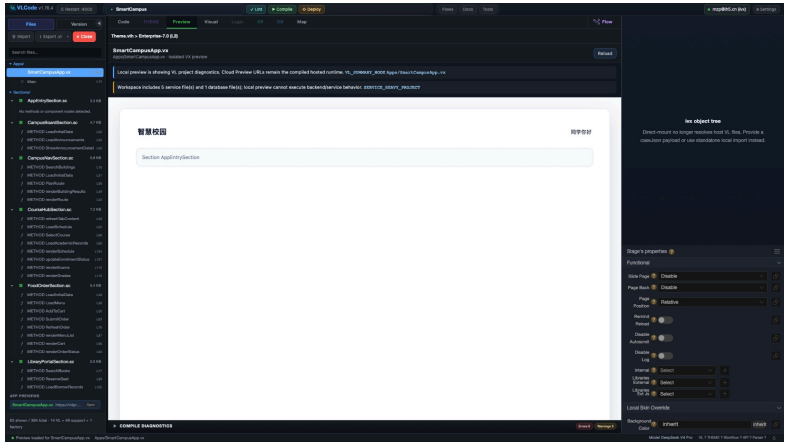
```
Provider: deepseek-api (chat) / kimi-api (workflow + fanout)
Chat model: DeepSeek V4 Pro
Workflow model: Kimi 2.5
chatModelPolicy: auto
workflowMaxConcurrency: 5
```

SecretRoot paste:

```
ANTHROPIC_API_KEY=sk-ant-...
```

PITFALL

Things To Avoid



A visual object tree failure context for debugging practice

CORE IDEA

This section gathers the most common traps that violate VL's core principles. Most stem from three rules: root-cause-first (no fallback code), local-first (cloud is optional), and fixed ports with English naming. Remembering these anti-patterns is easier than memorizing the rules.

OPERATIONS

1. Don't make Compile (parsevl) your first trial loop — run lintPjt first; the cheap check comes first.
2. Don't hide real errors with fallback / error-swallowing / silent-success code — surface real errors to Detail Log and the user.
3. Don't run generation workflows on an existing project — make incremental changes with add-page / feature-adjust, etc., to avoid broad rewrites.
4. Don't use Chinese/non-ASCII file names, and don't place projects outside VLProjects.
5. Don't silently fall back to a random port — ports are fixed at 4000-4010; and never commit secrets into project files.

EXAMPLE / EXERCISE

Typical anti-patterns and fixes:

```
✗ parsevl repeatedly as the trial loop  → ✓ lintPjt first
✗ try/catch that swallows errors as ok   → ✓ surface errors to Detail Log
✗ meta-direct on an existing project     → ✓ incremental add-page
✗ file name "TaskBoard中文.sc"          → ✓ TaskBoard.sc
✗ swap to a random port when busy        → ✓ clear the old process, keep 4000
```

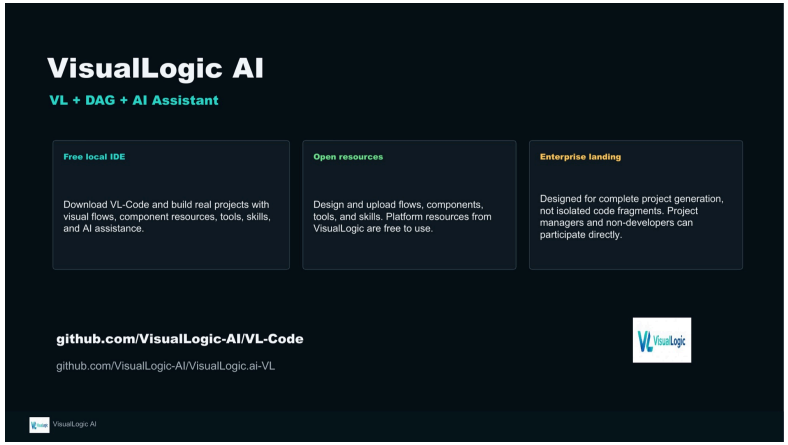
PITFALL

Blindly patching the same error more than twice or fixing by broad rewrite "to get lucky". If the second attempt still fails, stop and find the root cause in the source and ProjectMeta.

CHECKPOINT

You can list at least four "don'ts" and map each to root-cause-first, local-first, fixed-port, or English-naming.

Learning Path: Week One



DAG harness: final delivery

CORE IDEA

Week one's goal is to run one end-to-end loop: install, configure the model, generate a small project, pass the validation chain, and do one minimal fix. This week isn't about complexity — it's about building the mental model and closing the tool loop.

OPERATIONS

1. Days 1-2: Install VLCode (desktop .dmg, or from source with npm run dev on :4000) and configure provider and model in Settings → LLM.
2. Day 3: Create a PascalCase project under ~/Documents/VLProjects/ and get familiar with the mode-tab bar (left to right): Flow | Code | THEME | Preview | Visual | Logic | ER | DB | Data | Map (THEME/Preview/ER/DB/Data are disabled until the relevant artifacts exist).
3. Day 4: In AI Chat, use meta-direct-codegen to generate a small project (e.g. a TeamPM task board).
4. Day 5: Pass the validation chain lintPjt → parsevl → Preview and read the run ledger in Detail Log.
5. Days 6-7: Introduce and fix a compile error with compile-fix as a minimal fix to feel root-cause-first.

EXAMPLE / EXERCISE

Week-one practice goal:

```
In AI Chat:
  Use meta-direct-codegen to generate a TeamPM task board
Then: lintPjt → parsevl → open Preview /route/board
Finally: break one thing on purpose, fix it with compile-fix
```

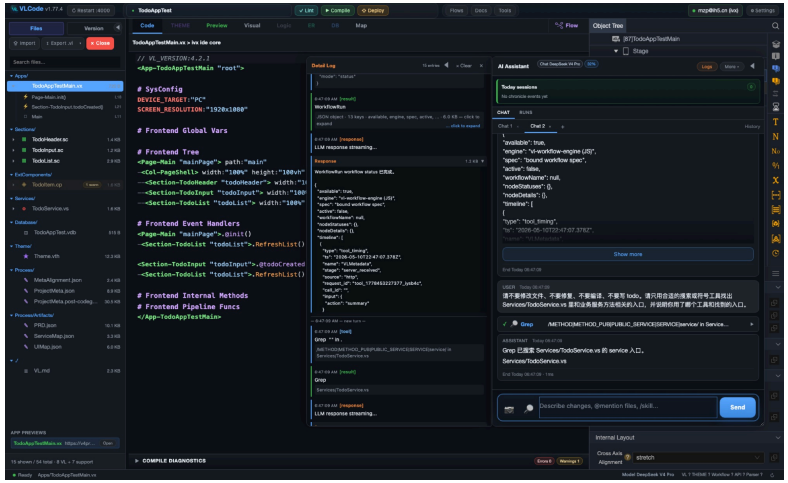
PITFALL

Reaching for a large workflow like enterprise-grand-cascade-codegen in week one — the chain is long and errors are hard to localize. Close the loop first with meta-direct-codegen.

CHECKPOINT

By the weekend you can independently generate a small project, pass lintPjt/parsevl/Preview, and use compile-fix to repair an error you introduced yourself.

Learning Path: First Month



Final IDE state after project creation

CORE IDEA

The first month builds on week one's loop: master incremental workflows, design-to-code, professional repair, and understand the two-product model with VL Agent (connected by .vflow, shipped through the Resource Center). The goal is to independently deliver and maintain a real small system.

OPERATIONS

1. Week 2: Practice incremental workflows — add-page, add-service, feature-adjust, data-adjust, batch-adjust, theme-customize — iterating steadily on an existing project.
2. Week 3: Practice design-to-code — smart-design-fullstack-codegen / html-first-vl-codegen — and try factory-turbo-codegen to reuse components for speed.
3. Week 4: Practice repair and stability — complete-files, debug-multi-file, heal-from-meta — and understand ProjectMeta's role in repair.
4. Throughout: upload a .vflow to the Flow panel and pick a category, understanding that .vflow is the connector VL Agent ships to VLCode through the Resource Center.
5. Run the pre-release checklist and hand over the local source directory as the deliverable (cloud deployment optional).

EXAMPLE / EXERCISE

First-month milestones:

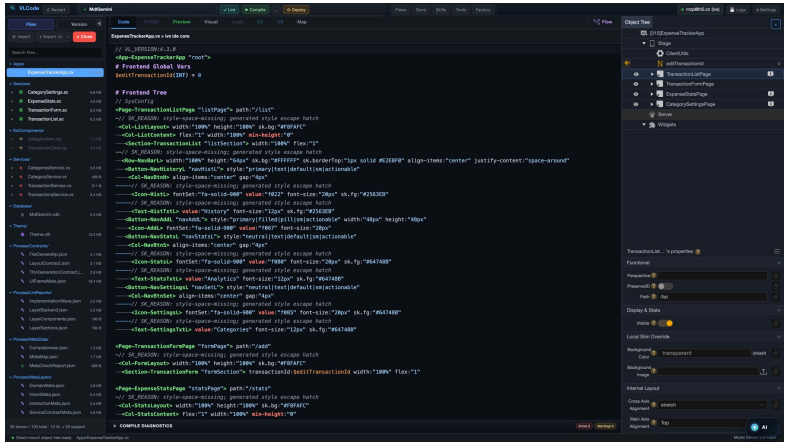
```
Week 2: add-page "My Tasks" and add-service a stats service to TeamPM
Week 3: turn one design into a CustomerCRM page via smart-design-fullstack-codegen
Week 4: introduce a multi-file error, fix with debug-multi-file + heal-from-meta
Month end: run the pre-release checklist, export ~/Documents/VLProjects/TeamPM
```

PITFALL

Spending the whole month only regenerating with generation workflows and never practicing incremental or repair work, so real maintenance forces a full rewrite. Get fluent with the incremental and repair workflows.

CHECKPOINT

Glossary



VLCode shell with workspace, editor, and assistant panels

CORE IDEA

This glossary collects the core terms that recur in VLCode, covering the seven VL source file types, the .vflow bundle, key concepts, and interface regions. Pinning these down first makes both the docs and your prompts flow more smoothly.

OPERATIONS

1. The seven VL source file types (compiled by the VL parser): .vx=App entry, .sc=Section (page/region), .cp=Component (pure UI, no business logic), .wc=WebComponent (project-local native web extension, under ExtComponents/), .vs=Service (backend logic), .vdb=Database (tables/schema), .vth=Theme (visual tokens).
2. .vflow is NOT a VL source file — it is a workflow / agent-app bundle (a zip of workflow JSON + manifest + support docs/prompts), imported via bundle tooling and never compiled by the VL parser. It is the connector VL Agent ships to VLCode through the Resource Center.
3. ProjectMeta: the machine-readable map of the project; root keys are apps / sections / components / services / dataSchema / theme / dependencyGraph (plus fileManifest, valueDomains), stored at .vl-code/ProjectMeta.json ($schema VL-ProjectMeta/3.0, Metadata Spec 3.2). Generation and repair revolve around it.
4. VL_VERSION: the declaration that must be the first line of every VL source file, written with // (not #), current value 4.3.1, with nothing before it.
5. Validation chain: Lint (low cost, first) → Compile (parsevl, heavier) → Preview (/route/<path>).
6. Interface trio: AI Chat (primary interaction + workflow console), Detail Log (full run ledger), Status Bar (current step / provider / compile status / ports).

EXAMPLE / EXERCISE

Quick reference:

```
seven sources: .vx App | .sc Section | .cp Component(pure UI) | .wc WebComponent(native ext)
               .vs Service | .vdb Database | .vth Theme
not a source:  .vflow = workflow / agent-app bundle (imported, not compiled)
ProjectMeta roots = apps/sections/components/services/dataSchema/theme/dependencyGraph
VL_VERSION = first line of every file   Validation chain = Lint → Compile → Preview
```

First line of every VL file, e.g.: `// VL_VERSION:4.3.1`